MERK User Guide

Open Source IRC Client




Author	Daniel Hetrick
License	GPL 3 (explained)
Source Code	https://github.com/nutjob-laboratories/merk
Current Release	https://github.com/nutjob-laboratories/merk/releases/latest
Development Builds	https://latest.merk.chat
Last modified	Saturday, May 9, 2026

Summary

IRC (**Internet Relay Chat**) is a text-based chat system for instant messaging. IRC is designed for group communication in discussion forums, called *channels*, but also allows one-on-one communication via private messages… .

Internet Relay Chat is implemented as an application layer protocol to facilitate communication in the form of text. The chat process works on a client–server networking model. Users connect, using a client—which may be a web app, a standalone desktop program, or embedded into part of a larger program—to an IRC server, which may be part of a larger IRC network. Examples of ways used to connect include the programs Mibbit, KiwiIRC, mIRC and the paid service IRCCloud.

From the Wikipedia entry on IRC, at https://en.wikipedia.org/wiki/IRC

MERK is a free and open source Internet Relay Chat client for Windows, Linux, and macOS. It uses a "multiple document interface", in which the application works as a parent window that contains other windows for servers, channels, and private chats. The popular Windows shareware client mIRC is an example of another IRC client that uses a multiple document interface.

MERK is written in the Python programming language, using the PyQt library for the graphical interface and the Twisted library for networking. MERK also comes bundled with four other open source libraries:

- **qt5reactor**, for getting PyQt and Twisted to work together
- **pyspellchecker**, which provides the spellchecking mechanism
- **emoji**, providing support for emoji shortcodes
- **pike**, providing support for plugins

MERK has a scripting engine allowing most functionality to be automated. The core concept of the scripting engine is the **context**: a context is a window, either a channel, private chat, or server window, that the commands executed are intended to interact with. Scripts can be executed on connection, or executed from text input. MERK comes with a script editor with features to make writing scripts easy and fun. MERK is extremely configurable, with over 350 settings that can be tweaked. MERK supports both emojis and ASCIImojis; they can be inserted into messages with shortcodes, and displayed in MERK.

MERK's plugin system allows users to extend MERK's capabilities with Python, no matter what platform they're running. Plugins can be created entirely within MERK, using the plugin manager and the Python editor, with support for Python syntax highlighting and auto-indentation. Plugins can react to over 40 different IRC or MERK events.

As IRC is a text-based protocol, MERK features a rich text display, which can be easily configured. MERK supports the display and entry of mIRC colors[1], which can optionally be stripped from messages.

1 https://en.wikichip.org/wiki/irc/colors

Running MERK

MERK comes in four different versions: the Python version, which uses the Python interpreter to run MERK, a PyInstaller version of MERK for Windows (which doesn't require a Python interpreter), and a PyInstaller version of MERK for Linux (which also doesn't require a Python interpreter), and a Flatpak version of MERK for Linux. All versions behave exactly the same way, and have only minor differences.

PyInstaller Version for Windows

Download the Windows version of MERK, and unzip the archive to anywhere you'd like. The archive contains a folder named **`lib`**, the **`merk.exe`** executable, the **`CHANGELOG`**, and **`README.html`**. Just double click **`merk.exe`**, to run MERK. That's it! There's also an installer for MERK. Download the Windows installer version and unzip the archive to wherever you'd like. Double click on **`setup.exe`**, which will guide you through installing MERK on your computer. MERK can be installed wherever you'd like, and can either be installed for a single user, or for all users on your computer.

PyInstaller Version for Linux

Download the Linux version of MERK, and unzip the archive to anywhere you'd like. The archive contains a folder named **`lib`**, the **`merk`** executable, the **`CHANGELOG`**, and **`README.html`**. Just double click **`merk`**, to run MERK. That's it!

Flatpack Version for Linux

Download the MERK Flatpak. Navigate to wherever you downloaded the Flatpak to, and enter this command:

```
flatpak install --user <filename>
```

Replace **`<filename>`** with the name of the downloaded file. For example, to install the latest development of MERK, as downloaded from the MERK repository, you would type:

```
flatpak install --user merk-latest.flatpak
```

Running MERK on macOS

There isn't a PyInstaller binary for MERK (yet), but you can still run MERK via the command-line. First, download and extract the MERK Python source code. Second, install Python 3.13 with HomeBrew:

```
brew install python@3.13
```

Now, navigate to wherever you extracted the MERK source code to, and execute the following commands:

```
python3.13 -m venv .venv
source .venv/bin/activate
pip install --upgrade pip
pip install pyqt5 Twisted
```

If you want to connect to IRC servers via SSL/TLS, run this command:

```
pip install pyOpenSSL service_identity
```

Once all of that is installed, you can run MERK with Python:

```
python merk.py
```

Please note that in the future, running MERK this way will require you to “initialize” the virtual environment before you can run MERK. For example, if you extracted MERK to a folder named **`Users/your_name/merk`**, you could initialize MERK’s virtual environment and start up MERK with:

```
source Users/your_name/merk/.venv/bin/activate && python Users/your_name/merk/merk.py
```

Running MERK with Python

MERK requires several libraries to be installed in order to run: **`Python 3.9+`**, **`PyQt`**, **`Twisted`**, and if you'd like to connect to servers via SSL/TLS, **`PyOpenSSL`** and **`service_identity`**. If you're running MERK on Windows, you may also need **`pywin32`**. All of these libraries can be installed easily with PIP, the Python package installer. To install the base requirements, open a terminal, and enter:

```
pip install PyQt Twisted PyOpenSSL service_identity
```

If you're using MERK on Windows, you may also have to enter:

```
pip install pywin32
```

Once all the requirements are installed, unzip the downloaded archive of MERK, use the terminal to navigate to the directory you unzipped MERK to, and type:

```
python merk.py
```

Updating MERK

As MERK stores all its configuration files separate from the executable/installation, updating MERK to the latest version is easy. Configuration files are stored separately from program files, and MERK will automatically update any configuration files as needed.

Windows

The Windows version of MERK comes in two different distributions: the **ZIP file version**, which is a ZIP file containing **`merk.exe`**, the MERK executable, and the **`lib`** folder, which contains files needed for **`merk.exe`** to run, and the **installer version**, which is a "setup" installer.

ZIP File Version

You can do this one of two ways: either delete **`merk.exe`** and the **`lib`** folder, wherever you extracted them do, and unzip the new version of MERK in the same folder; or unzip the new version of MERK in the same directory and overwrite all files.

Installer Version

This version of MERK is even easier to update. Just download the installer of the newer version of MERK, unzip **`setup.exe`**, and double click on it. You don't have to uninstall the older version, the new version will overwrite the old one.

Linux

ZIP File Version

You can do this one of two ways: either delete **`merk`** and the **`lib`** folder, wherever you extracted them do, and unzip the new version of MERK in the same folder; or unzip the new version of MERK in the same directory and overwrite all files.

Flatpak Version

Download the new MERK Flatpak, navigate your console to wherever you downloaded it to, and enter:

```
flatpak install --user <filename>
```

Replace **`<filename>`** with the name of the downloaded file.

Python and macOS Source Version

Overwrite the old MERK source code with the new source code.

Command-Line Arguments and Options

```
usage: python merk.py [--ssl] [-p PASSWORD] [-c CHANNEL[:KEY]]
                      [-C SERVER:PORT[:PASSWORD]] [-S SERVER:PORT[:PASSWORD]]
                      [-n NICKNAME] [-u USERNAME] [-a NICKNAME] [-r REALNAME]
                      [-h] [-d] [-x] [-t] [-R] [-o] [-f] [-s FILE] [-P] [-E]
                      [--config-name NAME] [--config-directory DIRECTORY]
                      [--config-local] [--scripts-directory DIRECTORY]
                      [--user-file FILE] [--config-file FILE] [--reset]
                      [--reset-user] [--reset-all] [--uninstall [FILE]]
                      [--install FILE] [-Q NAME] [-D] [-L]
                      [SERVER] [PORT]

  SERVER                Server to connect to
  PORT                  Server port to connect to (6667)
  --ssl, --tls          Use SSL/TLS to connect to IRC
  -p PASSWORD, --password PASSWORD
                        Use server password to connect
  -c CHANNEL[:KEY], --channel CHANNEL[:KEY]
                        Join channel on connection
  -C SERVER:PORT[:PASSWORD], --connect SERVER:PORT[:PASSWORD]
                        Connect to server via TCP/IP
  -S SERVER:PORT[:PASSWORD], --connectssl SERVER:PORT[:PASSWORD]
                        Connect to server via SSL/TLS
  -n NICKNAME, --nickname NICKNAME
                        Use this nickname to connect
  -u USERNAME, --username USERNAME
                        Use this username to connect
  -a NICKNAME, --alternate NICKNAME
                        Use this alternate nickname to connect
  -r REALNAME, --realname REALNAME
                        Use this realname to connect
  -h, --help            Show help and usage information
  -d, --donotsave       Do not save new user settings
  -x, --donotexecute    Do not execute connection script
  -t, --reconnect       Reconnect to servers on disconnection
  -R, --run             Don't ask for connection information on start
  -o, --on-top          Application window always on top
  -f, --full-screen     Application window displays full screen
  -s FILE, --script FILE
                        Use a file as a connection script
  -P, --disable-plugins
                        Disables plugins
  -E, --enable-plugins  Enables plugins
  --config-name NAME    Name of the configuration file directory (default:
                        .merk)
  --config-directory DIRECTORY
                        Location to store configuration files
  --config-local        Store configuration files in install directory
  --scripts-directory DIRECTORY
                        Location to look for script files
  --user-file FILE      File to use for user data
  --config-file FILE    File to use for configuration data
  --reset               Reset configuration file to default values
  --reset-user          Reset user file to default values
  --reset-all           Reset all configuration files to default values
  --uninstall [FILE]    Deletes an installed plugin
  --install FILE        Install plugin ZIP or Python module
  -Q NAME, --qtstyle NAME
                        Set Qt widget style (default: Windows)
  -D, --dark            Run in dark mode
  -L, --light           Run in light mode
```

Using the Command-Line Interface

MERK's command-line options allow users to do many things on startup. All of these uses are completely optional, and never have to be used. Most command-line options feature a long version (for example **`--donotexecute`**) and a shorter version (**`-x`**, which does the same thing). All versions of MERK (Python, Windows, Linux, and macOS) all use the same command-line interface, so all of these examples will work on any version of MERK.

If user settings are in place (that is, the default nickname, username, etc), command-line options can be used to connect to one or more IRC servers automatically on startup. For example, to automatically connect to the DALnet IRC network, you can use:

```
merk.exe us.dal.net 6687
```

This will automatically connect to DALnet, executing any connection script previously set up with MERK. To prevent the connection script from executing, try:

```
merk --donotexecute us.dal.net 6667
```

Multiple servers can be connected to, as well, though the method is a little different. Use the **`-C`** option to connect to normal IRC servers, and the **`-S`** option to connect to IRC servers via SSL/TLS. In the next example, we're going to connect to the Libera network via SSL/TLS, and DALnet:

```
python merk.py -S irc.libera.chat:6697 -C us.dal.net:6667
```

If you want MERK to skip asking for a server to connect to on startup, use the **`--run`** option:

```
merk.exe --run
```

Resetting MERK to Default Settings

If your installation of MERK becomes unusable or for any other reason, you can reset MERK back to default settings with the following command-line option:

```
python merk.py --reset
```

If you are running MERK with the PyInstaller executable, use:

```
merk.exe --reset
```

To reset all user settings, use the **`--reset-user`** command-line option. This will remove all user settings, including your nickname, alternate username, username, realname, connection history, and any connection scripts:

```
python merk.py --reset-user
```

To reset *all* settings, and return MERK configuration files to their default state with all default settings, use **`--reset-all`**. This will reset both your user file, **`user.json`**, and the settings file, **`settings.json`**, to all default values:

```
merk --reset-all
```

If a plugin has a fatal error in it that is causing MERK to crash or otherwise not function, there are two options. The first is to disable plugins from the command-line:

```
python merk.py --disable-plugins
```

This will allow the user to start up MERK without crashing. MERK will not load in any plugins until plugins are re-enabled; the plugin manager will also be disabled. This option will always save to the configuration file, even if **`--donotsave`** is enabled; plugins must be re-enabled manually, either with **`--enable-plugins`** or in settings. The other option is to delete all installed plugins:

```
merk.exe --uninstall all
```

This will delete any and all installed plugins and their icons! This will* not *move the files to the "recycle bin", and the files will be lost! If you know which plugin is causing MERK to crash, it might be easier to navigate to the **`plugins`** directory and delete the file manually.

The easiest and fastest way to “reset” MERK to all default settings is to delete MERK’s settings directory (by default, **`.merk`** in your home directory). The next time MERK starts up, it will recreate all necessary default configuration files and styles, with default settings. *This will delete all your stored logs!* If you copy your **`logs`** directory before you delete the settings directory, start up MERK (to recreate needed settings files), copy the files in the copied logs directory to the new **`logs`** directory, and restart MERK, MERK will use your old logs.

Making MERK Portable

MERK can be configured to run from a USB thumb drive, storing all its configuration files and logs on the drive. To do this, we use the **`--config-local`** command-line flag.

Making MERK Portable on Windows

First, extract the Windows binary distribution to the USB thumb drive. Then, open Notepad (or any other basic text editor) and enter this into a new file:

```
merk.exe --config-local
```

Save this file to wherever you extracted MERK to with the name **`merk.bat`**. Whenever you want to run MERK, double click on **`merk.bat`** instead of **`merk.exe`**. MERK will save all its configuration files and logs into a directory named **`.merk`** in the same directory you extracted MERK to.

Making MERK Portable on Linux

First, extract the Linux binary distribution to the USB thumb drive. Then, open a console in the directory you extracted MERK to, and type the following commands:

```
echo merk --config-local > merk.sh
chmod +x merk.sh
```

Whenever you want to run MERK, double click or execute **`merk.sh`** instead of **`merk`**. MERK will save all its configuration files and logs into a directory named **`.merk`** in the same directory you extracted MERK to.

Directories and Configuration Files

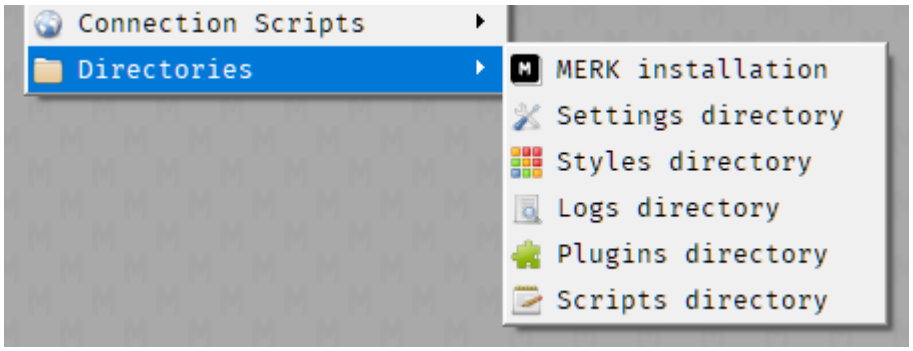
MERK stores all its settings in a directory it creates in the user's home directory[2], named **`.merk`**. Inside this directory, MERK creates:

- **`logs`**. This directory is where MERK stores channel and private chat logs.
- **`styles`**. This directory is where MERK stores text style files, and the palette used for dark mode.
- **`scripts`**. This directory is where MERK stores, and first looks for, scripts. This is the default directory chosen when running a script via the server window toolbar, input menu, or right click menus, or when saving a script in the editor.
- **`plugins`**. This directory is where MERK looks for and loads any installed plugins from. To install a plugin, simply copy the plugin's Python file into this directory, use the plugin manager, or use the **`--install`** command-line option. This is the default directory chosen when saving Python code in the Python editor.
- **`settings.json`**. This file is where MERK stores and loads application settings.
- **`user.json`**. This file is where MERK stores user information, such as the chosen nickname, username, and the like, as well as the application's connection history and any connection scripts.

When using the **`/script`** command, if a full filename is not provided, MERK will look for the script in several locations, in order:

1. The **`scripts`** directory.
2. The settings directory (by default, **`.merk`** in the user's home directory).
3. The application's installation directory (only if this option is turned on in the settings; it is turned off by default).

First, MERK will attempt to find the script using the provided filename, and if the script is still not found, it will append the default file extension (which is **`.merk`**) to the filename and search again. This same pattern is used with the **`/edit`** and **`insert`** commands.

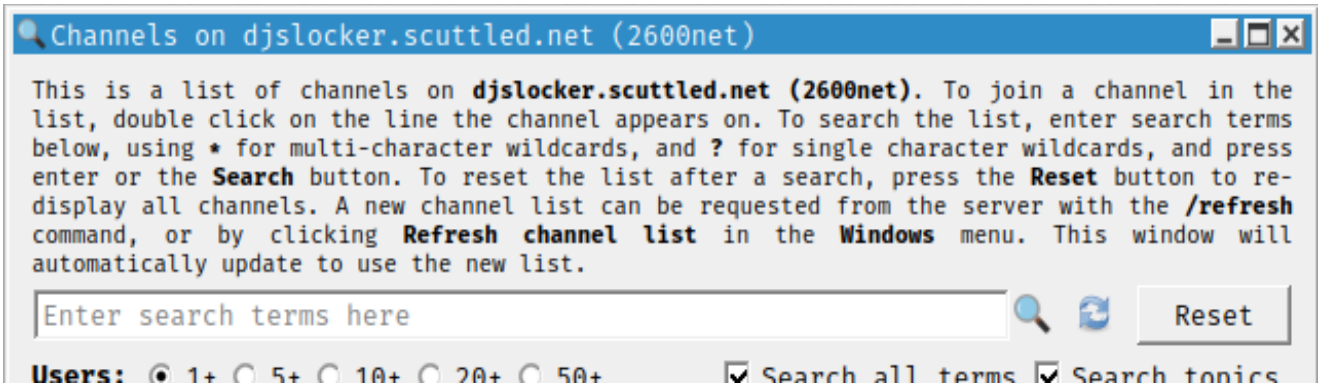


These folders can be opened in your default file manager from the client by clicking on the appropriate entry in the "Directories" sub-menu, near the bottom of the "Tools" menu.

2 *On Windows and macOS, this will be in the **`Users`** directory under the user's username. On Linux, or other UNIX-like environments, this will be the standard user's home directory.*

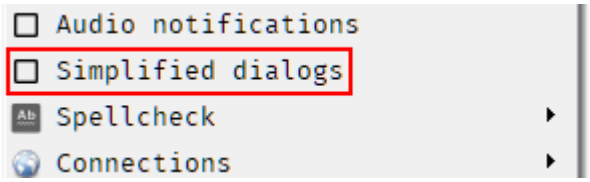
New User Help

Many dialogs feature text explaining how the dialog or the settings in it work.

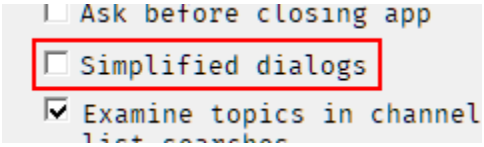


The explanation text from the channel list dialog.

While new users of MERK may find these helpful, experienced users may not want to see them. To hide the help text on dialogs, turn on "Simplified dialogs" in the settings menu or the settings dialog:



The "Simplified dialogs" option in the "Settings" menu. Click this entry to simplified dialogs on and hide the help text.

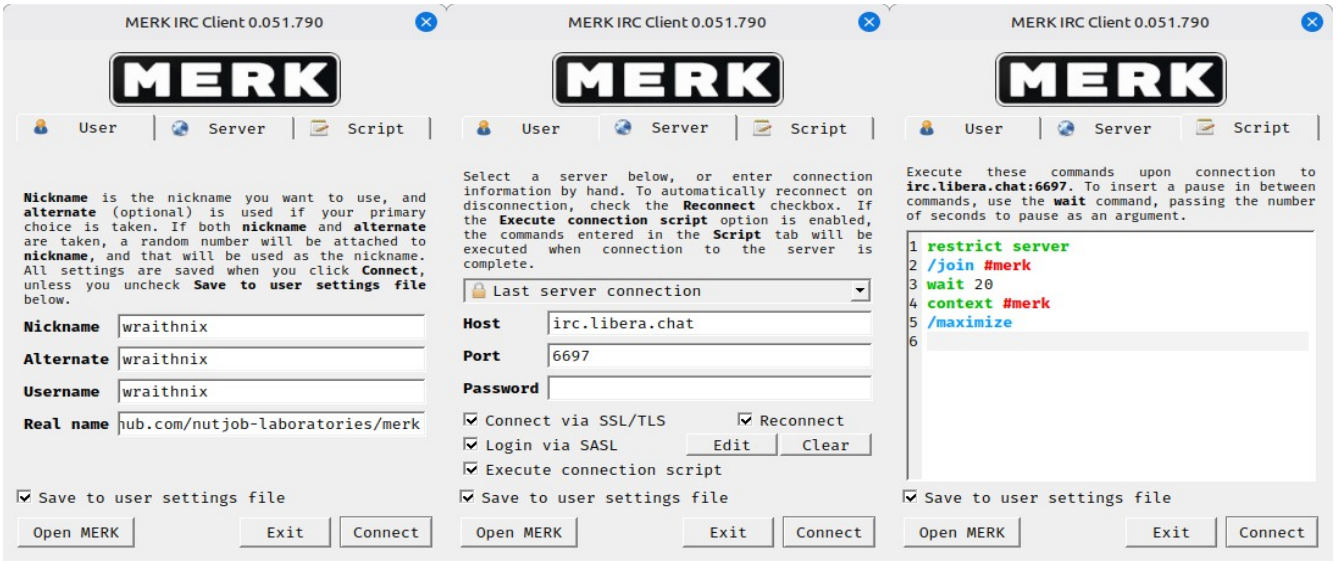


The "Simplified dialogs" option on the first page of the "Settings" dialog.

"Simplified dialogs" are turned off by default, showing the help text on dialogs every time a dialog is opened.

Connection Dialog

When you first run MERK, a connection dialog is displayed, allowing you to connect to an IRC server. The dialog has three tabs: **User**, **Server**, and **Script**.

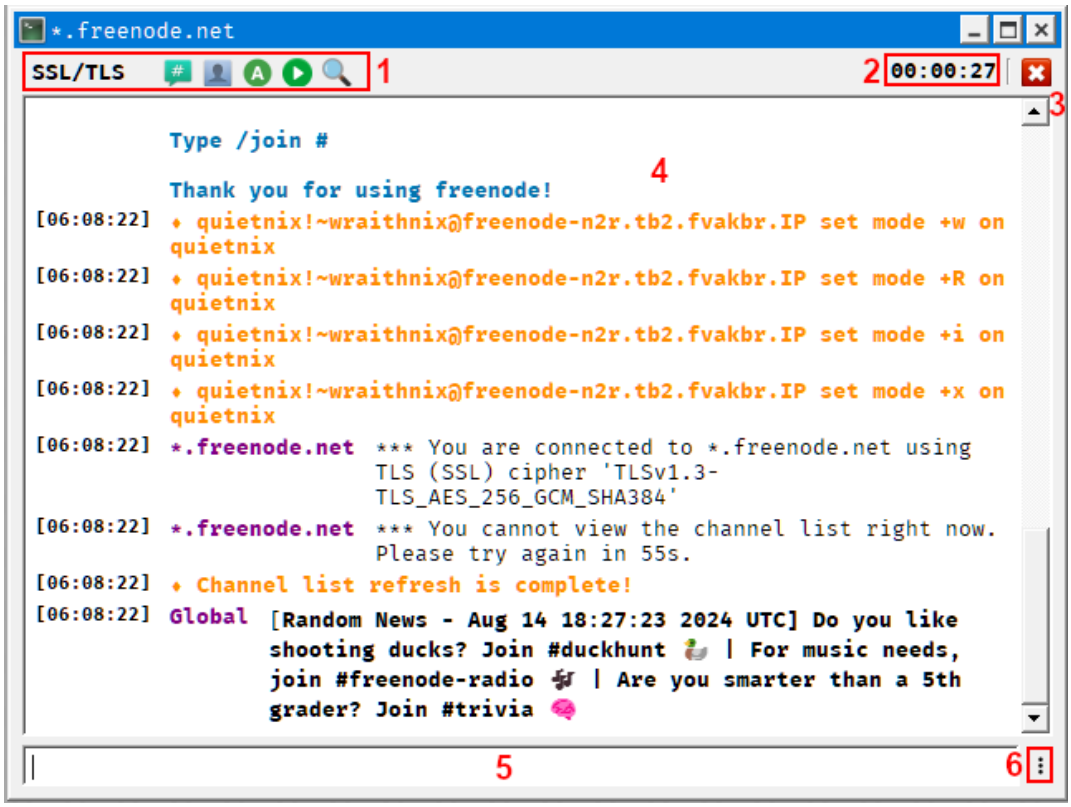


- **User** is where the user enters their various user information for connection. **Nickname** is the nickname you'd like to use, and **Alternate** (optional) will be used if that nickname is already taken; if both are taken, a random number is generated and added to **Nickname**.
- **Server** is where the user enters the IRC server to connect to. By default, the last server MERK connected to is pre-entered. **Host** is the IP address or hostname of the desired IRC server, and **Port** is the server's port. **Password** is the server's password, if one is required; leave blank if one is not required. Click **Connect via SSL/TLS** to connect via SSL/TLS, and click **Reconnect** if MERK should automatically try to reconnect on disconnection. Click **Login via SASL** to login to server services while connecting; when this is clicked, the user will be prompted for a username and password, which will be saved to the user configuration file. Click **Edit** to edit the current server's SASL username and password. Click **Clear** to clear the current SASL username and password. Click **Execute connection script** to execute the connection script on the next tab as soon as MERK connects to the server.
- **Script** has a small script editor for editing the script MERK will execute upon connection to the server. The syntax highlighting settings can be edited with the settings dialog. If **Execute connection script** is unchecked, this script will not be executed.
- **Save to user settings files**, if checked, will save any information entered into the dialog to **`user.json`**, and will be loaded automatically the next time MERK is started up.

Click **Connect** to connect to the IRC server using the entered information, or **Exit** to close MERK. To start MERK without connecting to a server, click **Open MERK**.

Server Windows

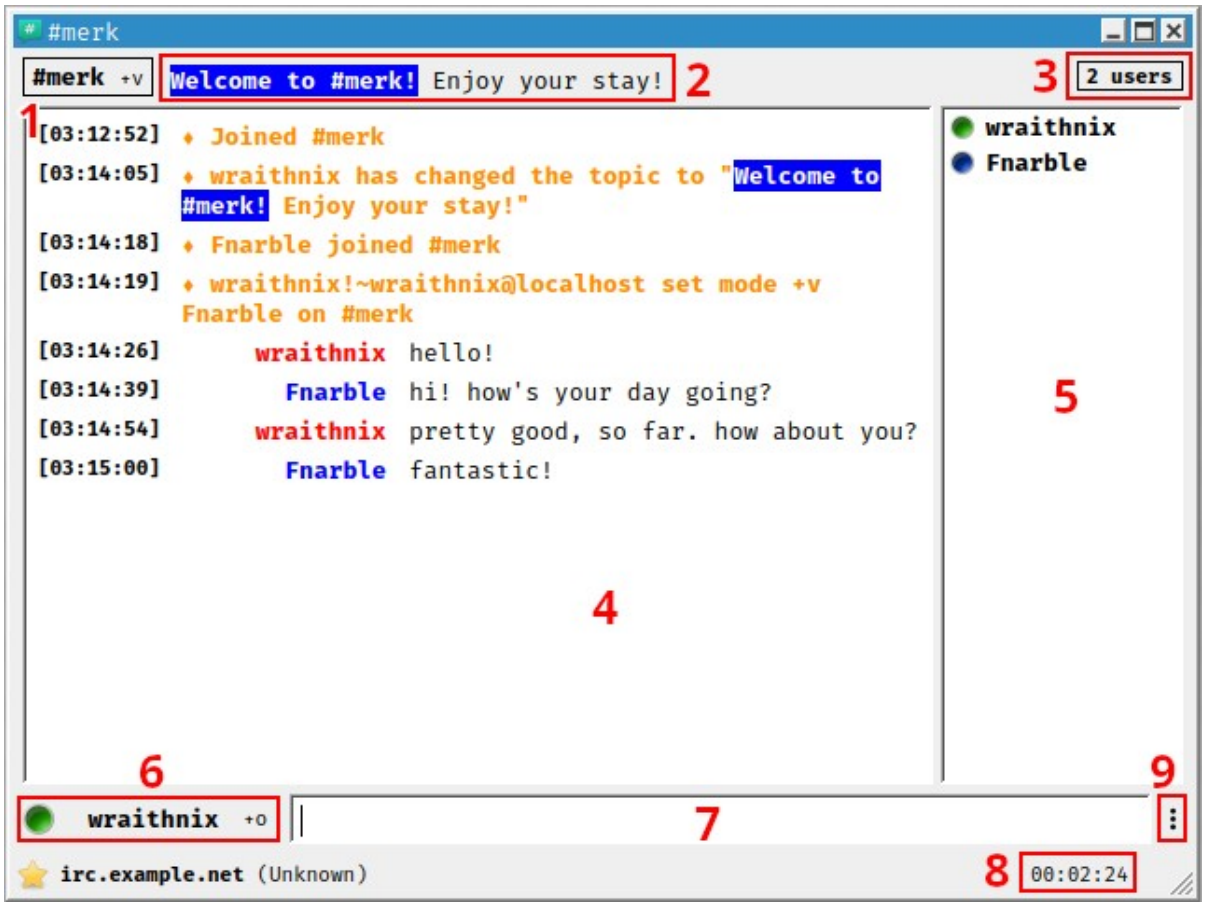
Server windows are the first windows you will see in MERK; they appear as soon as connection to an IRC server begins. Server windows behave differently from channels or private chat windows: closing a server window does not disconnect from the server, it only hides the window. To view a hidden server window, click on its entry in the "Windows" menu or the system tray menu. You cannot chat to other users from a server window without using commands like **`/msg`** or **`/notice`**.



1. **Toolbar**. Buttons that perform basic actions; some on the IRC server, such as joining a channel, changing your nickname, and setting your away status, and others on the client, like selecting a script to run, and opening the channel list dialog. Clicking the button labeled "TCP/IP" (for normal connections) or "SSL/TLS" (for encrypted connections) will show a menu with information about the server.
2. **Connection uptime**. This displays how long MERK has been connected to the server.
3. **Disconnect**. Pressing this button issues a **`QUIT`** command and quickly disconnects from the IRC server.
4. **Display**. Displays any messages from the server, as well as notices, outgoing private messages, and the like.
5. **Text input widget**. Type commands in here, and press "enter" to execute them.
6. **Input menu**. Clicking on this brings up a menu that allows you to change various input options. This button is present on channel and private message windows, too.

Channel Windows

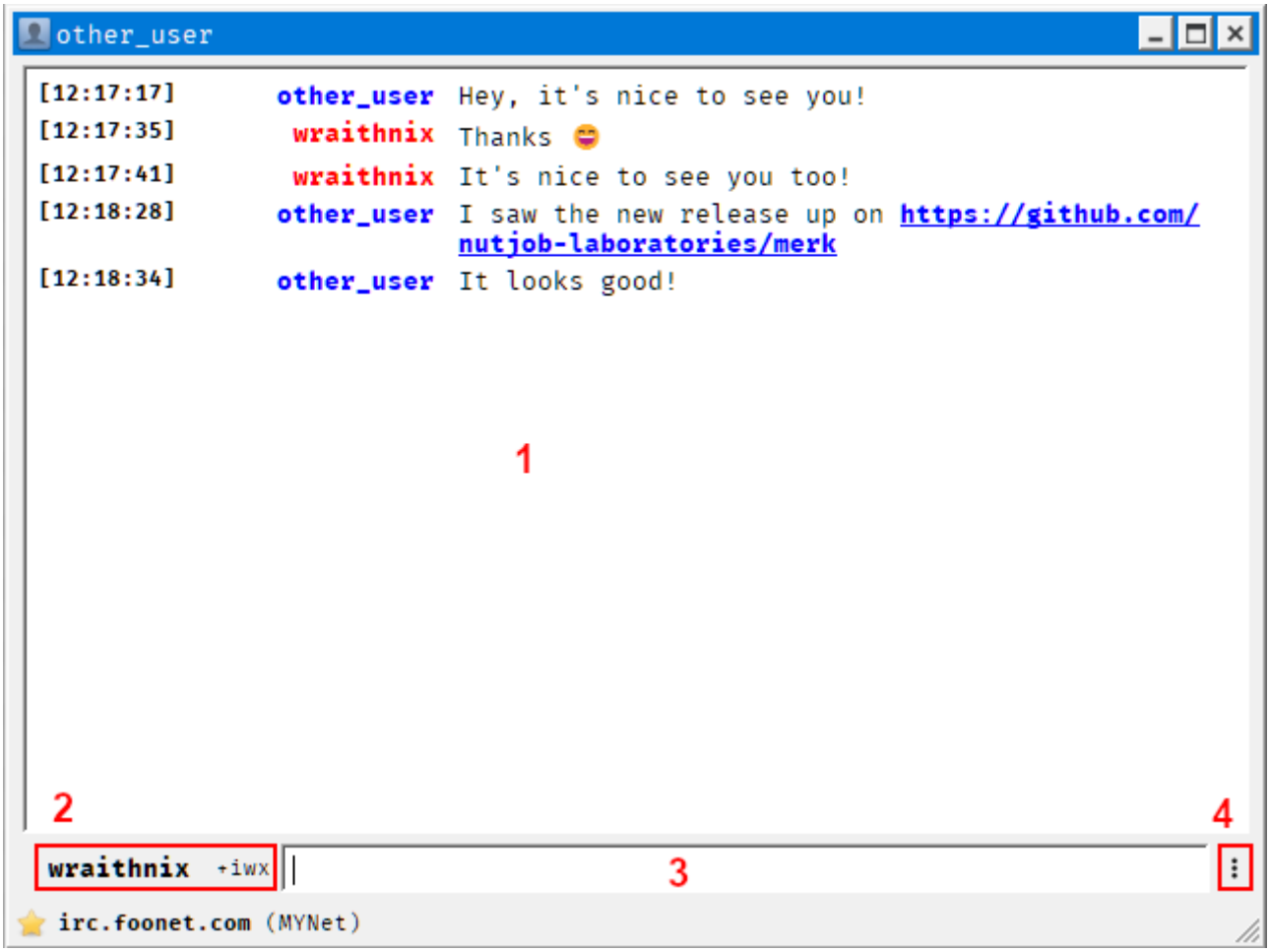
Closing a channel window leaves the channel.



1. **Name and mode display**. Here, the channel name and any channel modes are displayed. Right click on the display to either see a description of known active channel modes, or if you're an operator or privileged user, options to set channel modes; a list of users on the ban list is also provided as a submenu. If no channel modes are set, or the channel ban list is empty, this right click menu will not be available for non-privileged users.
2. **Topic**. The channels topic is displayed here. Click on the topic to edit it, and press enter to send any changes to the server. Click here for more information on editing channel topics.
3. **User count**. How many users are currently in the channel
4. **Chat display**. Channel chat, as well as system messages, are displayed here.
5. **User list**. A list of users in the channel is displayed here. Privileged users have special icons next to their name (green for channel operators, blue for voiced users, etc.), and normal users do not. Nicknames are displayed in bold if the users are present, and in normal weight and italics if they are away. Right click on a user for various options, like sending a CTCP message or performing channel administration duties. Double click a user's name to open a private chat window.
6. **Nickname**. This displays the currently used nickname, and any user modes set.
7. **Text input widget**. Type your chat or commands here, and press "enter" to send them to the server or client.
8. **Uptime**. This displays how long the client has been connected to the channel.
9. **Input menu**. Clicking on this brings up a menu that allows you to change various input options.

Private Chat Windows

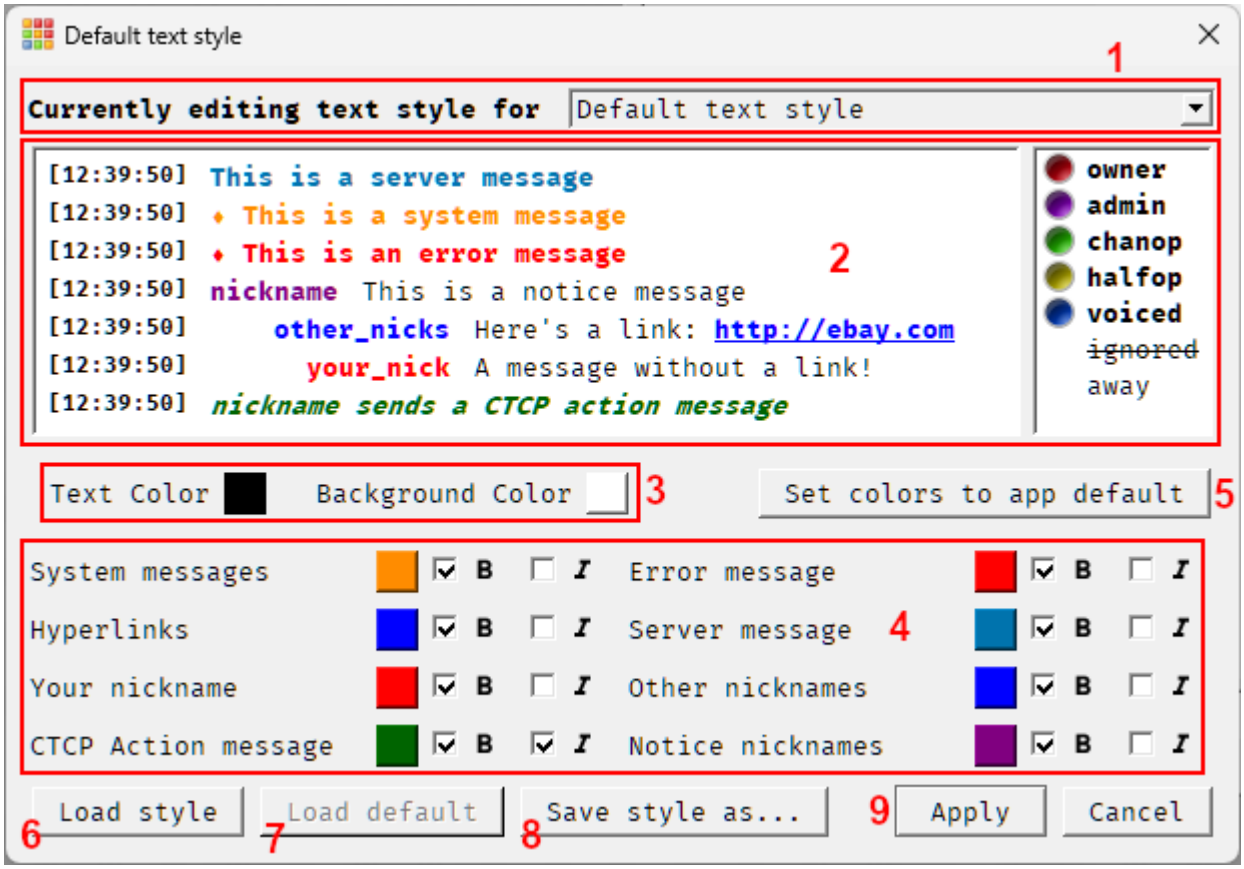
Closing a private chat window does not leave the chat, or block the sender; it only closes the window.



1. **Chat display**. Private chat is displayed here, as well as system messages.
2. **Nickname**. This displays the currently used nickname, and any user modes set.
3. **Text input widget**. Type your chat or commands here, and press "enter" to send them to the server or the client.
4. **Input menu**. Clicking on this brings up a menu that allows you to change various input options.

Style Editor

MERK has a text style engine that colors and styles all chat text, and can be edited by users with the style editor.



1. **Style selector**. Select which text style to edit. When launched from the "Tools" menu, this will default to editing the default text style; when launched from context menus or the **`/style`** command, the text style of the window that launched the style editor will be selected. The text style of any window currently in use can be selected. Both “dark” and “light” mode each have their own default text style.
2. **Display**. This is what the text style will look like in the client. Any changes in color or style will be displayed here instantly. If editing the default text style, or the text style of a channel, an example user list is shown as how it will appear in the client. The example user list is not shown when editing the text style of server windows or private chats.
3. **Background and foreground color**. Set the color of the text and the background color here.
4. **Message styles**. Change the color and style of individual message types here.
5. **Set colors to app default**. Set all colors to the default style that ships with MERK. This is different from the "default" style that is applied to server windows and any windows that do not have a style.
6. **Load style**. Here, you can open any existing MERK style file for editing. Colors and styles will be loaded and displayed.
7. **Load default**. This will load in whatever style the user has set as the default text style. This button is disabled when editing the default text style.
8. **Save style as…**. Save this style to a file. It will not be applied, only saved to a file.
9. **Apply** and **Cancel**. Applying this style automatically saves it. Pressing the "cancel" button closes the dialog, and all changes are discarded.

How Text Styles Are Applied

All chat windows start by using the default style. All text styles actively in use can be edited with the "Style Editor", found in the "Tools" menu.

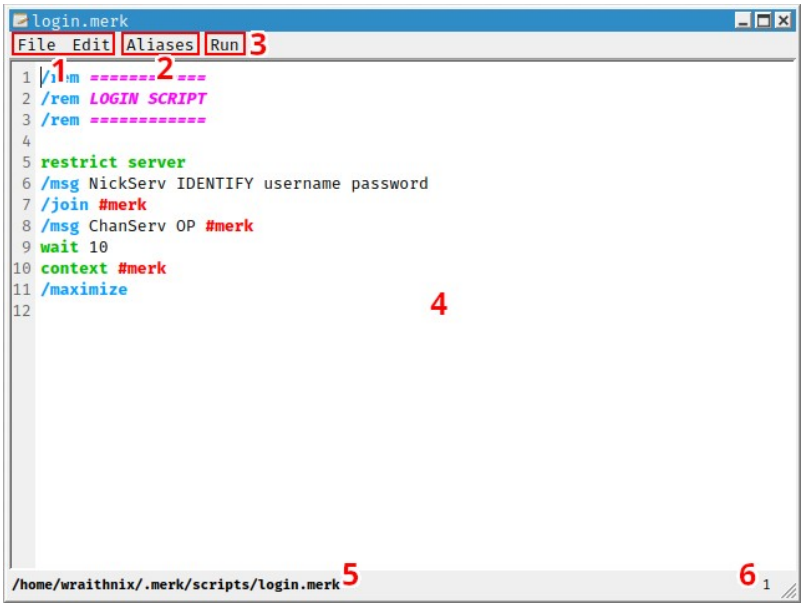
All chat windows can have their own styles which can be edited by selecting the "Style Editor" option from the "Tools" menu, or "Edit text style" in the input options menu, or the chat display right click menu. Styles for channel and private chat windows are saved with the IRC network of the channel or private chat in mind, so they will load no matter which server the client is connected to. For example, if the user has set a text style for the **#merk** channel on the EFnet network, it will load and be applied to the **#merk** channel window if the user is connected to **irc.underworld.no** on port 6667, **irc.choopa.net** on port 9999, or **irc.prison.net** on port 6667, as all of these servers are on the EFnet IRC network.

Server window text styles are specific to the server being connected to, regardless of what network the server is on. So, if the user has set a style for **irc.prison.net** on port 6667, the server window text style for that connection will be loaded. If the user connects to **irc.choopa.net** on port 9999, the default style will be loaded; even though both servers on the EFnet network, they are still different servers.

Please note that dark mode and regular mode have different default styles. When in one mode, the default style will *only* be used in that mode.

Script Editor

To launch the script editor, use the **`/edit`** command, or select “Script Editor” in the "Tools" menu.

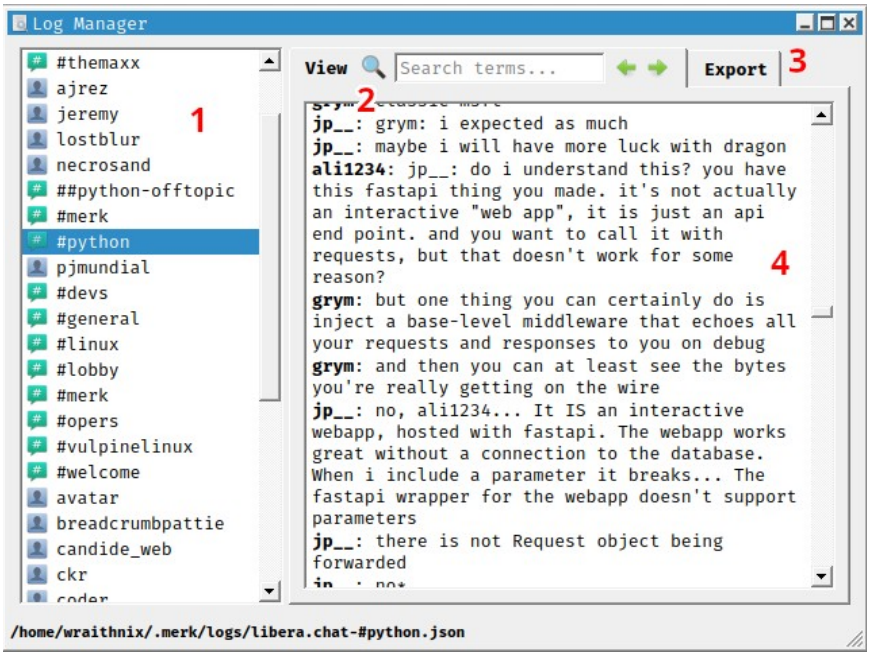


1. **File** and **Edit Menus**. All the normal selections of a text editor, like opening and saving files, cut and paste, find and replace, etc. Connection scripts can also be opened for editing, as well as created. Files can also be written directly into ZIP files, without the need for an archive utility.
2. **Aliases**. Insert built-in aliases into the script.
3. **Run**. Run the currently open script in any context/window available. The user also can run the script in all contexts/windows simultaneously. Scripts are executed with no arguments and no filename.
4. **Script display**. Features syntax highlighting and line numbers. Colors used for the display can be set in the settings dialog.
5. **Filename**. The currently open script's filename is displayed here.
6. **Line number**. The line number the cursor is currently on.

The colors and styles used for the syntax highlighting can be changed in the settings dialog. Comments, commands, channels, aliases, and script-only commands can be styled individually. These colors and styles will also be used in the "Script" tab of the connection dialog.

Log Manager

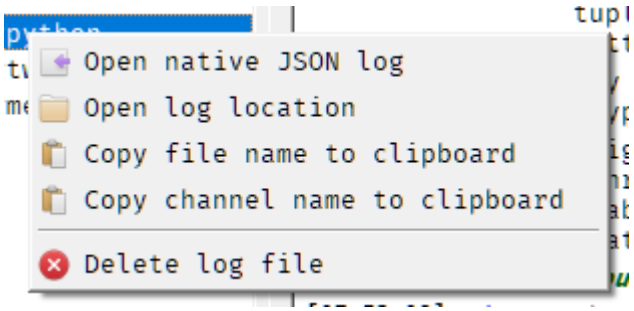
The log manager allows users to view, delete, and export MERK logs. It can be launched from the "Tools" menu.



1. **Logs**. A full list of all logs in MERK. Hover the mouse over the log name to see what IRC network the log is from. Click a log name to view information about the log, view the log, and use export options.
2. **View**. Here, a dump of the log can be searched. The dump does not contain any timestamps or user hostmasks, it only contains chat and whatever system or server messages are logged. Large logs may take a few seconds to render.
3. **Export**. Here, you can select whether to export to JSON, a custom format, or a “human readable” format, and an option to show epoch time or human readable time and dates. A view of what the log will look like when it’s exported is included.
4. **View**. The currently viewed log dump. This contains the entire log, and IRC colors and formatting are rendered. System messages are in bold, and CTCP ACTION messages are in bold and italics. URLs in chat, if clicked, will open in the default browser.

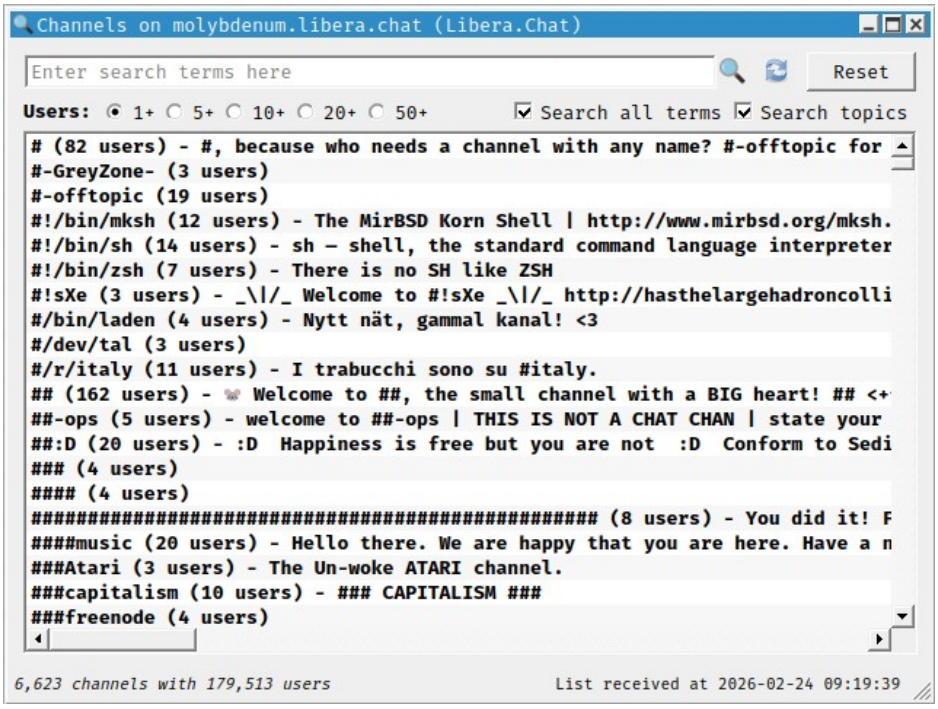
Additional log options can be seen by right-clicking on the log's name:

If the "Open native JSON log" option is selected, the JSON file that MERK uses will be opened in the default application that the user's operating system uses to open JSON files. There is additional information in the native log format that tells MERK how to render the log for viewing; to use MERK logs with other applications, the log should be exported to strip this information out.



Channel List

The server channel list can be brought up with the **`/list`** command, from the “Windows” menu, or from the server window toolbar.



The channel list, displaying the channels on the Libera.chat network

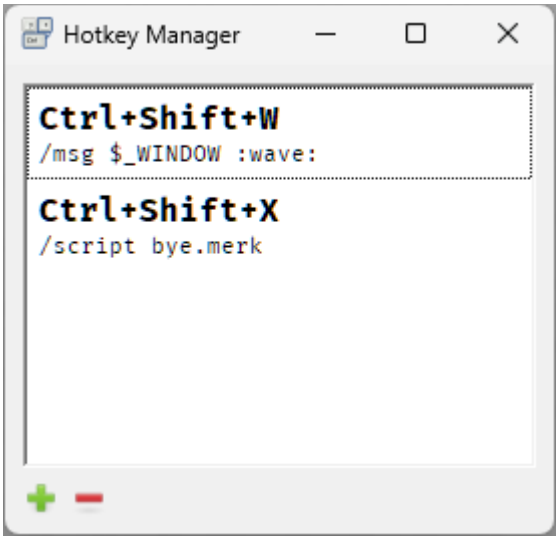
Here, you can see a list of all the channels on a server and the network the server belongs to. The list can be filtered by channel user count. To search the list, enter the search terms into the text input at the top of the window and press enter, or press the button directly to the right of the input. You can use **`*`** as a multi-character wildcard, or **`?`** as a single character wildcard. There are options to search all terms simultaneously, or to include search channel topics.

To view the channel list, use the **`/list`** command. To search the list with a command, pass the search terms as the arguments to the **`/list`** command. If the channel list is “old” (by default, older than 2 hours), the channel list will be automatically refreshed. The channel list can be manually refreshed with the **`/refresh`** command.

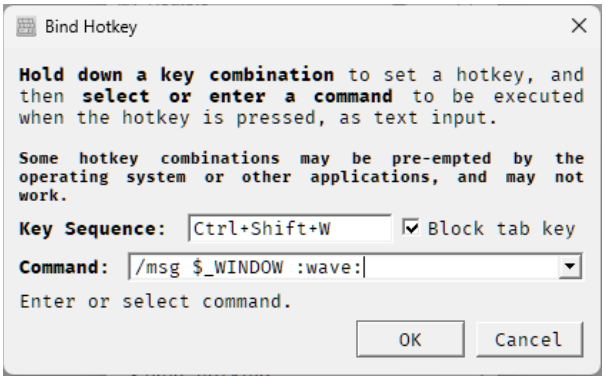
IRC colors and formatting codes are stripped out of all channel topics.

Hotkey Manager

The hotkey manager allows users to add, edit, or delete hotkeys that execute commands on whatever active server, channel, or private chat subwindow has focus when the hotkey is pressed. It can be launched from the “Tools” menu.



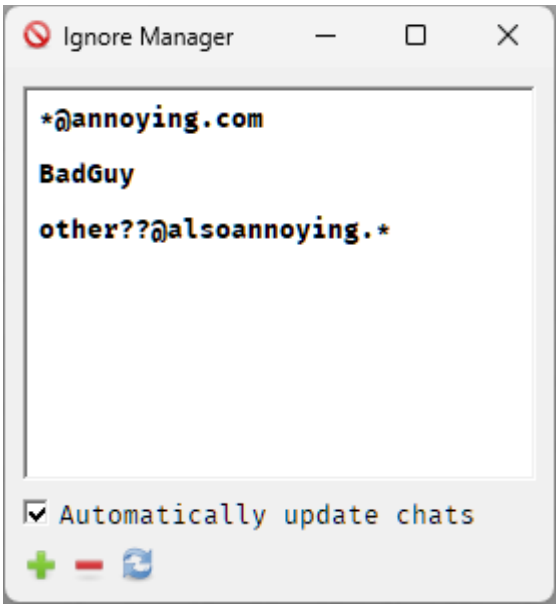
The hotkey manager, showing a few bound hotkeys. Hotkeys can be added by clicking the **+** button, or removed by selecting a hotkey in the list and clicking the **–** button. Edit a hotkey by double-clicking on that hotkey's entry. Hotkeys are immediately saved to the configuration file when they are created, edited, or deleted.



Commands executed by hotkeys are treated *exactly* as if the command was typed into a subwindow's text input widget and the return key was pressed.

Ignore Manager

The ignore manager allows users to add, edit, and remove entries in the ignore list, preventing chat from users with specific nicknames or hostmasks[3] from appearing in chat, or preventing them from sending the user private messages, notices, and the like. It can be launched from the “Tools” menu.



The ignore manager. Add nicknames or hostmasks to the ignore list by clicking the **+** button, and select an entry and click the **–** button to remove an entry. Double click on an entry to edit that entry. All chat windows will be automatically updated with the new ignore list immediately upon adding or removing an entry if "Automatically update chats" is checked. Click on the refresh button to update chat windows manually.

If a user is ignored, their chat will not appear in any chat window, and any private chat or notices will not open any chat windows. Any "ignored" chat is still received by MERK, and added to any chat logs, and will appear in chat windows if the ignore entry is removed.

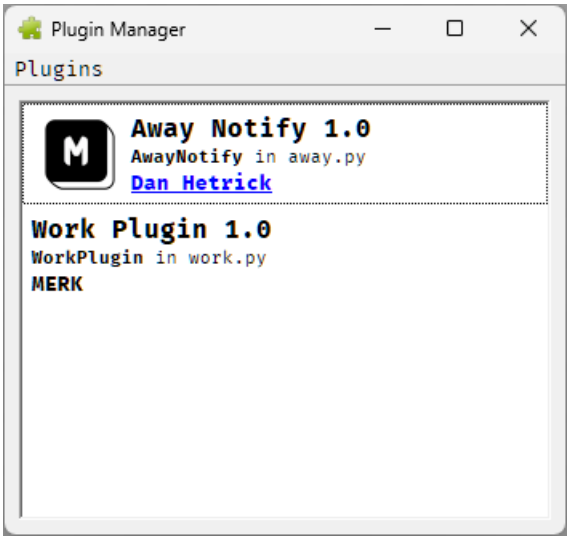
Updating chat windows may take a few seconds; chat must be re-rendered to hide or show any ignored or unignored entries. If "Automatically update chats" is checked, this update will be done *every time* an entry is added, edited, or removed; to manually update chats, click on the refresh button.

Entries can also be added and removed from the ignore list in the right click menu on channel user lists, as well as by the **`/ignore`** and **`/unignore`** commands.

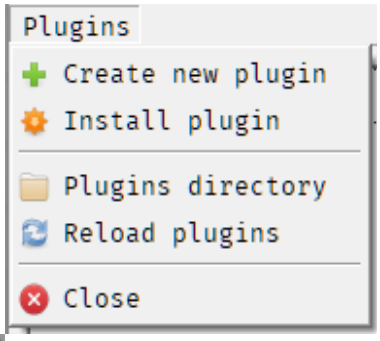
3 *A hostmask is a unique identifier of an IRC client connected to an IRC server.* Wikipedia

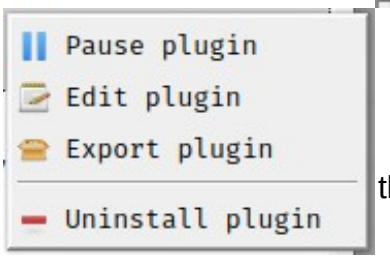
Plugin Manager

The plugin manager allows users to create, edit, delete, and install MERK plugins. A version of the script editor, modified for Python programming, is used to create and edit MERK plugins in-app. It can be launched from the “Tools” menu.



The plugin manager. To create, export, install, and uninstall plugins, click on the "Plugins" menu at the top of the window.

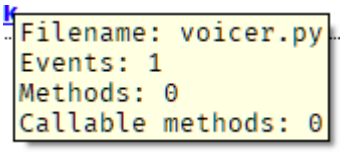




Right click on a plugin's entry for more options. Plugins can be “paused” to temporarily prevent them from being triggered by events.

For more information on how plugins work, and what they can do, please see ***Plugins and Plugin Development***. MERK doesn't come with any plugins, so unless you've installed or created any plugins, this list will be empty when you first install MERK.

Hover the mouse above an entry in the plugin manager to see information about that plugin. The filename, how many events the plugin responds to, how many methods the plugin has, and how many methods executable with the **`/call`** command the plugin has is displayed.

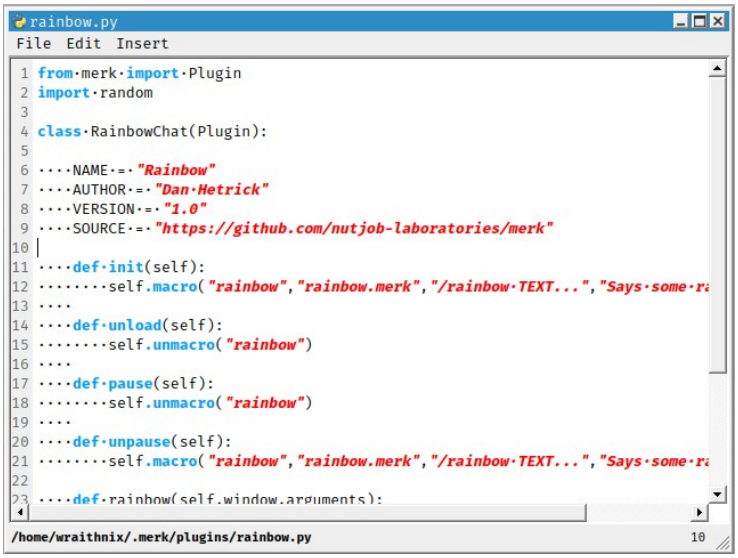


To install a plugin, simply copy the Python file for the plugin into the **`plugins`** directory, located in the same directory where MERK stores configuration files, logs, scripts, and styles, and restart MERK to load the plugin into memory. The other options are to select the "Install Plugin" option in the "Plugins" menu, which allows the user to select a Python file or zip archive to import into the plugins directory before automatically loading it, or to use the command-line option **`--install`**.

To view a plugin's console, double click on the plugin's entry, or select "Show console" from the plugin's right click menu. Not all plugins will have consoles. A plugin with an open console is marked with a small icon in the plugin list.

Selecting "Uninstall plugin" from a plugin's right click menu to uninstall a plugin will delete *any* plugin that is contained in the same file. The official documentation suggests that a Python file should contain only *one* plugin, but MERK does not enforce this. Check the tool tip of a plugin entry (which contains the filename that contains the plugin) before uninstalling it to make sure you don't unintentionally uninstall other plugins.

Selecting "Create new plugin" in the "Plugins" menu will open up the Python editor, a special version of the script editor for Python code, and load in a skeleton[4] plugin for the user to edit. The skeleton plugin has all plugin events pre-defined, and more information about writing plugins is in the comments of the skeleton.



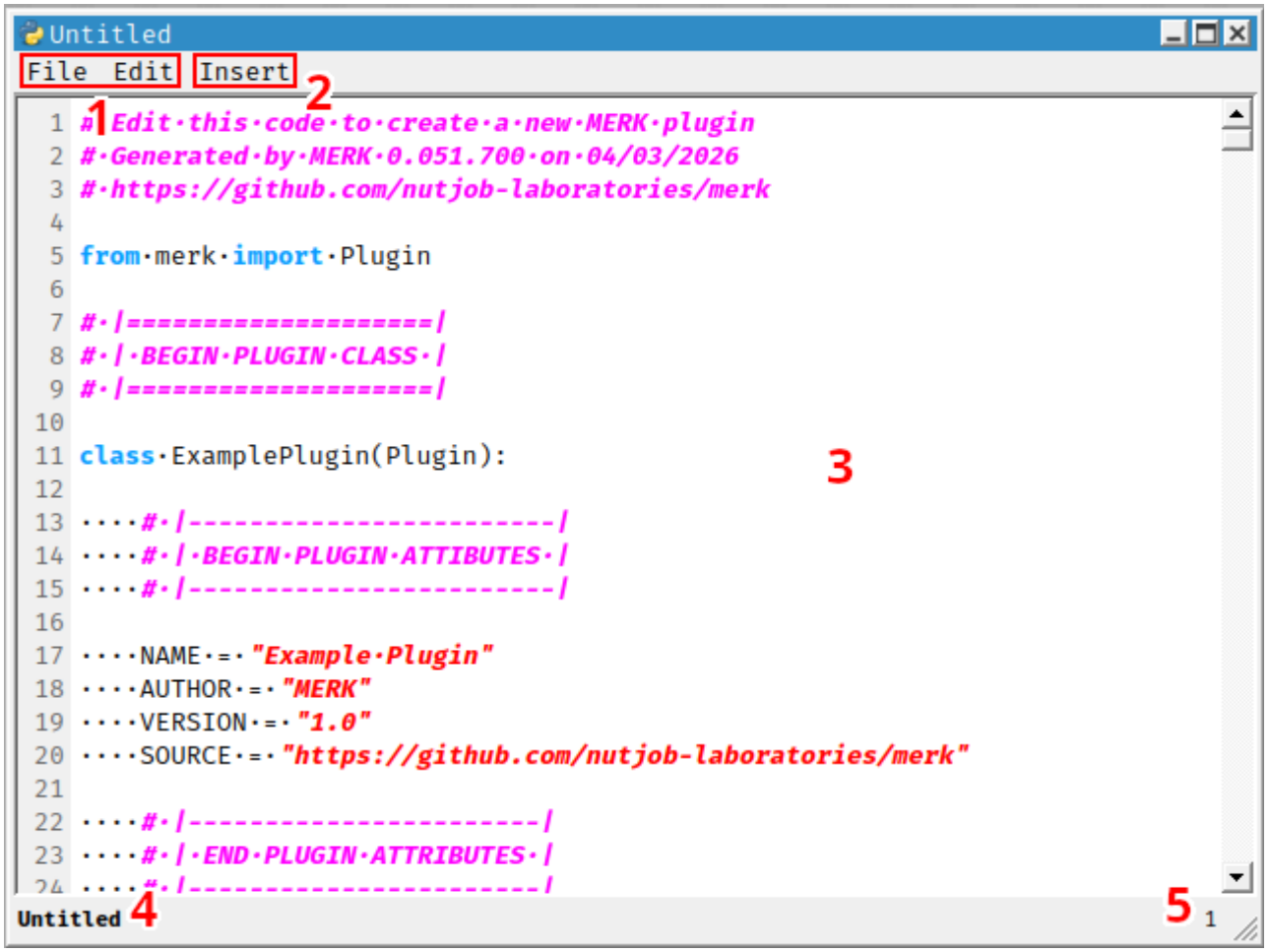
A plugin, open in the Python version of the script editor.
The "show whitespace" option is turned on, and word wrap is turned off.

The Python editor features syntax highlighting and auto-indent. The Python editor can be opened by choosing to create a plugin in the plugin manager, selecting "Edit plugin" from a plugin's right click menu, using the **`/window plugin`** command, or by opening an existing plugin file in the "Tools" menu. Python editor windows appear normally in the windowbar and the "Windows" menu, with the only difference being that the window's icon is the "plugin" icon rather than the normal "script" icon.

4 *A program skeleton may also be utilized as a template that reflects syntax and structures...* Wikipedia

Python Editor

The Python editor can be launched from the plugin manager to either edit an existing plugin, or create a new plugin. When opening a plugin from the "Tools" menu, that plugin will be opened in the Python editor. The Python editor can also be opened with the **`/python`** command.

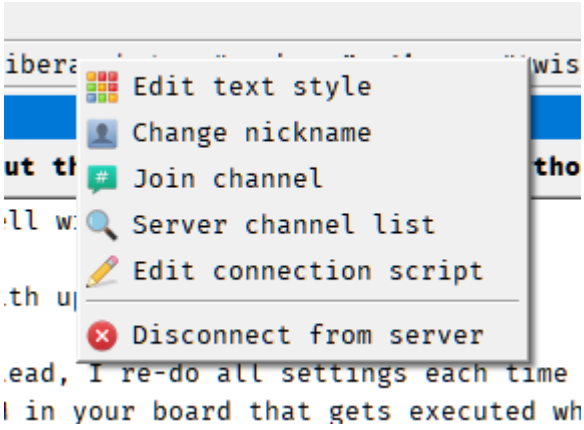


1. **File** and **Edit Menus**. All the normal selections of a text editor, like opening and saving files, cut and paste, find and replace, etc. Other options include auto-indentation, making whitespace visible, reloading plugins when the editor is closed, and turning word wrap on and off.
2. **Insert**. Each entry in this menu allows the user to insert Python code into a plugin, either a **`Plugin`** event method, or a method intended to be used with the **`/call`** command.
3. **Script display**. Features syntax highlighting and a line number display. Colors used for the display use the same settings used in the script editor, and can be set in the settings dialog. Special options for the Python editor include a “show whitespace” option, shown in the screenshot above.
4. **Filename**. The currently open Python file's name is displayed here.
5. **Line number**. The line number the cursor is currently on.

The Windowbar

The "windowbar" is a widget that is located by default on the top of the main window that displays a list of open subwindows. Clicking on a window's name switches "focus" to that window, bringing it to the front; double click the window name to bring the window to the front and maximize it. By default, the windowbar only shows channel windows, private chat windows, and script editor windows, but you can use the settings dialog (or the windowbar's right click menu) to show other window types. Think of the windowbar as a sort of task manager, only for windows in MERK.

Right click on an entry in the windowbar for more options. Each window type shows different options. Right click on the windowbar itself to change settings for the windowbar. Most things about the windowbar can be changed, including the order windows are shown, what windows are shown, whether icons are shown on entries, and more.

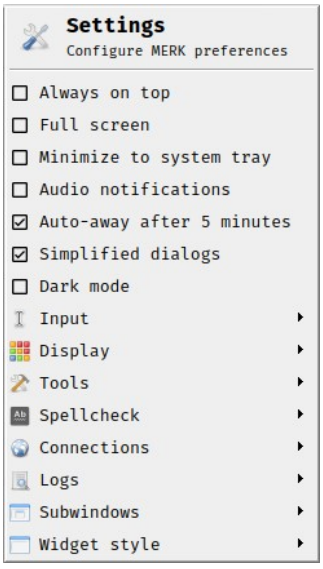


The windowbar right click menu for a server window.

Although immobile by default, the right click menu and the settings dialog can make the windowbar movable; the windowbar can float, or be "docked" at the bottom or the top of MERK's main window.

Settings

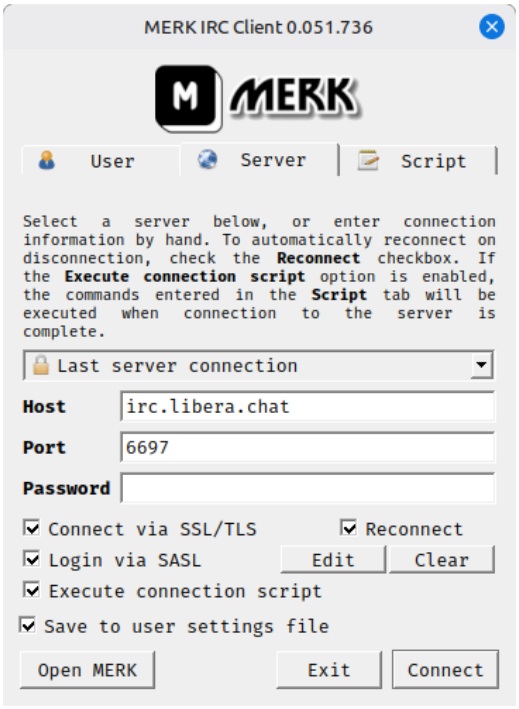
The settings dialog, available as the top entry in the "Settings" menu, allows users to change over 350 different settings, and apply almost all of them instantly. Settings are separated into sections, selectable with the menu on the left side of the dialog. Each section displays a number of options for various ways that MERK functions. To change settings, check, uncheck, or otherwise change the various options in the dialog; changes will not be applied until the "Apply" button is pressed. All changes are saved to the configuration file when applied. If a specific setting requires a restart of MERK, the user will be notified by the dialog when the option if changed, and an option to restart MERK will be offered.



The "Settings" menu offers a number of options that can be changed without having to open the settings dialog. Most settings in the menu are for options that are toggle-able; that is, they are either enabled or disabled. A box *with* a checkmark next to a setting means that option is enabled; a box *without* a checkmark means that setting is disabled. To toggle a setting, click on it; if the setting is enabled, it will be disabled, and vice versa. The setting change will be saved to the configuration file, and the setting will be applied immediately. Two settings (the spellcheck language and widget style) have circles; these options are multiple choice, and only one choice can be selected. An *empty* circle means that option is not selected, and a *full* circle means that option is selected. For either of these settings, click on the option to switch to that setting. The setting will be saved and applied immediately.

Regular and Dark Mode

MERK can be operated in "light" mode, seen below to the left, or in "dark" mode, on the right. To switch to "dark" mode, select it in the "Settings" menu or in the settings dialog. MERK will have to be restarted for it to take effect, and you'll be prompted to restart MERK automatically. Both “dark” and “light” mode have their own, separate, default text styles.



Normal Mode

Dark Mode

For more advanced users, if you want to edit the palette that "dark" mode uses, all of the colors used by the application are stored in a file in the **`styles`** directory named **`dark.palette`**. The file format is specific to MERK (similar to CSS[5]), but you can edit it with a text editor. All colors are stored in the hexadecimal format used by HTML. To re-create the file and reset the "dark" mode values back to the default, delete **`dark.palette`** and restart MERK; the application will regenerate the file with default values.

Each mode has their own text style, which is only used in that mode.

5 *Cascading Style Sheets (CSS) is a style sheet language used for specifying the presentation and styling of a document written in a markup language such as HTML or XML…* Wikipedia

Drag and Drop

To install plugins into MERK without the plugin manager or the **`--install`** command-line flag, you can simply drag the plugin's ZIP file onto MERK's main window. The plugin's code, icon, and any scripts in the ZIP will be extracted, installed in the **`plugins`** and **`scripts`** folder, and the plugin will be loaded into memory.

Dragging a MERK script (with the **`.merk`** file extension) onto MERK's main window will open the script in the script editor.

Dragging a Python script (with the **`.py`** file extension) onto MERK's main window will open the script in the Python editor.

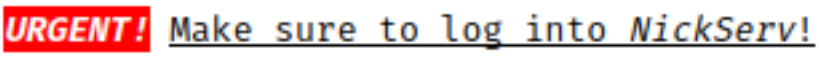
Files do not have to be dragged and dropped one-by-one. Multiple files can be dragged and dropped, of various types, and they will all be dealt with. Multiple plugin packages will be extracted and installed before all the plugins are loaded into memory, and each MERK script or Python file will be opened up in their own editor window.

Formatting Input with “MERK Markup”

Any messages or text input widget in the topic editor can be formatted with markdown or IRC colors. This is called “MERK markup”, or just “markup”. For an example of MERK markup, take this message:

```
<0,4***URGENT!***> __Make sure to log into *NickServ*!__
```

In MERK markup, this will end up looking like:



If IRC color display is turned off in MERK, the formatting and colors will not be displayed in MERK’s chat display, but will still be sent to the server for display on other clients. Both IRC color and markdown input can be turned off in the settings dialog or menu. They are both turned on by default.

Colors

Both foreground (text) and background colors can be set. To set the foreground text color only, type **<NUMBER** at the beginning of the block of text to color (with **NUMBER** set to the IRC color to use). To set both the foreground and background color, type **<NUMBER,NUMBER** at the beginning of the block to color (with the first **NUMBER** setting the IRC color to use for the text, and the second **NUMBER** setting the IRC color to use for the background). Valid IRC color numbers are 0 (zero) to 15:

0) **White** (#FFFFFF)
1) **Black** (#000000)
2) **Blue** (#00007F)
3) **Green** (#009300)
4) **Light Red** (#FF0000)
5) **Brown** (#7F0000)
6) **Purple** (#9C009C)
7) **Orange** (#FC7F00)
8) **Yellow** (#FFFF00)
9) **Light Green** (#00FC00)
10) **Cyan** (#009393)
11) **Light Cyan** (#00FFFF)
12) **Light Blue** (#0000FC)
13) **Pink** (#FF00FF)
14) **Grey** (#7F7F7F)
15) **Light Grey** (#D2D2D2)

Invalid IRC color numbers (numbers greater than 15 or less than 0) will result in the color block being stripped from the message; the text will be sent un-colored. Nesting color blocks may result in unexpected output.

As an example, to send the message “Hello” in pink text, you would enter **<13Hello>**, which would look like **Hello**. To send the message “Hello, world!” in white text on a grey background, you would type **<0,14Hello, world!>**, which would look like `Hello world!`.

When editing channel topics with IRC color codes, the channel's topic will be converted to this "markup" style. That is, if the channel's topic is RED ALERT, when you click on the topic editor, it will be converted into **<0,4RED ALERT>**. Not all clients (or users) properly close color tags in their messages or topics, and though the colored text will be displayed properly in MERK, it may not convert into "markup" cleanly, and tags may remain open.

Markdown

Text can be displayed in *italics*, **bold**, underline, and ~~strikethrough~~:

- **Italics**. To display text in italics, place **`*`** (one asterisk) before and after the text to italicize.
- **Bold**. To display text in bold, place **`**`** (two asterisks) before and after the text to bold.
- **Underline**. To display underlined text, place __ (two underscores) before and after the text to underline.
- **Strikethrough**. To display strikethrough text, place **`~`** (one tilde) before and after the text to underline. Not all IRC clients can display this formatting.
- **Escaping characters**. To send a message that displays one of the characters used for markdown without turning it into markdown, put a forward slash in front of the character. For example, to send the message "**`*test*`**" without using italics, you would type "**`\*test\*`**".

For example, to send the text "**Hello**", you would enter __**`**Hello**`**__. To send the text "*Hello*, ~~world~~!", enter **`*Hello*, ~world~!`**.

Emojis and ASCIImoji Shortcodes

To insert emojis or ASCIImojis into chat, topics, or messages, MERK supports shortcodes.

For emojis, shortcodes are surrounded by colons (**`:`**), and are the same used by many other applications and messaging programs. For example, to display the 😄 emoji, use **`:smile:`**. If autocomplete is turned on, you can type the first few characters of a shortcode and press the tab key to complete the shortcode. For a list of the shortcodes supported by MERK, please see this website.

ASCIImojis are sort of like emojis, but consist of ASCII characters rather than UTF8 characters. ASCIImoji shorts codes are surrounded by parenthesis (starting with **`(`** and ending with **`)`**). For example, to display the ASCII emoji **`\( ﾟヮﾟ)/`**, use **`(yay)`**. Autocomplete also supports ASCIImojis; to use, type the first few characters of an ASCIImoji and press the tab key. To see what ASCIImojis MERK supports, please see this website.

Both emoji and ASCIImoji shortcode support can be turned off in settings.

Channel Topics in the Topic Editor

When the channel topic editor is clicked, channel topics are *always* converted into “MERK markup”, converting IRC colors into the format described earlier, and the same with the markdown format. Emoji and ASCIImojis are also *always* converted into shortcodes, even if emoji or ASCIImoji shortcodes are disabled. When the topic has been edited, and the user hits the “return” button, it will be converted into the appropriate IRC colors, formatting, and emojis/ASCIImojis before being sent to the server. If IRC colors in topics are turned off, they will not be displayed in MERK, but they will be sent to the server, and other clients will see them.

This is to prevent MERK users from “damaging” topics set by others. This way, even if MERK users have colors turned off, they have the option of not removing other people’s IRC color codes or formatting.

Commands and Scripting Guide

MERK features over 80 different commands that can be used in server, channel, and private chat windows, and 20 script-only commands that can be used in MERK scripts. With these commands, users can send any needed commands to an IRC server or do nearly anything in MERK that can be done with a mouse. Scripts can automate server connections, create new commands (with the **/macro** command), configure MERK, create "variables" (called aliases) that can be interpolated[6] into input and output, create hotkeys (with the **/bind** command), and much more. Scripts feature limited flow control[7] (with the **if**, **goto**, and **loop** commands), the ability to combine multiple scripts together during execution (with the **insert** command), and have commands to get information from the user (with the **input** and **number** commands). To see what MERK scripts can do, this guide contains a number of example scripts, including setting a custom "away" message for each day of the week and creating the infamous "/trout" macro.

Command List

IRC Commands

All of these commands are related to IRC, in some way, shape, or form. Commands that are limited to server operators begin with an underscore, and usually display the responses to that command in server windows. Information about the details of many of these commands can be found in RFC 1459 and RFC 2812, which can be found in the "Help" menu.

Command	Description
/admin [SERVER]	Requests administration information from the server
/away [SERVER] [MESSAGE]	Sets status as "away". To specify what server to set the "away" status on, pass a hostID (the host and port used to connect to the server, in the format **host:port**, or the hostname used to connect) as **SERVER**
/back [SERVER]	Sets status as "back". To specify what server to set the "back" status on, pass a hostID (the host and port used to connect to the server, in the format **host:port**, or the hostname used to connect) as **SERVER**
/_connect SERVER PORT [REMOTE]	Instructs a server to connect to another server. May only be issued by server operators
/ctcp REQUEST USER	Sends a CTCP request to a user. Valid requests are TIME, VERSION, USERINFO, SOURCE, or FINGER
/_die	Instructs a server to shut down. May only be issued by server operators
/finger TEXT...	Sets the CTCP FINGER response. Pass ***** as the argument to clear the FINGER response text
/info [TARGET]	Requests server information

6 *...string interpolation...is the process of evaluating a string literal containing one or more placeholders, yielding a result in which the placeholders are replaced with their corresponding values.* Wikipedia

7 *...control flow...describes how execution progresses from one command to the next.* Wikipedia

Command	Description
/invite NICKNAME CHANNEL	Sends a channel invitation
/ison NICKNAME(S)...	Displays if the specified nicknames are online
/join CHANNEL [KEY]	Joins a channel
/kick CHANNEL NICKNAME [MESSAGE]	Kicks a user from a channel
/_kill CLIENT COMMENT...	Forcibly removes **CLIENT** from the network. May only be issued by IRC operators.
/knock CHANNEL [MESSAGE]	Requests an invitation to a channel
/links [REMOTE [MASK]]	Requests a list of servers the server is connected to
/list [TERMS]	Lists or searches channels on the server; use "*" for multi-character wildcard and "?" for single character
/lusers [MASK [SERVERS]]	Requests statistics about the server
/me MESSAGE...	Sends a CTCP action message to the current chat
/mode TARGET MODE...	Sets a mode on a channel or user
/msg TARGET MESSAGE...	Sends a message
/nick NEW_NICKNAME	Changes your nickname
/notice TARGET MESSAGE...	Sends a notice
/oper USERNAME PASSWORD	Logs into an operator account
/part CHANNEL [MESSAGE]	Leaves a channel
/ping USER [TEXT]	Sends a CTCP ping to a user
/quit [MESSAGE]	Disconnects from the current IRC server
/quitall [MESSAGE]	Disconnects from all IRC servers
/quote [SERVER] TEXT...	Sends unprocessed data to the server. To specify what server to send the message to, pass a hostID (the host and port used to connect to the server, in the format **host:port**, or the hostname used to connect) as **SERVER**
/refresh	Requests a new list of channels from the server
/_rehash	Causes the server to reprocess and reload configuration files. May only be issued by IRC operators
/time	Requests server time
/topic CHANNEL NEW_TOPIC	Sets a channel topic
/_trace TARGET	Executes a trace on a server or user. May only be issued by IRC operators
/userhost NICK(S)...	Requests information about users from the server
/userinfo TEXT...	Sets the CTCP USERINFO response. Pass ***** as the argument to clear the USERINFO response text
/version [SERVER]	Requests server version
/wallops MESSAGE	Sends a message to all operators
/who NICKNAME [o]	Requests user information from the server
/whois NICKNAME [SERVER]	Requests user information from the server
/whowas NICKNAME [COUNT] [SERVER]	Requests information about previously connected users

All Other Commands

These commands are specific to MERK, and are for using and manipulating the client. Commands with a gray background are for use in scripts *only*; they cannot be used in the text input widget, and if used there, will only be sent to the server as a message.

Command	Description
`/alias [TOKEN] [TEXT...]`	Creates an alias that can be referenced by **`$TOKEN`**. Call with only **`TOKEN`** as the argument to see **`TOKEN`**'s value. If **`TEXT`** is a mathematical statement, it will be evaluated, with the resulting value stored as the alias' value. Call without any arguments to see all aliases and their values.
`append FILENAME CONTENTS...`	Writes **`CONTENTS`** to **`FILENAME`**, followed by a newline, appending to the file if it already exists. Arguments to this command are parsed like script arguments; if a filename contains spaces, contain it in quotes. This command is blocking. Can only be called from scripts.
`/bind SEQUENCE COMMAND...`	Executes **`COMMAND`** every time key **`SEQUENCE`** is pressed. **`SEQUENCE`** should contain no spaces, and should be a string like "Ctrl+M" or "Alt+Space". **`COMMAND`** will be executed in whatever window/context has focus when **`SEQUENCE`** is pressed.
`/browser URL`	Opens **`URL`** in the default browser
`/call METHOD ARGUMENTS...`	Executes a **`METHOD`** in a plugin.
`/clear [SERVER] [WINDOW]`	Clears a window's chat display. **`SERVER`** is optional if **`WINDOW`** belongs to the same context. Pass **`*`** as the **`WINDOW`** to clear the server window
`/close [SERVER] [WINDOW]`	Closes a subwindow. **`SERVER`** is optional if **`WINDOW`** belongs to the same context. Pass **`*`** as the **`WINDOW`** to hide the server window
`/config [SETTING] [VALUE...]`	Changes a setting, or searches and displays one or all settings in the configuration file
`/config export [FILENAME]`	Exports the current configuration file
`/config import [FILENAME]`	Imports a configuration file into settings
`/connect SERVER [PORT] [PASSWORD]`	Connects to an IRC server
`/connectssl SERVER [PORT] [PASSWORD]`	Connects to an IRC server via SSL
`context [HOSTID] WINDOW_NAME`	Moves execution of the script to **`WINDOW_NAME`**. Pass a **`HOSTID`** as the first argument to reference a context on a specific server. Can only be called from scripts

decimal ALIAS LOW HIGH MESSAGE...	Requests a decimal number, between **LOW** and **HIGH**, from the user in a dialog (with **MESSAGE**), and stores the input in **ALIAS**. If the user cancels the dialog, **ALIAS** will be set to *****. This command is blocking, and will stop a script's execution until the user either enters something into the dialog and clicks "ok", or clicks "cancel". Can only be called from scripts
/delay SECONDS COMMAND...	Executes **COMMAND** after **SECONDS** seconds
/edit [FILENAME]	Opens a script in the editor; if called without an argument, opens an editor window
end	Immediately ends a script. Can only be called from scripts.
escape ALIAS TEXT...	Escapes MERK markdown in text and stores the result in an alias. Can only be called from scripts.
exclude WINDOW...	Prevents a script from executing in **WINDOW**'s context. Multiple **WINDOW**s can be specified. Can only be called from scripts.
/exit [SECONDS]	Exits the client, with an optional pause of **SECONDS** before exit
/fade [SERVER] [WINDOW] PERCENTAGE	Sets the transparency of a subwindow by **PERCENTAGE**, from 1 to 100. Call without arguments to see the current subwindow's transparency. Pass ***** as the **WINDOW** to set the transparency of the server window
/find [TERMS]	Finds filenames that can be found by other commands, like **/script** or **/edit**. If called without any arguments, **/find** will list all files visible to commands. Can use ***** for multi-character wildcards and **?** for single character wildcards.
/folder PATH	Opens **PATH** in the default file manager.
getfile ALIAS MESSAGE...	Displays an "open file" dialog and stores the result in **ALIAS**. If no file is selected, **ALIAS** will be set to *****. This command is blocking. Can only be called from scripts.
goto TARGET	Moves execution of the script to a line number or a target. One of the only script-only commands that can be issued from an **if** command. Can only be called from scripts.
halt [MESSAGE...]	Asks the user if they want to halt the script's execution, and displays an error **MESSAGE**. This command is blocking. One of the only script-only commands that can be issued from an **if** command. Can only be called from scripts.
/help [COMMAND]	Displays a list of available commands, and command usage information. This list will be modified as features are enabled or disabled.
/hide [SERVER] [WINDOW]	Hides a subwindow. **SERVER** is optional if **WINDOW** belongs to the same context. Pass ***** as the **WINDOW** to hide the server window

hostmask ALIAS NICKNAME	Retrieves the hostmask associated with a **NICKNAME** in the same context the script is ran in, and stores it in an **ALIAS**. If the hostmask is not known or can't be found, **ALIAS** will be set to **unknown**. Can only be called from scripts.
if VALUE1 OPERATOR VALUE2 COMMAND...	Executes **COMMAND** if **VALUE1** and **VALUE2** are true, depending on **OPERATOR**. Valid **OPERATOR**s are **(is)** (result is true if **VALUE1** and **VALUE2** are equal), **(not)** (result is true if **VALUE1** and **VALUE2** are not equal), **(in)** (result is true if **VALUE1** is contained in **VALUE2**), **(nin)** (result is true if **VALUE1** is not contained in **VALUE2**), **(gt)** (result is true if **VALUE1** is a greater number than **VALUE2**; if either value is a string, the length of that string is used as the number), **(lt)** (result is true if **VALUE1** is a lesser number than **VALUE2**; if either value is a string, the length of that string is used as the number), **(ne)** (result is true if **VALUE1** and **VALUE2** are numbers and are not equal; if either value is a string, the length of that string is used as the number), and **(eq)** (result is true if **VALUE1** and **VALUE2** are numbers and are equal; if either value is a string, the length of that string is used as the number). Can only be called from scripts.
/ignore USER	Hides a **USER**'s chat in all chat windows. This can be set to a nickname or hostmask. Capitalization is ignored. Use ***** as multiple character wildcards, and **?** as single character wildcards.
input ALIAS MESSAGE...	Requests input from the user in a dialog (with **MESSAGE**), and stores the input in **ALIAS**. If the user cancels the dialog or doesn't input anything, **ALIAS** will be set to *****. This command is blocking, and will stop a script's execution until the user either enters something into the dialog and clicks "ok", or clicks "cancel". Can only be called from scripts
insert FILE [FILE...]	Inserts the contents of **FILE** where it appears in the script. **FILE** should be a MERK script. If a filename contains spaces, put it in quotation marks. Multiple files can be passed as arguments. Can only be called from scripts.
loop COUNT	Begins a **loop** block, executing any scripting until the script encounters a **pool** command, before moving execution back to the line after the **loop** call. The code in between **loop** and **pool** will be repeated **COUNT** times. Can only be called from scripts.
/macro NAME SCRIPT [USAGE] [HELP]	Creates a macro, executable with **/NAME**, that executes **SCRIPT**. **USAGE** and **HELP** are used for the command list displayed with **/help**.

/maximize [SERVER] [WINDOW]	Maximizes a subwindow. **SERVER** is optional if **WINDOW** belongs to the same context. Pass ***** as the **WINDOW** to maximize the server window
/minimize [SERVER] [WINDOW]	Minimizes a subwindow. **SERVER** is optional if **WINDOW** belongs to the same context. Pass ***** as the **WINDOW** to minimize the server window
/move [SERVER] [WINDOW] X Y	Moves a subwindow to **X** (left and right) and **Y** (up and down) coordinates. **SERVER** is optional if **WINDOW** belongs to the same context. Call without arguments to see the current subwindow's coordinates. Pass ***** as the **WINDOW** to move the server window
msgbox MESSAGE...	Displays a message box with a short message. This command is blocking. One of the only script-only commands that can be issued from an **if** command. Can only be called from scripts.
number ALIAS LOW HIGH MESSAGE...	Requests a number, between **LOW** and **HIGH**, from the user in a dialog (with **MESSAGE**), and stores the input in **ALIAS**. If the user cancels the dialog, **ALIAS** will be set to *****. This command is blocking, and will stop a script's execution until the user either enters something into the dialog and clicks "ok", or clicks "cancel". Can only be called from scripts
only WINDOW...	Restricts a script to only executing in **WINDOW**'s context. Multiple **WINDOW**s can be specified. Can only be called from scripts.
/play FILENAME	Plays a WAV file
pool	Ends a **loop** block. Can only be called from scripts.
/print [SERVER] [WINDOW] TEXT...	Prints text to a window. **SERVER** is optional if **WINDOW** belongs to the same context. Pass ***** as the **WINDOW** to print the server window. If **SERVER** or **WINDOW** can't be found, text will be printed to the current window
/prints [SERVER] [WINDOW] TEXT...	Prints a system message to a window. **SERVER** is optional if **WINDOW** belongs to the same context. Pass ***** as the **WINDOW** to print the server window. If **SERVER** or **WINDOW** can't be found, text will be printed to the current window
/private NICKNAME [MESSAGE]	Opens a private chat subwindow for **NICKNAME**
/python [FILENAME]	Opens a file in the Python editor; if called without an argument, opens a Python editor window
read ALIAS FILENAME	Reads **FILENAME** as a text file, and stores the contents in **ALIAS**. If the file doesn't contain anything or consists of only whitespace, **ALIAS** will be set to *****. This command is blocking. Can only be called from scripts.
random ALIAS START FINISH	Generates a random integer from **START** to **FINISH**, and stores it in **ALIAS**. Can only be called from scripts.

/reconnect SERVER [PORT] [PASSWORD]	Connects to an IRC server, reconnecting on disconnection
/reconnectssl SERVER [PORT] [PASSWORD]	Connects to an IRC server via SSL, reconnecting on disconnection
/rem [TEXT...]	Does nothing. Can be used for comments.
restrict channel|server|private	Prevents a script from running if it is not being executed in a **channel**, **server**, or **private** chat window. Up to two window types can be set. Can only be called from scripts.
/restore [SERVER] [WINDOW]	Restores a subwindow. **SERVER** is optional if **WINDOW** belongs to the same context. Pass ***** as the **WINDOW** to restore the server window
setfile ALIAS MESSAGE...	Displays a “save file” dialog and stores the result in **ALIAS**. If no file is set, **ALIAS** will be set to *****. This command is blocking. Can only be called from scripts.
/script FILENAME [ARGUMENTS]	Executes a list of commands in a file. If the script has a file extension of **.merk**, it may be omitted from **FILENAME**.
/show [SERVER] [WINDOW]	Shows a subwindow, if hidden, and shifts focus to that subwindow. **SERVER** is optional if **WINDOW** belongs to the same context. Pass ***** as the **WINDOW** to show the server window
/size [SERVER] [WINDOW] WIDTH HEIGHT	Resizes a subwindow. **SERVER** is optional if **WINDOW** belongs to the same context. Call without arguments to see the current subwindow’s size. Pass ***** as the **WINDOW** to resize the server window
/style [SERVER] [WINDOW]	Opens a window's text style editor. Pass ***** as the **WINDOW** to select the server window
target LABEL	Creates a target for the **goto** command. If used as a target for **goto**, script execution will move to the line this appears on. Can only be called from scripts
/unalias TOKEN	Deletes the alias referenced by **$TOKEN**. Does nothing in scripts
/unbind SEQUENCE	Removes a bind for key **SEQUENCE**. To remove all binds, pass ***** as the argument
/unignore USER	Un-hides a **USER**'s chat in all chat windows. This can be set to a nickname or hostmask. Capitalization is ignored. To un-hide all users, use ***** as the argument.
/unmacro NAME	Deletes the macro named **NAME**
usage NUMBER [MESSAGE...]	Prevents a script from running unless **NUMBER** or more arguments are passed to it, displaying **MESSAGE**. Can only be called from scripts. If a script should take 1 or more arguments, set **NUMBER** to **+**.
/user [SETTING] [VALUE...]	Changes a user setting, or searches and displays one or all settings in the user file. Pass ***** as **VALUE** to set a setting as blank
wait SECONDS	Pauses script execution for SECONDS; can only be called from scripts

/warn [SERVER] [WINDOW] TEXT...	Prints an error message to a window. **SERVER** is optional if **WINDOW** belongs to the same context. Pass ***** as the **WINDOW** to print the server window. If **SERVER** or **WINDOW** can't be found, text will be printed to the current window
/window [COMMAND] [X] [Y]	Manipulates the main application window. Valid commands are **cascade**, **fade**, **fullscreen,** **hotkey**, **ignore**, **install**, **layout**, **logs**, **maximize**, **minimize**, **move**, **next**, **ontop**, **pause**, **plugin**, **previous**, **readme**, **restart**, **restore**, **settings**, **size**, **tile**, and **uninstall**. Some of these commands may not be available if certain features are turned off. Call with no arguments to see window information and a list of subwindows.
write FILENAME CONTENTS...	Writes **CONTENTS** to **FILENAME**, followed by a newline, overwriting the file if it already exists. Arguments to this command are parsed like script arguments; if a filename contains spaces, contain it in quotes. This command is blocking. Can only be called from scripts.
/xconnect SERVER [PORT] [PASSWORD]	Connects to an IRC server & executes connection script
/xconnectssl SERVER [PORT] [PASSWORD]	Connects to an IRC server via SSL & executes connection script
/xreconnect SERVER [PORT] [PASSWORD]	Connects to an IRC server & executes connection script, reconnecting on disconnection
/xreconnectssl SERVER [PORT] [PASSWORD]	Connects to an IRC server via SSL & executes connection script, reconnecting on disconnection

Script-Only Commands

These commands can only be called from scripts. Attempts to use them in the text input widget will fail and be sent to servers as messages.

Command	Description
append FILENAME CONTENTS...	Writes **CONTENTS** to **FILENAME**, followed by a newline, appending to the file if it already exists. Arguments to this command are parsed like script arguments; if a filename contains spaces, contain it in quotes. This command is blocking. Can only be called from scripts.
context [HOSTID] WINDOW_NAME	Moves execution of the script to **WINDOW_NAME**. Pass a **HOSTID** as the first argument to reference a context on a specific server. Can only be called from scripts
decimal ALIAS LOW HIGH MESSAGE...	Requests a decimal number, between **LOW** and **HIGH**, from the user in a dialog (with **MESSAGE**), and stores the input in **ALIAS**. If the user cancels the dialog, **ALIAS** will be set to *****. This command is blocking, and will stop a script's execution until the user either enters something into the dialog and clicks "ok", or clicks "cancel". Can only be called from scripts
end	Immediately ends a script. Can only be called from scripts.
escape ALIAS TEXT...	Escapes MERK markdown in text and stores the result in an alias. Can only be called from scripts.
exclude WINDOW...	Prevents a script from executing in **WINDOW**'s context. Multiple **WINDOW**s can be specified. Can only be called from scripts.
getfile ALIAS MESSAGE...	Displays an "open file" dialog and stores the result in **ALIAS**. If no file is selected, **ALIAS** will be set to *****. This command is blocking. Can only be called from scripts.
goto TARGET	Moves execution of the script to a line number or target. The only script-only command that can be issued from an **if** command. Can only be called from scripts.
halt [MESSAGE...]	Asks the user if they want to halt the script's execution, and displays an error **MESSAGE**. This command is blocking. Can only be called from scripts.
hostmask ALIAS NICKNAME	Retrieves the hostmask associated with a **NICKNAME** in the same context the script is ran in, and stores it in an **ALIAS**. If the hostmask is not known or can't be found, **ALIAS** will be set to **unknown**. Can only be called from scripts.

Command	Description
if VALUE1 OPERATOR VALUE2 COMMAND...	Executes **COMMAND** if **VALUE1** and **VALUE2** are true, depending on **OPERATOR**. Valid **OPERATOR**s are **(is)** (result is true if **VALUE1** and **VALUE2** are equal), **(not)** (result is true if **VALUE1** and **VALUE2** are not equal), **(in)** (result is true if **VALUE1** is contained in **VALUE2**), **(nin)** (result is true if **VALUE1** is not contained in **VALUE2**), **(gt)** (result is true if **VALUE1** is a greater number than **VALUE2**; if either value is a string, the length of that string is used as the number), **(lt)** (result is true if **VALUE1** is a lesser number than **VALUE2**; if either value is a string, the length of that string is used as the number), **(ne)** (result is true if **VALUE1** and **VALUE2** are numbers and are not equal; if either value is a string, the length of that string is used as the number), and **(eq)** (result is true if **VALUE1** and **VALUE2** are numbers and are equal; if either value is a string, the length of that string is used as the number). Can only be called from scripts.
input ALIAS MESSAGE...	Requests input from the user in a dialog (with **MESSAGE**), and stores the input in **ALIAS**. If the user cancels the dialog or doesn't input anything, **ALIAS** will be set to *****. This command is blocking, and will stop a script's execution until the user either enters something into the dialog and clicks "ok", or clicks "cancel". Can only be called from scripts
insert FILE [FILE...]	Inserts the contents of **FILE** into the script. **FILE** should be a MERK script. If a filename contains spaces, put it in quotation marks. Multiple files can be passed as arguments. Can only be called from scripts.
loop COUNT	Begins a **loop** block, executing any scripting until the script encounters a **pool** command, before moving execution back to the line after the **loop** call. The code in between **loop** and **pool** will be repeated **COUNT** times. Can only be called from scripts.
msgbox MESSAGE...	Displays a message box with a short message. This command is blocking. One of the only script-only commands that can be issued from an **if** command. Can only be called from scripts.
number ALIAS LOW HIGH MESSAGE...	Requests a number, between **LOW** and **HIGH**, from the user in a dialog (with **MESSAGE**), and stores the input in **ALIAS**. If the user cancels the dialog, **ALIAS** will be set to *****. This command is blocking, and will stop a script's execution until the user either enters something into the dialog and clicks "ok", or clicks "cancel". Can only be called from scripts
only WINDOW...	Restricts a script to only executing in **WINDOW**'s context. Multiple **WINDOW**s can be specified. Can only be called from scripts.
pool	Ends a **loop** block. Can only be called from scripts.

Command	Description
read ALIAS FILENAME	Reads **FILENAME** as a text file, and stores the contents in **ALIAS**. If the file doesn't contain anything or consists of only whitespace, **ALIAS** will be set to *****.This command is blocking. Can only be called from scripts.
random ALIAS START FINISH	Generates a random integer from **START** to **FINISH**, and stores it in **ALIAS**. Can only be called from scripts.
restrict channel|server|private	Prevents a script from running if it is not being executed in a **channel**, **server**, or **private** chat window. Up to two window types can be set. Can only be called from scripts.
setfile ALIAS MESSAGE...	Displays a "save file" dialog and stores the result in **ALIAS**. If no file is set, **ALIAS** will be set to *****. This command is blocking. Can only be called from scripts.
target LABEL	Creates a target for the **goto** command. If used as a target for **goto**, script execution will move to the line this appears on. Can only be called from scripts
usage NUMBER [MESSAGE...]	Prevents a script from running unless **NUMBER** or more arguments are passed to it, displaying **MESSAGE**. Can only be called from scripts. If a script should take 1 or more arguments, set **NUMBER** to **+**.
wait SECONDS	Pauses script execution for **SECONDS**; can only be called from scripts
write FILENAME CONTENTS...	Writes **CONTENTS** to **FILENAME**, followed by a newline, overwriting the file if it already exists. Arguments to this command are parsed like script arguments; if a filename contains spaces, contain it in quotes. This command is blocking. Can only be called from scripts.

Context-less Commands

These commands can be called without specifying the channel, chat, server, or window they are for. They will run in the current context. Commands with a gray background are IRC specific commands.

Command	Description
/away [MESSAGE]	Sets status as "away"
/back	Sets status as "back"
/clear	Clears the current window's chat display
/close	Closes the current subwindow
/hide	Hides the current subwindow
/invite NICKNAME	Sends a channel invitation to the current channel
/kick NICKNAME [MESSAGE]	Kicks a user from the channel
/maximize	Maximizes the current subwindow
/me MESSAGE...	Sends a CTCP action message to the current chat
/minimize	Minimizes the current subwindow
/mode MODE...	Sets a mode on the current channel
/move X Y	Moves the current subwindow to **X** (left and right) and **Y** (up and down) coordinates.
/part [MESSAGE]	Leaves the channel
/print TEXT...	Prints text to the current window
/prints TEXT...	Prints a system message to the current window
/quote TEXT...	Sends unprocessed data to the server
/size WIDTH HEIGHT	Resizes the current subwindow
/restore	Restores the current subwindow
/show	Shows a subwindow, if hidden, and shifts focus to that subwindow.
/style [SERVER] [WINDOW]	Opens a window's text style editor.
/topic NEW_TOPIC	Sets the current channel's topic
/warn TEXT...	Prints an error message to the current window

Subwindow and Window Management Commands

These commands are used to manipulate both the main application window, and the subwindows it contains. For all of these commands except for **`/window`**, if the **SERVER** and **`WINDOW`** arguments are left out, the command will be enacted on the subwindow the command was entered in.

Command	Description
`/close [SERVER] [WINDOW]`	Closes a subwindow. Pass **`*`** as the **`WINDOW`** to hide the server window
`/fade [SERVER] [WINDOW] PERCENTAGE`	Sets the transparency of a subwindow by **`PERCENTAGE`**, from 1 to 100. Call without arguments to see the current subwindow's transparency. Pass **`*`** as the **`WINDOW`** to set the transparency of the server window
`/hide [SERVER] [WINDOW]`	Hides a subwindow. Pass **`*`** as the **`WINDOW`** to hide the server window
`/maximize [SERVER] [WINDOW]`	Maximizes a window. Pass **`*`** as the **`WINDOW`** to maximize the server window
`/minimize [SERVER] [WINDOW]`	Minimizes a window. Pass **`*`** as the **`WINDOW`** to minimize the server window
`/move [SERVER] [WINDOW] X Y`	Moves a subwindow to **`X`** (left and right) and **`Y`** (up and down) coordinates. Call without arguments to see the current subwindow's coordinates. Pass **`*`** as the **`WINDOW`** to move the server window
`/restore [SERVER] [WINDOW]`	Restores a window. Pass **`*`** as the **`WINDOW`** to restore the server window
`/show [SERVER] [WINDOW]`	Shows a subwindow, if hidden, and shifts focus to that subwindow. Pass **`*`** as the **`WINDOW`** to show the server window
`/size [SERVER] [WINDOW] WIDTH HEIGHT`	Resizes a subwindow. **SERVER** is optional if **`WINDOW`** belongs to the same context. Call without arguments to see the current subwindow's size. Pass **`*`** as the **`WINDOW`** to resize the server window
`/window [COMMAND] [X] [Y]`	Manipulates the main application window. Valid commands are **`cascade`**, **`fade`**, **`fullscreen`**, **`hotkey`**, **`ignore`**, **`install`**, **`layout`**, **`logs`**, **`maximize`**, **`minimize`**, **`move`**, **`next`**, **`ontop`**, **`pause`**, **`plugin`**, **`previous`**, **`readme`**, **`restart`**, **`restore`**, **`settings`**, **`size`**, **`tile`**, and **`uninstall`**. Some of these commands may not be available if certain features are turned off. Call with no arguments to see window information and a list of subwindows.

Additional Command Help

- **`wait`** – ***Can only*** *be called from* ***scripts***
- **`context`** – ***Can only*** *be called from* ***scripts***
- **`end`** – ***Can only*** *be called from* ***scripts***
- **`usage`** – ***Can only*** *be called from* ***scripts***
- **`restrict`** – ***Can only*** *be called from* ***scripts***
- **`insert*`** – ***Can only*** *be called from* ***scripts***
- **`only`** – ***Can only*** *be called from* ***scripts***
- **`exclude`** – ***Can only*** *be called from* ***scripts***
- **`if`** – ***Can only*** *be called from* ***scripts***
- **`goto`** – ***Can only*** *be called from* ***scripts***
- **`halt*`** – ***Can only*** *be called from* ***scripts***
- **`random*` – *Can only*** *be called from* ***scripts***
- **`target` – *Can only*** *be called from* ***scripts***
- **`read*` – *Can only*** *be called from* ***scripts***
- **`loop` – *Can only*** *be called from* ***scripts***
- **`pool` – *Can only*** *be called from* ***scripts***
- **`input*` – *Can only*** *be called from* ***scripts***
- **`number*` – *Can only*** *be called from* ***scripts***
- **`hostmask` – *Can only*** *be called from* ***scripts***
- **`escape` – *Can only*** *be called from* ***scripts***
- **`write*` – *Can only*** *be called from* ***scripts***
- **`getfile*` – *Can only*** *be called from* ***scripts***
- **`setfile*` – *Can only*** *be called from* ***scripts***
- **`append*` – *Can only*** *be called from* ***scripts***
- **`decimal*` – *Can only*** *be called from* ***scripts***
- **`msgbox*` – *Can only*** *be called from* ***scripts***

Most commands can be issued in both the text input widget and scripts. There are twenty-six commands, however, that can *only* be issued in scripts: **`wait`**, **`context`**, **`usage`**, **`restrict`**, **`insert`**, **`only`**, **`exclude`**, **`if`**, **`goto`**, **`random`**, **`halt`**, **`target`**, **`read`**, **`loop`**, **`pool`**, **`input`**, **`number`**, **`hostmask`**, **`escape`**, **`write`**, **`getfile`**, **`setfile`**, **`append`**, **`decimal`**, **`msgbox`**, and **`end`**. These twenty-six commands *cannot* be used in the text input widget, and cannot be called as the result of a successful **`if`** call (with the exception of **`goto`** , **`halt`**, and **`msgbox`**, the only script-only commands that can be used with **`if`**). Commands marked with a **`*`** are blocking[8].

Most commands require a context to be executed in (see ***Context***). Commands that can be issued without explicitly specifying a context are **`/clear`**, **`/invite`**, **`/kick`**, **`/me`**, **`/mode`**, **`/part`**, **`/topic`**, **`/maximize`**, **`/minimize`**, **`/hide`**, **`/show`**, **`/close`**, **`/print`**, **`/prints`**, **`/warn`**, **`/size`**, **`/move`**, **`/away`**, **`/back`**, **`/quote`**, and **`/restore`**; they will be executed in whatever the current context is, and may not function correctly if the current context does not support that command (for example, calling **`/invite`** from a server window). The only

8 *... a process that is blocked is waiting for some event, such as a resource becoming available or the completion of an I/O operation.* Wikipedia

exception is **/me**: if called from the text input widget of a channel or private chat window, it will send a CTCP action to the current window, using *all* arguments as the text to send in the message. If **/me** is called from a server window, the first argument specifies the channel or private chat to send the CTCP message to. For example, to send a CTCP message containing "is using MERK" to the **#merk** channel, you could call **/me #merk is using MERK** from a server window. When calling **/me** from a script, *the command will always send to the current chat if running in a channel or private chat window, and must specify the context if running in a server window*. If a context is specified as the first argument to **/me**, and the command is executed in a channel or private chat window, the specified context will be sent as part of the CTCP action message.

The **/call** command can be used to call one (or more) Python methods in plugins. This allows for users to write complex commands in Python.

Most commands that require a numerical argument require an integer; that is, a whole or natural number[9]. Several commands can take floating-point numbers[10] as arguments, allowing for fractional numbers: **if**, **wait**, **/delay**, and **/exit**. As most of these commands' numerical arguments relate to the amount of time to "pause" a command or script's execution, floating-point numbers allow these commands to "pause" for time periods of less than a second. For example, to "pause" a script for 500 milliseconds, or one half of a second:

```
wait 0.5
```

To convert milliseconds into seconds, divide the number of milliseconds by 1000 to determine the number of seconds. 1 millisecond is 0.001 seconds, 2ms is 0.002 seconds, and so on.

random *only* takes integers as arguments (besides the name of the alias to store the generated number in). Passing a float as an argument to **random** will result in an error.

HostIDs

Many of the commands that affect subwindows can take an optional server and window name as arguments, like **/show**, **/fade**, **/move**, and the like. The most reliable way to designate a server in one of the commands is to use a connection's "hostID". This is the hostname of the server used to connect to the server, and the port, separated by **:**. So, to reference a window associated with a connection to **us.dal.net** on port 6667, the hostID would be **us.dal.net:6667**. If you are connecting to a server via a round-robin server, the server host that *you used to connect to the server* should be used in the hostID. For example, if you initially connected to **irc.libera.chat**, but you ended up on **uranium.libera.chat**, use **irc.libera.chat** in the hostID. The port may be left out of a hostID. If you have multiple connections to the same server, the first subwindow that uses the hostID that is *found* will be used, usually in order of connection. For example, if you are connected to **us.dal.net** twice, with both connections in the **#merk** channel, the hostID/channel combo

9 ...Natural numbers are the numbers 0, 1, 2, 3, and so on. Wikipedia
10 Wikipedia article on floating-point arithmetic

us.dal.net #merk will probably find the **#merk** window from whichever connection connected first.
As an example, let's assume you are connected to **us.dal.net** on port 6667; on this server you are in the channel **#merk**. You're also connected to **irc.libera.chat** on port 6697, and are in **#merk** there, too. Let's say you want to use the **/fade** command to set the transparency of the **#merk** window on **us.dal.net** to 80%, and the transparency of the **#merk** window in **irc.libera.chat** to 75%. You can do this easily with two commands. In the first, we'll leave the port in the hostID, and we'll omit it in the second hostID:

```
/fade us.dal.net:6667 #merk 80
/fade irc.libera.chat #merk 75
```

/bind and **/unbind**

The **/bind** command allows users to create their own hotkeys for MERK. The first argument to **/bind** should be the key sequence that triggers the hotkey; it should contain no spaces. Valid key signifiers include "Ctrl" (for the control key), "Shift" (for the shift key), "Alt" (for the alt key), and "Meta" (for the meta key). For example, the sequence "Ctrl+M" will trigger if the control key and the letter "M" are pressed at the same time. Capitalization does not matter (so "Ctrl+M" is the same bind as "ctrl+m" or "Ctrl+m").

All other arguments after the first are the command to execute when the key sequence is pressed. The command will be executed in the context of whatever window is active when the sequence is pressed. So, to print the words "Hello world" to the current window with the control key and the letter "H" are pressed, you could use:

```
/bind Ctrl+H /print Hello world
```

Only a single command can be assigned to a bind; if the desired action is complex or contains more than one command, use the **/script** command.

Commands triggered by a bound hotkey are treated *exactly* as if the command was typed into the window's text input widget and the return key was pressed. That means that they can execute any command that can be entered into the text input widget, and can *not* use any script-only commands. If the bound command does not contain any actual command, the hotkey will not do anything, and an error message is displayed. Any built-in aliases in the command will reference the context of the window the command is being executed in. Hotkeys only work on active subwindows. If no subwindows are open, *hotkeys will not work*.

For an example, let's create a bind that sends a greeting to the current window. It will use a built-in alias to send a message to the current channel or private chat. First, we'll create our script:

```
restrict channel private
/msg $_WINDOW Hello!
```

Save this script to a file named **hello.merk** in your "scripts" directory. Now that we have our script, we'll create our bind:

```
/bind Ctrl+H /script hello
```

Now, no matter what channel or private chat MERK is in, every time the control key is pressed at the same time as the "H" key is pressed, the message "Hello!" will be sent to the current chat. If a server window is active when the key sequence is pressed, an error will be shown.

Alternately, another way to do the exact same thing is to just bind the command:

```
/bind Ctrl+H /msg $_WINDOW Hello!
```

Only one bind for a given sequence can be active at a time. For example, if there is a bind for the "Ctrl+H" sequence, creating another bind for "Ctrl+H" will display an error.

Be aware that this command can "overwrite" default hotkeys, like "Ctrl+C" for copy and "Ctrl+V" for paste, and MERK will not prevent this. Some hotkeys can not be overwritten, and the bind will not work. If a set hotkey doesn't seem to work, try a different key sequence.

To remove an existing bind, use the **`/unbind`** command. Pass the key sequence of the bind to remove as the first argument; just like with the **`/bind`** command, capitalization doesn't matter, and the sequence should not contain spaces.

To remove all created hotkey binds, pass **`*`** as the only argument to **`/unbind`**.

To see a list of all the binds currently in effect, call **`/bind`** without any arguments.

Binds can also be set with MERK's GUI. To bind or remove a hotkey, or save hotkeys to the configuration file, or save all current hotkeys to the configuration file, use the hotkey manager.

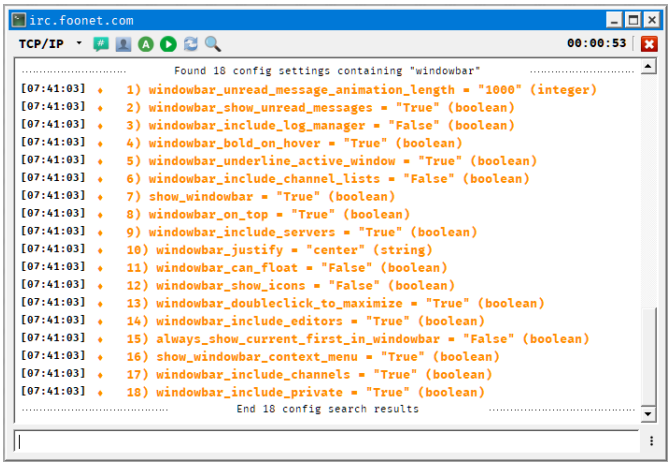
`/call`

The **`/call`** command is used to call Python methods in plugins. The first argument should be the name of the method, and all other arguments are tokenized like the arguments to the **`/script`** command. Executing **`/call`** is blocking; unless the code in the method executes in another thread, MERK will halt until execution of the method is complete. For more information on how to write and use these methods, see ***Writing Methods for /call***.

`/config`

The **`/config`** command allows users to edit the main MERK configuration file, **`settings.json`**, from within the client. ***Warning!*** It is possible to break or otherwise "mess up" MERK's configuration with this command. If this occurs, see ***Resetting MERK to Default Settings*** to undo the damage.

If called with no arguments, **`/config`** will list all the settings that can be edited with the command. To search settings, pass search terms to the command as an argument. For example, to search all settings that have the word "windowbar" in them, you could execute **`/config windowbar`**; this will print a list of all the settings that match the search term, as seen here to the right.



Each listing contains the name of the setting, what the setting's value currently is, and what type of variable the setting is. To change a setting, pass the name of the setting as the first argument to **/config**, followed by the new setting value. The new value will be checked to make sure it's valid, and if so, stored as the new setting's value. For example, to change the **show_windowbar** setting to "False", you could execute:

```
/config show_windowbar false
```

If a setting's value is a string, all arguments after the setting will be assumed to be part of the new value. For example, if using **/config** to change the default "away" message to "I'm busy right now!", you could execute:

```
/config away_message I'm busy right now!
```

Values are sanity checked[11] to make sure that they are the correct type. For example, if a value is intended to be a boolean (a "true" or "false" value), **/config** will not allow that setting to be set to an integer or a string.

/config can also be used to import and export configuration files. To export the current configuration file, pass **export** as the first argument to **/config**, followed by the filename to export the configuration to; if no filename is passed, the user will be prompted for a filename. To import a configuration file, pass **import** as the first argument to **/config**, followed by the filename of the configuration file to import; if no filename is passed, the user will be prompted for a file to import. After importing a configuration file, MERK should be restarted as soon as possible.

Not all settings can be edited with the **/config** command. The internal custom dictionary for the spellchecker, hotkeys, the application font, the "ignore" list, and the option to save *all* messages to the log cannot be edited with this command, due to the possibility of breaking MERK with badly formatted data. Most of these settings can be changed with the settings dialog, hotkey manager, or ignore manager dialogs. These settings will not be searchable or settable with **/config**, nor can they be viewed with **/config**.

A full list of all settings that can be changed with **/config**, along with their default values, can be found at the end of this document.

MERK should be restarted after changing settings with /config. Although many settings are applied instantly, some settings will only be applied after restarting. MERK can be restarted from scripts or as a command with /window restart.

11 *...a simple check to see if the produced material is rational (that the material's creator was thinking rationally, applying sanity).* Wikipedia

`/connect`, `/connectssl`, `/xconnect`, and `xconnectssl`

These commands are used to connect MERK to IRC servers:

- **`/connect`** connects to an IRC server, and does *not* execute any existing connection scripts
- **`/connectssl`** connects to an IRC server via SSL/TLS, and does *not* execute any existing connection scripts
- **`/xconnect`** connects to an IRC server, and executes any existing connection scripts
- **`/xconnectssl`** connects to an IRC server via SSL/TLS, and executes any existing connection scripts

Pass the hostname or IP address of the server as the first argument to these commands, and the port number to connect to as the second argument. These commands can be issued from the text input widget or from scripts. Please see ***Connecting to Servers*** for an example connection script that uses these commands. Call any of these commands without arguments to open the connection dialog.

All of these commands can take a hostID as a single argument. HostIDs are in the format **`host:port:password`**. If **`port`** is left out, the default port for the connection type will be used (6667 for TCP/IP, 6697 for SSL/TLS). If **`password`** is left out, no password will be used. **`password`** can contain spaces. So, to connect to Libera.chat via SSL/TLS, the hostID would be **`libera.chat:6697`**.

These commands will not automatically reconnect on disconnection. To ensure that MERK reconnects to the server on disconnection, add **`re`** before connect in all of these commands:

- **`/reconnect`** connects to an IRC server, and does *not* execute any existing connection scripts, automatically reconnecting on disconnection
- **`/reconnectssl`** connects to an IRC server via SSL/TLS, and does *not* execute any existing connection scripts, automatically reconnecting on disconnection
- **`/xreconnect`** connects to an IRC server, and executes any existing connection scripts, automatically reconnecting on disconnection
- **`/xreconnectssl`** connects to an IRC server via SSL/TLS, and executes any existing connection scripts, automatically reconnecting on disconnection

`context`

The **`context`** command is used to move a script's context to another context. It can take one or two arguments.

If **`context`** is called with one argument, pass the name of a window as the argument: the name of the channel or private chat. The command will search for windows associated with the context of the window the script is ran in, so it can't "find" windows associated with another server connection.

If **`context`** is called with two arguments, pass the hostID of the server connection to search, followed by the name of the window. So, to switch to the context of the **`#merk`** channel, and **`#merk`** is connected to the **`irc.libera.chat`** server on port **`6667`**, you'd call:

```
context irc.libera.chat:6667 #merk
```

/delay

The **`/delay`** command is used to delay a command's execution for a set amount of time. Pass the number of seconds to delay the execution of the command as the first argument, and all other arguments will be used as the command to execute. The command to be executed with **`/delay`** cannot contain any script-only commands, *even in scripts*. The command will fail with a "no command found" error. For example, to wait 5 minutes (which is 300 seconds) before changing your nick to **`merk_user`**, you could use:

```
/delay 300 /nick merk_user
```

goto and target

The **`goto`** command is extremely powerful, and allows a script to "jump" from one line to another line; that is, it can change the sequence of execution of a script. Using the **`insert`** command can alter the line count of a script, so **`goto`** should not be used to jump to a specific line number in scripts that use **`insert`**. Instead, if your script has the **`insert`** command, use the **`target`** command to jump to a target rather than a line number.

Using **`goto`** is easy: pass the line number (or target) you wish to "jump" to as the only argument. Execution of the script will immediately move to the desired line. For example:

```
1 /print This is the beginning of the script!
2 goto 4
3 /print This line will never be executed.
4 /print This line will ALWAYS be executed!
5 end
```

With the **`goto`** command in place, line 3 will never be executed, as the **`goto`** command skips right over it.

If you want to use **`goto`** in **`insert`**ed scripts, you can use the **`target`** command. This creates a "target" for the **`goto`** command. If you call **`goto`** with the name of the **`target`**, the script's execution will "jump" to the target:

```
goto goodbye
/print This will not execute.
target goodbye
/print Now we say goodbye!
```

“Targets” created with the **`target`** command must have a unique name in the script, and cannot contain spaces. If a **`target`** is created with the name of an existing **`target`**, an error will be displayed, and the script will not execute. **`target`** names are *not* case sensitive; “do_something” is the same as “Do_Something”. Due to the **`goto end`** shortcut usable with **`if`**, the only forbidden target name is “end”.

WARNING! There are no protections in place preventing a script from entering an infinite loop[12]. **`goto`** can lock up or crash MERK. **Please use `goto` carefully and sparingly.**

`if`

The **`if`** command allows MERK scripts to have a small amount of flow control[13]. It compares two values, and if the values' comparison is true, executes a command. The entity that sets how the comparison works is called the **operator**. MERK has eight operators:

Operator	Description
`(is)`	True if the first value is equal to the second value. Values are treated as strings, and are case in-sensitive.
`(not)`	True if the first value is *not* equal to the second value. Values are treated as strings, and are case in-sensitive.
`(in)`	True if the first value is contained in the second value; for example, **`o (in) pop`** evaluates to true because "pop" has "o" in it. Values are treated as strings, and are case in-sensitive.
`(nin)`	True if the first value is *not* contained in the second value; for example, **`o (nin) pop`** evaluates to false because "pop" has "o" in it. Values are treated as strings, and are case in-sensitive.
`(lt)`	True if the both values are numbers, and the first value is less than the second value. If either value is not a number, that value is treated like a string, and the length of the value is used.
`(gt)`	True if both values are numbers, and the first value if greater than the second value. If either value is not a number, that value is treated like a string, and the length of the value is used.
`(eq)`	True if both values are numbers, and the first value is equal to the second value. If either value is not a number, that value is treated like a string, and the length of the value is used.
`(ne)`	True if both values are numbers, and the first value is not equal to the second value. If either value is not a number, that value is treated like a string, and the length of the value is used.

The first value is passed as the first argument, followed by the **`OPERATOR`**, followed by the second value. All other arguments should contain the command to execute if the comparison is true. **`if`** can execute almost any command available, with one major exception: **`if`** *cannot execute any script-only commands other than* **`goto`**, **`halt`**, and **`msgbox`**. Any attempt to use

12 ...An infinite loop (or endless loop) is a sequence of instructions that, as written, will continue endlessly, unless an external intervention occurs, such as turning off power via a switch or pulling a plug. Wikipedia

13 ...Control flow (or flow of control) describes how execution progresses from one command to the next. Wikipedia

if to execute a script-only command besides **goto**, **halt**, or **msgbox** will result in a "Line contains no command" error, and script execution will end. If you wish to immediately end a script, you can **goto end**, which will immediately exit the script. You can use the **goto** command to jump to a line number or a target, and **halt** to ask the user if they want to end the script.

Values are tokenized like script arguments; values can have whitespace in them as long as they are contained in quotes. Values can also be mathematical statements, which are evaluated before comparing with the **if** statement's operator. If the command to execute on a “true” condition contains an apostrophe, contain the command in double quotes so that it is tokenized properly:

```
if o (in) word "/print That letter's in the word!"
```

/ignore

The /**ignore** command hides chat from a given nickname or user. However, messages from an /**ignore**d user are still received and logged, they are just not displayed. That user's chat is hidden from *all* chat displays, no matter what server they are on. You can pass a nickname (which will hide all chat from any user with that nickname) or a hostmask (which will hide chat from only users with that hostmask) to the /**ignore** command. The /**ignore** list is saved to the configuration file, and will be applied universally until the user is /**unignore**d. The **/ignore** list *cannot* be edited by the **/config** command; the only way to unignore a user is either through the right click user list menu, the **/unignore** command, or the ignore list manager.

The **/ignore** command can also be used with wildcards. Use ***** to substitute for any number of characters, and **?** to substitute for a single character. Users **/ignore**d in this way *must* be **/unignore**d with the **/unignore** command, or the ignore manager. Right clicking on an ignored user in the user list will not give an option to unignore the user. To show the user's messages again, either call **/unignore** with the specific entry used to ignore them as an argument (so, **/unignore *annoying.com*** in the example above), call **/unignore *** to clear the ignore list, or remove the entry from the ignore list with the ignore manager.

Call **/ignore** with no arguments to see the ignored user list in full. Attempting to add an entry to the ignore list that already exists will result in an error.

Entries in the ignore list can be added, removed, or edited with the GUI Ignore Manager, by selecting "Ignores" in the "Tools" menu.

`input`, `number`, `decimal`, and `msgbox`

The **`input`**, **`number`**, and **`decimal`** commands allow scripts to get input from users, and stores the input in an alias. Unlike all other commands, these three commands are blocking; that is, script execution will pause until the user either enters input or cancels the input dialog.

`input` gets string input from the user. Pass the name of the alias to store the input in as the first argument; the rest of the arguments will be used as the prompt to the user for the input. For example, the command:

```
input result What is your name?
```

This call results in this dialog:

If the user doesn't enter anything into the dialog or cancels the dialog, the resulting alias will contain **`*`**.

`number` works exactly the same way, only it prompts the user for a number in a range of numbers. Pass the name of the alias to store the number in as the first argument, the lower end of the range as the second, the higher end of the range as the third, and all other arguments as the prompt for the user. For example, the command:

```
number result 1 99 How old are you?
```

This call displays the dialog:

If the user cancels the dialog, the resulting alias will contain **`*`**. **`decimal`** works the same way as **`number`**, only it prompts the user for a decimal number.

`msgbox` shows a message box dialog, with a short message. This command is also blocking.

HTML can be used in the message to display, though this may lead to unpredictable results. Supported tags are generally HTML4; a list of supported tags for PyQt6 may be helpful.

insert

The **insert** script-only command reads in the contents of any file passed as an argument to it, and "inserts" it into the script where it is called. Any built-in aliases (see ***Built-In Aliases***) in the inserted script will reference the script being executed, not the script being **insert**ed, including any arguments passed to the calling script. For example, assume you have a script named **stuff.merk**, and it contains:

```
/print Hello from $_SCRIPT!
```

In another script, we use the **insert** command to insert this file into the script **test.merk**:

```
/print This is my main script!
insert stuff.merk
/print And now my script is complete!
```

Once processed, the script that will be executed will look like:

```
/print This is my main script!
/print Hello from test.merk!
/print And now my script is complete!
```

The **insert** command can be used to insert multiple files into a script; pass each file's name as a separate argument to **insert**, or issue **insert** multiple times. Arguments passed to the **insert** command are tokenized like script arguments, so filenames with spaces or single quotes (**'**) in them can be passed to **insert**, as long as they are contained in double quotation marks.

inserted files may contain **insert** as well, up to a maximum "depth" of 10 files. That is, a file can **insert** a file that calls **insert**, which can **insert** call a file that calls **insert**, which can **insert** a file that calls **insert**, which can **insert** a file that calls **insert**, and so on, up to a maximum of 10 "layers" of files that call **insert**. Changing this behavior is only possible by using the **/config** command on the **maximum_insert_file_depth** setting, or by editing **settings.json** directly with a text editor.

Using the **insert** command can alter the line count of a script, and thus the **goto** command should be used with **target** rather than line numbers in **insert**ed scripts.

*Do not use aliases in **insert** filenames.* It does not reliably work.

loop and pool

The **loop** command allows scripts to repeat sections of scripts without resorting to **goto** and **target**. Pass the number of times you want the code in the loop block to repeat as the first argument to **loop**, and any code from the line after the **loop** call until a call to the **pool**

command will be repeated that number of times. For example, to print “Hello!” to the current window three times, counting them as we go, you could use:

```
/alias counter 0
loop 3
/alias counter $counter+1
/print $counter) Hello!
pool
```

Any commands available can be used inside of a loop block. However, using **goto** inside **loop** blocks will immediately exit the block, and stop the **loop**. Using **if** with a **goto** as the command to execute on a true condition will also immediately exit the **loop** block as well.

Loops cannot be nested. If attempted, an error will be displayed and the script will exit.

/macro and /unmacro

The **/macro** command allows users to create their own "commands"; it creates a named command, just like MERK's other commands, that executes a script, passing any arguments to the script. Arguments to the **/macro** command are tokenized like arguments passed to scripts, so if you have an argument that contains whitespace in it, contain it in quotes. Macro names, much like aliases, cannot contain punctuation (with the exception of the underscore, _). They also cannot have the same name as existing commands.

As an example, we're going to create a macro named **/hello**. First, we'll write the script the macro will execute. Our macro will take one argument, a name, and send a message containing a greeting to that name to the current context. Any and all script commands can be used, including script-only commands:

```
restrict channel private
usage 1 /hello NAME
/msg $_WINDOW Hello, $_1!
```

Save this script to a file named **hello.merk** in MERK's "scripts" folder. Now that our script exists, we can create the macro for our command:

```
/macro hello hello.merk
```

Thanks to the **usage** script-only command, if we call our macro with less arguments than we need, the scripts usage text will be displayed. And thanks to the **restrict** script-only command, if this macro is called from a server window, the macro will not execute and an error will be displayed.

Arguments to macros are tokenized (and treated) just like arguments to scripts. In the example above, the command **/hello bob** is treated exactly like the command **/script**

`hello.merk bob` was called. If you wanted to send a greeting to a user that contains spaces in the "name" argument, use quotes to contain the argument; for example, to send a greeting to "Bob the Builder", you could use the **`/hello`** macro like:

```
/hello "Bob the Builder"
```

Macros are not saved by MERK, so they must be recreated every time MERK is started. An easy way to make sure that the macros you use are always available is to create them in your connection scripts.

The **`/macro`** command can take two optional arguments; these arguments set the text that is displayed in the command list displayed by the **`/help`** command. The first optional argument sets the usage text (displayed in the left hand column of the **`/help`** display), and the second optional argument sets the command description (displayed in the right hand column of the **`/help`** display). To modify the previous example to use the usage and help arguments, we could create the macro with:

```
/macro hello hello.merk "/hello NAME" "Says hello to another user"
```

This adds our usage and help information to the **`/help`** display:

It is not possible to pass the help text to the **`/macro`** command without passing the usage text first; while both arguments are optional, and usage text may be passed *without* help text, if help text is desired, usage text *must* be passed as an argument first. If both usage and help text arguments are omitted, the macro is still added to the **`/help`** display; usage is set to the name of the macro, and the help text is set to "Executes script ", with the name of the script the macro executes.

Macros *cannot* have the same name as existing commands. Macro names must also start with a letter, and not a number or other symbol. Multiple macros *cannot* have the same name; a new macro with the same name as an existing macro will "overwrite" the older macro.

Macros are treated like commands in a lot of respects. If autocomplete for commands is turned on, autocomplete will work for macros. Macros are added to the list of commands that displays when the **`/help`** is used, and macros will be highlighted just like commands in the text input widget. If scripting is turned off in settings, the **`/macro`** command will be disabled.

Call **`/macro`** with no arguments to see a list of currently defined macros.

Macros can be removed after creation with the **/unmacro** command, passing the name of the macro as the first argument. For example, to remove the macro created in the above example, you could use:

```
/unmacro hello
```

/print, /prints, and /warn

The **/print** command can be used to print text to the current or another window. There are two optional arguments: the server of the targeted window, and the name of the targeted window. The server argument it optional if the targeted window belongs to the same context. To print to a context's server window, use ***** as the window name; the server argument must be used to use **/print** or **/prints** in this way.

The **/prints** command works exactly the same way, only it prints a "system" message, like the messages emitted by most commands. The **/warn** command, like the others, prints an error message.

Both **/print**, **/prints**, and **/warn** can print HTML, and are not written to the log.

```
/rem This will print to the current window
/print Hello world!

/rem This will print to the #merk channel window.
/print #merk Hello world!

/rem This will print to a the server window in the current context
/print $_HOSTID * Hello, world!
```

/quit and /quitall

These commands are used to disconnect MERK from an IRC server. The **/quit** command disconnects from the IRC server associated with the context the command was issued in. It can be issued without an argument, or with a "quit" message as all arguments to the command; this will be used instead of the default "quit" message.

The **/quitall** works exactly the same way as **/quit**, only it will disconnect from *all* servers that MERK is currently connected to.

read, write, append, getfile and setfile

These commands enable file input/output for MERK scripts. **write** writes text to a file, overwriting the file if the file already exists, adding a newline to the text. Pass the filename to write to as the first argument, and all other arguments as the data to write. If the filename contains spaces, contain the filename argument in quotes; arguments to this command are

parsed like script arguments. If a the filename doesn't contain a full path, the filename will be written to MERK's current working directory[14]. **append** works the same way, only it appends the data to the file rather than overwriting it.

```
/rem This will write to a file in the scripts directory
write "$_DSCRIPTS/file.txt" "I'm writing this to file!"
```

Using the **read** command reads in the contents of a file and stores the contents in an alias. Pass the name of the alias as the first argument, and all other arguments as the filename to read from. The command looks for files much like how the **/script** command does.

```
/rem Read the file from the last example and print it
read file $_DSCRIPTS/file.txt
/print $file
```

getfile and **setfile** display an "open file" and "save file" dialog, respectively, and store the result in an alias. If a file is not selected or set, the alias is set to *****.

```
/rem Gets a file to read, and reads it into an alias
getfile file Select a file to read
if $file (is) * goto end
read contents $file
/print $contents

/rem Get a new filename to save the contents to
setfile out Select a file to write to
if $out (is) * goto end
write $out $contents
```

read, **write**, **append**, **getfile**, and **setfile** are all blocking commands.

restrict, only, and exclude

The **restrict** script-only command restricts a script's execution to a specific context. The first argument sets what type of context the script will function in: **server** restricts the script's context to server windows or connection scripts, **channel** restricts the script's context to channel windows, and **private** restricts the script's context to private chat windows. A restricted script will *not* execute in another context, and will show an error. Up to two context types can be passed, so **restrict private channel** would prevent a script from being executed in server windows. So, to restrict a script's execution to chat windows only, you could use:

```
restrict private channel
```

The similar **only** script-only command can be used to restrict a script's execution to specific contexts; for example, specific channels, server windows, or private chats. Pass the name of

14 Wikipedia

the window as an argument to the command; an unlimited number of arguments can be passed to the command. For example, to restrict a script's execution to channels named **#merk** or **#merkirc**, you could use:

```
only #merk #merkirc
```

The **exclude** script-only command works just like the **only** command, only it prevents a script from executing in specific contexts. Pass the name of the window as an argument to the command; an unlimited number of arguments can be passed to the command. To prevent a script's execution in any channel named #merk or #merkirc, you could use:

```
exclude #merk #merkirc
```

Aliases should not be used with these commands, as these commands are processed *before* any other commands are processed. If an alias is set in the same script before **restrict**, **only**, or **exclude** is called, those aliases will *not* be interpolated it into the input or the command.

/show, /hide, /close, and context

/show and **context** command switch contexts programmatically, but in slightly different ways. **/show** will show a window hidden with the **/hide** command, and also move focus to that window; that means that any commands issued without context will now be issued in that window's context. **/hide** simply "hides" a window without closing that window. A hidden window is no longer visible, but will still appear in the windowbar; it is *not* closed, and if the window's context is a channel, the client will still be present in that channel. **/show** and **/hide** can be issued from the text input widget. **context** can only be issued in scripts, and *completely* moves the script's context to the new window.

All these commands can take up to two arguments. Pass the name of the window to target as the first argument if the window shares a context (that is, the same server connection) as the window issuing the command. If the window to be targeted is in another context (a different server connection), pass the hostID as the first argument, followed by the name of the window to be targeted as the second argument. To target the server window with one of these commands, use ***** as the name of the window along with the hostID of the server, though this second argument is not needed with **context**.

For the following example, assume that we have two active connections: we are in the channels **#merk** and **#python** on **silver.libera.chat**, and the channels **#qt** and **#merk** on **lawnmower.undernet.org**. Our test script is executed in **#python**'s context, on **silver.libera.chat**:

```
/rem Here, we hide #merk's window in the current context
/hide #merk

/rem Now, we switch context to lawnmower.undernet.org's window
context lawnmower.undernet.org

/rem This shows the #merk window, only it's on the UnderNet server,
/rem not the Libera server
/show #merk
```

/close works exactly the same as **/show** and **/hide**, only instead of showing or hiding a window, it closes a window. If the window is for a channel, the channel will be left.

/size and /move

The **/size** and **/move** commands can be used to resize and move MERK subwindows, respectively. Much like **/hide** and **/show**, **/size**'s and **/move**'s first two arguments set what window the command is going to work on. Pass the name of the window as the first argument if the window shares the same context as the window the command is being executed in. If the window belongs to the context of another server, pass the hostID of the window's context followed by the window name as the first two arguments.

/size takes two additional arguments: the new width of the subwindow, and the new height of the subwindow. So, to set a subwindow's size to a width of 800 pixels, and a height of 600 pixels, you can use:

```
/size #merk 800 600
```

/move works similarly, and also takes two additional arguments: the new **X** and **Y** values of the subwindow. The **X** value sets where the top left corner of the subwindow will be located from left to right, and the **Y** value sets where the top left corner of the subwindow will be located from top to bottom. An error will be shown if invalid values are used (like negative numbers, or if it would move the window to where it would no longer be visible).

This example uses the **/move** command to move the subwindow for **#merk** on **tungsten.libera.chat** to a new location 950 pixels to the right, and 500 pixels from the top:

```
/move tungsten.libera.chat #merk 950 500
```

Call **/size** or **/move** with no arguments to see the size or location of the current subwindow.

/user

/user works exactly the same way as **/config**, only it edits the user configuration file. For most settings, this just changes default values. However, for **finger** and **userinfo**, this setting changes the values sent when MERK receives a CTCP FINGER or USERINFO request, respectively. To set a value to a blank string, pass ***** as the value to set.

/user can also be used to create and delete SASL account records. To add a SASL account, call **/user sasl add**, and pass the “hostID” of the server (**hostname:port**), the username, and the password as arguments. To delete an SASL account, call **/user sasl remove**, and pass the “hostID” to remove as the only argument. To edit an SASL account, call **/user sasl edit**, and pass the “hostID”, username, and password as arguments. To see a list of all SASL accounts, call **/user sasl** with no arguments.

/window

The **/window** command is used to manipulate the main application window, as well the subwindows "contained" in it. To move the main application window, pass **move** as the first argument to **/window**, followed by the **X** and **Y** values. To resize the window, pass **size** as the first argument to **/window**, followed by the **width** and **height**:

```
/rem Moves the main application window to the coordinates 200,200
/window move 200 200

/rem Resizes the main window to 1024x768
/window size 1024 768
```

To maximize, minimize, or restore the main application window, pass **maximize**, **minimize**, or **restore** to **/window** as the only argument, respectively.

Call **/window** without any arguments to see information about the application window and all the chat subwindows it contains.

/window can also be used to open dialogs and other types of subwindows. To open up the README, pass **readme** as the only argument to **/window**. To open up the settings dialog, pass **settings** as the only argument to **/window**. To open the log manager, pass **logs** as the only argument to **/window**. To open the log manager with only logs from certain targets, pass the name of the IRC network to show only logs from that network as an argument to **/window logs**, or pass a search term to show only logs with that term in the name of the channel or private chat to **/window logs**. You can also use **/window** to open up the hotkey, ignore, and plugin managers.

/window can also arrange subwindows in common patterns. Call with **cascade** as the only argument to cascade all subwindows, and call with **tile** as the only argument to arrange all subwindows in a tiled pattern. **/window** can also be used to "move" focus to another

subwindow. Call with **`next`** as the only argument to move focus to the "next" subwindow, and call with **`previous`** as the only argument to move to the "previous" subwindow.

MERK can be restarted, using the same command-line used to start the app, by passing **`restart`** as the only argument to **`/window`**.

```
/rem This cascades all subwindows
/window cascade

/rem This tiles all subwindows
/window tile

/rem This moves focus to the "next" subwindow
/window next

/rem This moves focus to the previous subwindow
/window previous

/rem This toggles full-screen mode
/window fullscreen

/rem This toggles "always on top" mode
/window ontop

/rem This restarts MERK with the same command-line used to start it
/window restart
```

The order subwindows are ordered in can be set in the settings dialog, or by changing the **`subwindow_order`** setting with the **`/config`** command. Valid values are **`creation`** (the default; the order subwindows were created in), **`stacking`** (the "visual" order the subwindows are in), and **`activation`** (the order subwindows have been activated in).

The **`/window`** command can also be used to install and uninstall plugins. Pass **`install`** as the first argument to **`/window`**, followed by the ZIP or Python file to install. Pass **`uninstall`** as the first argument to **`/window`**, followed by the name of the Python plugin file to uninstall. To see a full list of installed plugins, pass **`uninstall`** to **`/window`** without any other arguments.

```
/rem This shows a list of all installed plugin files
/window uninstall

/rem This installs the plugin ZIP file C:\away.zip
/window install C:\away.zip

/rem This installs the plugin Python file /home/user/away.py
/window install /home/user/away.py

/rem This uninstalls the plugin Python file away.py
/window uninstall away.py
```

Plugins installed with the **`/window`** command will *never* overwrite existing files in the **`plugins`** and **`scripts`** directories unless overwriting files on plugin installation is turned on in the settings dialog.

The **`/window`** command can also be used to pause and unpause plugins. To see a list of currently installed plugins and their status, call **`/window pause`** without any arguments. This will list each plugin's class name, the name and version of the plugin, the plugin's filename, and whether that plugin is currently paused or not. Pass a plugin's class name as the only argument to **`/window pause`** to pause a plugin, or to unpause an already paused plugin. The class name of the plugin is case sensitive; thus, calling **`/window pause plugin`** is *not* the same as calling **`/window pause Plugin`**. Make sure that the argument passed is the *exact* name of the plugin's class.

The **`/window`** command can also be used to create a MERK script that will "recreate" the current window layout. Calling **`/window layout`** without any arguments gets the size, location, and transparency of all currently visible windows (as well as the main application window), and creates a MERK script that will recreate this layout. The script is generated and opened in the Script Editor; it must be manually saved. To recreate your layout, execute the saved script; this will call **`/move`**, **`/size`**, and **`/fade`** on subwindows to recreate the layout. **Note**: if any of the windows do not exist when the script is ran later, this will raise an error and most likely end the script's execution.

The **`/window fade`** command can be used to set the transparency of the main application window. Call it with a percentage, from 1% visible to 100% visible. For example, to set the transparency of the main application window to 80%, call **`/window fade 80`**.

Scripting MERK

There are two types of scripts in MERK: connection scripts, and all other scripts. Each script runs in its own thread, and most commands are not blocking; each command is executed immediately after being issued, without waiting for the command's process to end or resolve. The only blocking commands are **`halt`**, **`input`**, **`msgbox`**, **`number`**, **`decimal`**, **`read`**, **`write`**, and **`random`**.

If the “all other scripts” category, there are three kinds of “other” scripts: the global script, channel scripts, and all other scripts.

Connection Scripts

Connection scripts are the scripts entered into the connection dialog, and are intended to be executed as soon as the client connects to the server. Unlike other scripts, they are stored in the user configuration file, and, outside of connection, can only be executed with the script editor. Connection scripts have the context of the associated server window created when connection begins (see ***Context*** and ***Writing Connection Scripts***).

All Other Scripts

All other scripts are, well, *scripts*: a list of commands, one per line, issued in order. Scripts have a context, which is the window that they are called from or executed in. They can be executed in several ways:

- From the "**Run**" button on a server window's toolbar. The script will be executed in the server window's context.
- From the "**Run**" entry in a window's input menu. The text in the text input widget will be replaced with a call to the **`/script`** command to execute the selected file.
- From the "**Run**" entry in a window's chat display right-click menu. The text in the text input widget will be replaced with a call to the **`/script`** command to execute the selected file.
- By issuing the **`/script`** command. The script will be executed in the window that the command was called from's context.
- From the "**Run**" menu in a script editor window. The user can select which context to run the script in, or optionally select to run the script on *all* windows simultaneously (with each window running that script in the window's context).

When using the **`/script`** command, scripts are searched for as outlined in ***Directories and Configuration Files***; if the script can be found in this directory search, the path to that script can be omitted. For example, if a script named **`example.merk`** is in the **`/scripts`** directory, calling **`/script example.merk`** will execute that script. If the file extension to the script is **`.merk`**, than the file extension can be omitted; **`/script example`** will also execute the script in the previous example.

Scripts can have comments. For single line comments, the **`/rem`** command can be used; the command does nothing, and any arguments to it are ignored.

The Global Script

If you want to set up aliases that are available to all scripts, create macros, or run specific commands the first time MERK connects to a server in a session, you can use the "global script". Create a script named **global.merk** in your scripts directory, and place any commands you'd like to execute in it. This script will be executed in the context of the first server MERK connects to's server window, before that server's connection script (if it has one), as soon as connection is complete, and will only be executed *once*. The script will be executed in a separate thread, just like other scripts, and has all commands available to it (including script-only commands); the only difference is that aliases created in **global.merk** will continue to exist after it completes execution.

Errors encountered in **global.merk** will halt (or prevent) execution, just like normal scripts.

If **global.merk** does not exist, it will not be executed when connecting to the first server; if **global.merk** is created *while* MERK is running, MERK will have to be restarted in order to execute it. If **global.merk** is not readable, an error will be displayed, but MERK will continue running. Two arguments will be passed to **global.merk**: the host and port of the server being connected to.

For example, let's assume that we use the same **NickServ** username and password on every server that we connect to (don't do this, it's a bad security practice). We want to create aliases for both of these that will be available to all other scripts. Create a script named **global.merk** in your scripts directory, and put this inside it:

```
/alias username my_username
/alias password my_password
```

Save this file, start up MERK, and connect to any server. Now, every script that you execute after connection can use the **$username** and **$password** aliases. To login to **NickServ**, all you have to do is enter:

```
/msg NickServ IDENTIFY $username $password
```

This behavior is enabled by default. It can be disabled on the "Scripting" section of the Settings dialog, or by using the **/config** command to set **execute_global_script** to **false**. Disabling the scripting engine will also prevent **global.merk** from executing.

The name of the file to execute, global.merk, can also be changed with the **/config** command, by setting **global_script_filename** to the desired filename. Either a filename with a complete path can be used, or if the script is in the **scripts** directory, the filename without path can be used.

If the global script does not exist or cannot be found or cannot be read, MERK will run normally.

Channel Scripts

Channel scripts are scripts that, if they exist, are executed as soon as a channel window completes loading. That is, when MERK joins a channel, and creates the channel window for that channel, the script will execute in that window's context. Channel scripts are "bound" by the IRC network the channel is on, and will execute no matter what server that MERK is connected to. The script will execute *every time* the channel is joined or re-joined in a session.

To create a network script, select "Edit channel script" from either the channel window's right click menu. This will open up a script editor window to edit the channel script, which you can save. To create one "manually", create a file in the **`scripts`** directory with the name of the channel's network (in lower case), followed by a "-", followed by the channel name, with the file extension ".merk". So, to create a channel script for the channel **`#merk`** on the EFnet IRC network, you would name the file **`efnet-#merk.merk`**.

Channel scripts are just like normal scripts, and can use all the commands that regular scripts can, including script-only commands. If scripting is disabled, channel scripts will not execute. Aliases created in a channel script will be "destroyed" when the script ends, just like normal scripts. Errors in channel scripts can prevent script execution, just like normal scripts. To remove a channel script from a channel, delete the file from the **`scripts`** directory. If a channel script is completely "empty" (it contains no commands and only whitespace), MERK will not execute the channel script.

Channel scripts are passed two arguments: the channel name (the first argument, referenced by the **`$_1`** alias in the script), and the host ID of the server the channel is connected to (the second argument, **`$_2`**).

Channel script execution is enabled by default. Channel scripts can be disabled in the settings dialog, or by using **`/config`** to set **`execute_channel_scripts`** to "false".

Errors

For the most part, MERK handles script errors in two ways. Errors in most script-only commands will prevent execution, while errors in other commands will halt the script execution when the error is encountered. If any of the following script-only command errors or conditions are detected, ***the script will not execute at all***. Entries marked with a ***** will not display what file the error occurred in or the line number in the error message:

- Calling **`usage`** with the wrong number of arguments
- Calling **`usage`** with a non-number as the first argument
- Calling **`restrict`** with the wrong number of arguments
- Calling **`restrict`** with an argument that is not server, channel, or private
- Calling **`only`** with no arguments
- Calling **`end`** with any arguments
- Calling **`target`** with the name of a target that already exists*
- Calling **`target`** without any arguments or more than one argument*
- Calling **`target`** if **`goto`** is disabled
- Calling **`insert`** no arguments*
- Calling **`insert`** with a file that cannot be found, doesn't exist, or can't be read*
- Errors tokenizing **`insert`** arguments*
- Calling **`context`** with a context that doesn't exist
- Calling **`/alias`** if aliases have been disabled
- Calling **`/alias`** with the wrong number of arguments
- Calling **`/unalias`** in a script
- Calling **`if`** if **`if`** has been disabled
- Calling **`if`** with the wrong number of arguments
- Calling a script-only command other than **`goto`** or **`halt`** as the result of a successful **`if`** call
- Errors tokenizing **`if`** arguments*
- Calling **`insert`** if **`insert`** has been disabled
- Calling **`goto`** if **`goto`** has been disabled
- Calling **`goto`** with too many arguments
- Calling **`loop`** with too many arguments
- Calling **`pool`** with any arguments
- Calling **`pool`** without being “in” a **`loop`** block
- Calling **`read`** if **`read`** has been disabled
- Calling **`read`** with the wrong number of arguments
- Calling **`write`** if **`write`** has been disabled
- Calling **`write`** with the wrong number of arguments
- Calling **`append`** if **`append`** has been disabled
- Calling **`append`** with the wrong number of arguments
- Calling **`read`** if aliases are disabled
- Calling **`random`** if aliases are disabled
- Calling **`random`** with the wrong number of arguments
- Calling **`input`** if aliases are disabled
- Calling **`input`** with the wrong number of arguments
- Calling **`number`** if aliases are disabled
- Calling **`number`** with the wrong number of arguments
- Calling **`decimal`** if aliases are disabled
- Calling **`decimal`** with the wrong number of arguments
- Calling **`hostmask`** if aliases are disabled
- Calling **`hostmask`** with the wrong number of arguments
- Calling **`escape`** if aliases are disabled
- Calling **`escape`** with the wrong number of arguments
- Calling **`message`** with the wrong number of arguments
- Calling **`getfile`** if aliases are disabled
- Calling **`getfile`** with the wrong number of arguments
- Calling **`setfile`** if aliases are disabled
- Calling **`setfile`** with the wrong number of arguments
- Calling **`context`** with the wrong number of arguments
- Executing a script in a context that is not allowed by **`restrict`**
- Executing a script in a context that is not allowed by **`only`**
- Executing a script in a context that is not allowed by **`exclude`**

Every other command error will display an error message (if script error messages are turned on in settings), and halt execution of the script. For example, error messages will be displayed for:

- Lines that do not contain a command
- Lines that start with **`/`** and are not followed by a valid command
- Calls to commands with an incorrect number of arguments
- Calls to commands with invalid arguments
- Errors in command execution
- Calling **`goto`** with a target that doesn’t exist
- Calling **`read`** on a binary file, or a file that doesn’t exist
- Calling **`wait`** with a non-number argument
- Calling **`loop`** with a non-number argument

This is not a complete list of errors that may be shown. If an error is encountered, an error message with be displayed. The error message will contain:

- **The name of the file the error is located in**. If the error is in a connection script, the filename displayed will be **`SERVER:PORT`** for the server's connection script. If the filename is not known or does not exist, the filename will be set to **`script`**. Errors in calling the **`insert`** command will *not* contain the filename the error occurred in, only what the erroneous call was. The filename displayed will not contain a path to where that file is located (as, most likely, it will be in MERK’s **`scripts`** directory).
- **The line number the error occurred on**. On files that contain the **`insert`** command, the line number may not be accurate, as the script will include the **`insert`**ed file. Blank lines in scripts may not be counted accurately, leading to line numbers being “estimated”.
- **A description of the error**.

Some errors mention using “double quotation marks”. By this, MERK means the **`"`** character.

Context

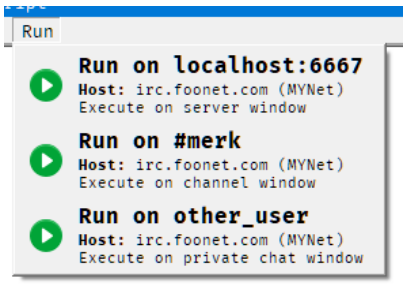
A script or command's *context* is a reference to the window the script or command is being executed in. Context is, for the most part, only necessary for scripts; the context for any commands issued by a window's text input widget is the window the command is being issued in.

Some commands can ignore an argument if they are for the current context; for example, when issuing the **`/part`** command, you can ignore the **`CHANNEL`** argument if the command is intended to be executed in the current window's context:

```
/rem This leaves the current channel
/part

/rem This invites a user to the current channel
/invite my_friend

/rem This kicks a user from the current channel
/kick my_enemy

/rem This gives a user operator status in the current channel
/mode +o my_friend

/rem This sets the topic in the current channel
/topic Welcome to my channel!
```

Context-less commands should *not* be issued by scripts, as it can get confusing if you run the script in the wrong context. However, if the script is being ran in any window's context, context-less commands are available to the script.

When running a script from a window, either through the server window's toolbar, or the input menu, the script is always ran in that window's context. The script editor "Run" menu allows you to choose which context to run the script in. If MERK is connected to more than one server, and in more than one channel or private chat on each server, the "Run" menu will give options to run the script in all of each context; for example, you can run a script in all connected channels.



*An example "Run" menu from the script editor. The client is connected to a server on **`localhost:6667`**, and is in the channel **`#merk`** while having a private chat with **`other_user`**. Each selection will run the script in the specified context.*

A window's context is "connected" to any other window contexts that share the same network stream. Commands issued from the text input widget in server or chat windows can only effect other windows that share the same server connection. This is called a "shared context".

For example, let's assume that MERK is connected to two servers, **irc.example.com** and **irc.other.net**. On **irc.example.com**, MERK is connected to two channels, **#merk** and **#python**. On **irc.other.net**, MERK is connected to the channel **#qt**. In this example, **#merk** and **#python** have a shared context, while **#qt** doesn't have a shared context with the other two window. Commands issued in **#qt** will not be able to have an effect on **#merk** or **#python**, and vice versa.

Scripts can use the **context** command to "move" the script to another context. The **context** command will search for all windows, no matter what context. The order **context** looks for windows is:

1. Windows that have a shared context with the context the script was executed in.
2. Windows from all contexts.
3. Server windows.

context will move to the *first* window it finds with the name passed to it. If you are in multiple channels with the same name, this may be problematic. To specify what server a window is connected to, pass the hostID of a connected server as the first argument to **context**.

If a script is intended to only be ran in a specific type of context, the **restrict** command can be used, in one of three ways:

- **restrict server** – The script will only run in server windows or connection scripts
- **restrict channel** – The script will only run in channel windows
- **restrict private** – The script will only run in private chat windows

A script that has been "restricted" will *only* run in the context specified, and will not execute and show an error if it is ran in another context. Up to two contexts can be passed to **restrict**. For example, to make sure that a script is ran *only* in chat windows, use **restrict channel private**.

Outside of scripts, it *is* possible to "move" to another window's context via commands. The **/show** command can move to the context of any other window.

Using context-dependent built-in aliases in script-only commands can sometimes result with the built-in aliases of the window the script is being ran in being used, rather than the built-in aliases of the “new” context switched to with the **context** command. If this problem arises, a workaround is to place the offending code in a separate script and execute it with the **/script** command in the “new” context.

Aliases

Aliases are tokens that can be created to insert specific strings into your input in the client (if the "Interpolate aliases into input" setting is turned on, which is the default) or into your scripts. They function kind of like variables[15] do in programming languages. As an example, let's create an alias named 'GREETING', and set it to the value 'Hello world!':

```
/alias GREETING Hello world!
```

Now, if you want to insert the string "Hello world!" into a command or any output, you can use the alias interpolation symbol, which is **$** by default, followed by the alias's name, to insert your alias into the command or output:

```
/msg #mychannel $GREETING
```

This sends a message to **#mychannel** that says "Hello world!" to everyone in the channel!

Alias names *must* start with a letter, and not a number or other symbol. This is to prevent overwriting built-in aliases created for each window's context (see ***Built-In Aliases***). Alias names also cannot contain punctuation, except for underscores. To create an alias, use the **/alias** command. To see a list of all aliases set for the current window, issue the **/alias** command with no arguments. Aliases created with the **/alias** command in the text input widget are *global* in scope; that is, they are available to all and every script executed on the client after they are created, as well as any command entered into the text input widget. Aliases created in *scripts*, however, are not; they are “destroyed” when the script completes execution. The **/alias** command can only edit aliases in the same scope; so, **/alias** entered into the text input widget can only edit aliases that were entered into the text input widget, and **/alias** used in a script can only edit aliases created in that script, and cannot be used to edit aliases in other scripts. The only exception to this rule is that aliases created in the global script are global in scope, and can be edited in the text input widget of server, channel, or private chat windows, but cannot be edited from scripts.

Aliases created with the **.alias()** inherited method in plugins are global in scope. They will become immediately available for use in the text input widget and scripts. If a script is running at the time the alias is created, it most likely will not be available for use while the script is still running.

Aliases can also be used as macros, and can contain an entire command. For example, let's say that you like to issue a greeting to everyone that enters a channel, but typing **/msg #mychannel Hello, and welcome!** is a pain to type every time someone joins, you could create this alias:

```
/alias GREETING /msg $_WINDOW Hello, and welcome!
```

15 https://en.wikipedia.org/wiki/Variable_(computer_science)

Now, whenever someone joins your channel, just type **`$GREETING`** into the text input widget to send your message! The above example uses a built-in alias, which is explained in ***Built-In Aliases***.

Aliases can be deleted manually with the **`/unalias`** command. Built-in aliases cannot be deleted with the **`/unalias`** command, and will display an error. As script-created aliases are “destroyed” on the end of the script, calling **`/unalias`** in a script does nothing, and will result in an error that prevents the script from running.

Built-In Aliases

Each window has a number of aliases for use that are built-in to the window's context, and do not require the user to create them. Built-in alias names start with an underscore (_) and are all uppercase. Some built-in aliases (see ***Script Arguments***) are only created in certain circumstances; these have a gray background.

Alias	Value
_0, _1,...	Any arguments that have been passed to the currently running script; the first argument will be set to **$_1**, the second argument will be set to **$_2**, and so on. **$_0** will contain all arguments, separated by spaces; if no arguments have been passed, **$_0** will be set to **none**.
_ARGS	The number of arguments passed to a script. If no arguments have been passed to the script, this will be set to **0**.
_CLIENT	The name of the IRC client, MERK.
_CONNECTION	If the connection type of the server the window is connected to; if connected via SSL/TLS, this will be set to **SSL/TLS**, otherwise it will be set to **TCP/IP**.
_CONNECTED	The hostIDs of all the servers MERK is connected to, separated by spaces.
_COUNT	The number of users in the current channel. For server and private chat windows, this will be set to **0**.
_CUPTIME	The number of seconds that MERK has been running.
_HCHANNELS	The number of hidden channels on the server the window is connected to. If this is not known, this will be set to **0**.
_HOSTID	The “hostID” of the server the window is connected to; this will be the host name and port number used to connect to the server, separated by a colon (**server:port**).
_HOSTNAME	The hostname of the server the window is connected to. If the hostname is not known, this will be set to the server’s “hostID’.
_DATE	The current date, in "MM/DD/YYYY" format.
_DAY	The current day of the week.
_DLOGS	The directory MERK is storing logs in.
_DONATE	The URL to donate to MERK’s development.
_DPLUGINS	The directory MERK is using to store and load plugins.
_DSCRIPTS	The directory MERK is using to store and load scripts.
_DSETTINGS	The directory MERK is using to store settings.
_DSTYLES	The directory MERK is using to store text style files.
_EDATE	The current date, in "DD/MM/YYYY" format.
_EPOCH	The current time in UNIX epoch format.
_FILE	The full filename of the currently running script. If called from an **/insert**ed file, this will contain the name of the script being executed, not the **/insert**ed file. If the current script does not have a filename, this will be set to **script**.
_GLOBAL	If the global script has been executed, this will be set to **1**; if the global script has *not* been executed, this will be set to **0**.
_LATEST	The URL to a directory containing the latest development builds of MERK.
_MODE	Any modes set on the user associated with the window. If no modes are set, this will be set to **none.**
_MONTH	The name of the current month.

Alias	Value
_NETWORK	The network the server the window is connected to is on; if this is not known, this will be set to **unknown**.
_NICKNAME	The user's current nickname.
_ORDINAL	The current day of the month.
_PORT	The port on the server the window is connected to.
_PRESENT	If the window the alias is being used in is a channel window, this will contain a list of users in that channel, separated by commas. If there are no users in the channel, or if this is used in a non-channel context, this will be set to **none**.
_REALNAME	The user's realname, as set in user settings.
_RELEASE	The URL for the current latest release of MERK, as known at the release of the running version.
_RVERSION	The current release version of MERK, as known at the release of the running version.
_SCHANNELS	The number of visible channels on the server the window is connected to. If this is not known, this will be set to **0**.
_SCOUNT	The number of visible users on the server the window is connected to. If this is not known, this will be set to **0**.
_SCRIPT	The name of the file, without the full path, of the currently running script. Only present in scripts that have been executed with the **/script** command. If called from an **/insert**ed file, this will contain the name of the script being executed, not the **/insert**ed file. If the current script does not have a filename, this will be set to **script**.
_SERVER	The server the window is connected to; this will be the address used to connect to the server, not the server's reported hostname.
_SOFTWARE	The server software the server the window is connected to is using. If this is not known, this will be set to **unknown**.
_SOURCE	The URL to MERK's source code.
_STAMP	The current time, following the format setting for timestamps.
_STATUS	If the window is associated with a channel, this will contain the window's channel status (**operator**, **voiced**, etc.); otherwise, this will be set to **normal**.
_SUPTIME	How long, in seconds, the client has been connected to the server associated with the current window.
_TIME	The current time, in 24-hour format.
_TOPIC	If the window the alias is being used in is a channel window, this will contain the channel's topic, if there is one. If the channel does not have a topic, or if the window is not a channel, this will be set to **No topic**.
_UPTIME	How long the window the script is being ran in has been connected or has been in use, in seconds.
_VERSION	The current version of MERK in use.
_USERNAME	The user's username, as set in user settings.
_WEB	The URL of the MERK website.
_WINDOW	The name of the window the script is being used in.
_WTYPE	The type of window the alias is being used in; either **server** for server windows, **channel** for channel windows, or **private** for private chat windows.
_YEAR	The current year.

Script Arguments

Arguments can be passed to a script with the **/script** command; just pass them as arguments to the command following the script file name. The **/script** command is the *only* way to pass arguments to a script.

Script arguments are tokenized[16] differently than arguments for commands. In arguments for most commands, arguments are considered to consist of either a single word, with no spaces, or a number of words separated by spaces. A channel name, for example, or a nickname may be an argument; the **/msg** command looks for a channel or nickname as the first argument, with all other arguments being the message to be sent. Arguments to scripts can contain spaces, and the number of arguments is important. To use an argument with spaces in it, or single quotation marks (**'**), contain the argument with quotation marks.

```
/rem This calls a script with a single argument
/script myscript.merk "Hello, world!"

/rem Here are multiple arguments, with spaces in each
/script test.merk "First argument!" "Second argument!" "It's the third one!"
```

To access these arguments, a built-in alias is created for each one, and another is created for all arguments. Each built-in alias is named with the number of the argument: **$_1** for the first argument, **$_2** for the second, **$_3** for the third, and so on. The built-in alias **$_0** contains all arguments passed to the script, joined by single spaces; if no arguments have been passed to the script, **$_0** will contain the string **none**.

To make sure your script is called with the right number or arguments, use the **usage** script-only command. As the first argument to **usage**, pass the number of arguments your script requires; if your script should take one or more arguments, pass **+** as the argument. All arguments after this first will be displayed as the error message if your script it called with an improper number of arguments.

As an example, let's write a script that sends a greeting to someone in the current chat. Our script will require a single argument, a name. When executed with the right number of arguments, it will send the greeting to chat, and if executed with too few arguments, will tell the user how to use the script. Open the script editor, and paste the following code into it, saving the file as **greet.merk**.

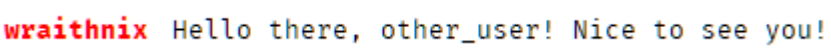
```
/rem This script requires a single argument.
/rem If none or more than one argument is passed,
/rem display script usage information.
usage 1 Usage: /script $_SCRIPT NAME

/msg $_WINDOW Hello there, $_1! Nice to see you!
```

16 https://en.wikipedia.org/wiki/Lexical_analysis#Tokenization

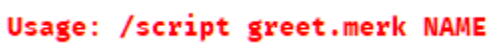
Let's execute our script! In the text input widget, type the following and hit enter:

```
/script greet other_user
```

Our greeting is sent to the current chat:



If we executed our script with no arguments, an error message is displayed:



Connection scripts are *never* called with any arguments. Scripts executed by any way other than the **`/script`** command are *never* called with any arguments, with the exception of the global script and channel scripts.

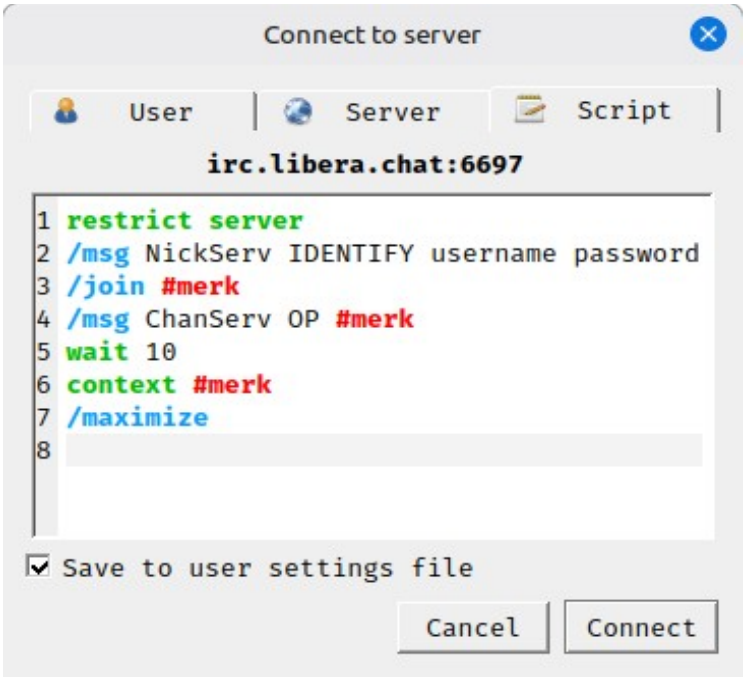
Several built-in aliases are created for scripts. **`$_FILE`** contains the full filename of the script (including path) being called, **`$_SCRIPT`** contains the filename of the script without the path, and **`$_ARGS`** contains the number of arguments the script was called with. The example above has a use of the **`$_SCRIPT`** built-in alias. If a script is executed from the script editor (and thus doesn't have a filename), both **`$_FILE`** and **`$_SCRIPT`** will be set to **`script`**.

Scripts executed with the "Run" menu in the editor are *never* called with any arguments, and will only have the **`$_SCRIPT`** or **`$_FILE`** built-in aliases if the script has been saved to or loaded from a filename. Scripts executed with the "Run script" button on server window toolbars will have the **`$_SCRIPT`** and **`$_FILE`** built-in aliases, but as they are *never* called with arguments, will have the **`$_ARGS`** built-in alias set to **`none`**.

Argument built-in aliases can be used with **`insert`**ed scripts, but they will always reference the script that is being executed, not the **`insert`**ed script. For example, if an **`insert`**ed script includes the built-in alias **`$_1`**, that alias will interpolate to the first argument that was passed to the script that is being executed, *not* the **`insert`**ed file.

Writing Connection Scripts

Connection scripts are the scripts that can be entered in the connection dialog, and are executed as soon as the client completes connecting to a server. A connection script's context is the server window created when connecting. In fact, calling **`restrict server`** to restrict the connection script's execution to a server window will pass successfully, while using **`restrict`** with any other context type will cause the connection script to not execute.



To issue commands that will have an effect on another window, use the **`context`** command to move the script to that window's context.

Before **`context`**ing to another context, be aware that that window (and the context) may "not exist" yet. The channel window may not be rendered yet, the private chat that you intended to start has not started yet, etc. The **`wait`** command will help you in these situations, so you can make sure that all the contexts for your script have been created before you issue commands.

For example, let's say that when you connect to your favorite server, automatically join your favorite channel, **`#merk`**, say hello, and maximize the channel window. Your connection script might look like:

```
/alias FAVORITE #merk
/join $FAVORITE
/msg $FAVORITE Hello, everybody!
wait 10
context $FAVORITE
/print $_HOSTID $_WINDOW Maximizing $FAVORITE!
/maximize
```

How long to **`wait`** after connection will take some trial and error, due to many factors: the speed of your Internet connection, the speed of your computer, how busy the server is, how big of a log the client is loading for display, among other things. When in doubt, a longer **`wait`** is preferable to a shorter one, to make sure that your script executes properly. When first writing a connection script, try **`wait 30`** to pause the script for 30 seconds, and tweak from there.

Example Scripts

Wave

This is a simple script that sends an emoji to the current chat, and adds a shortcut to executing the script. When executed in a channel or private chat window, it will send the "wave" emoji to the current chat. When executed in a server window, it will ask for a target to send the “wave” emoji to.

```
/rem Wave Script
/rem By Dan Hetrick

if $_WTYPE (is) server goto server
/msg $_WINDOW :wave:
end
target server
input t Where do you want to send the wave?
if t (is) * goto end
/msg $t :wave:
```

What this script does specifically:

1. Checks what kind of window the script is being executed in. If the window is a server window, the script jumps to step 4.
2. Sends the “wave” emoji to the current chat…
3. ...and ends the script.
4. Sets a **`goto`** target.
5. Asks the user who to send the wave to, either a channel or user.
6. If the user didn’t enter anything or canceled the dialog, exits the script.
7. Sends the “wave” emoji to the user or channel the user entered.

Greeting

This script sends a greeting to the as a private message to a user. It takes a single argument, the username of the person the greeting is being sent to.

```
/rem Greeting Script
/rem By Dan Hetrick

usage 1 Usage: /script $_SCRIPT nickname
/msg $_1 Hello! Nice to see you!
```

What this script does specifically:

1. Makes sure the script is called with a single argument
2. Sends a greeting as a private message to the user set in the single argument

To make this a little easier, we'll create a macro for our greeting script. Assuming that our greeting script was saved to a file named **`greeting.merk`** in MERK's "scripts" directory:

```
/macro greet greeting "/greet USER" "Sends a greeting to the current chat"
```

Now, we can call our script with a new custom "command", **`/greet`**. So, if we wanted to send a greeting to a user named "Bob":

```
/greet Bob
```

This would send a message to the user "Bob" that consists of "Hello! Nice to see you!".

Example Connection Script

This script should be set as a connection script. Upon connection to the server, it will log the user into **NICKSERV**, join a channel the user owns, tell **CHANSERV** to give them operator status in the channel, set the channel topic, maximize the channel's window, and send a greeting to the channel

```
/rem Example Server Connection Script

restrict server
/alias USERNAME my_username
/alias PASSWORD my_password
/alias CHANNEL #my_channel
/alias TOPIC Welcome to my channel!
/alias GREETING $_NICKNAME is here, everybody!

/msg nickserv IDENTIFY $USERNAME $PASSWORD
wait 5
/join $CHANNEL
/msg chanserv OP $CHANNEL
wait 10
context $CHANNEL
/topic $TOPIC
/maximize
wait 1
/msg $_WINDOW $GREETING
```

What this script does specifically:

1. Restrict the script's execution to server windows
2. Sets an alias for the user's **NICKSERV** username
3. Sets an alias for the user's **NICKSERV** password
4. Sets an alias for the user's channel
5. Sets an alias for the channel's topic
6. Sets an alias for the greeting to send once everything else is done
7. Logs into **NICKSERV** with the set username and password
8. Waits 5 seconds
9. Join the set channel
10. Tells **CHANSERV** to give the user operator status in the set channel
11. Waits 10 seconds
12. Switches contexts to the channel window
13. Sets the current channel's topic
14. Maximizes the current channel window
15. Waits 1 second
16. Sends the greeting to the current channel

Inserting Files

If there's data or aliases that you want to use in more than one script, the **/insert** command makes that easy. In the last example, a script was used to login to **NICKSERV**. In this example, we're going to store our login information in one script, and use it in another.

login.merk

```
/rem NICKSERV Login

/alias USERNAME my_username
/alias PASSWORD my_password
/alias LOGIN_TO_NICKSERV /msg NICKSERV IDENTIFY $USERNAME $PASSWORD
```

Now, to login to **NICKSERV** from another script, use the **/insert** command to insert this file into the script, and issue the full command:

```
insert login.merk
$LOGIN_TO_NICKSERV
```

Once all aliases have been interpolated into the script, this will end up being the script that is executed:

```
/msg NICKSERV IDENTIFY my_username my_password
```

Connecting to Servers

Scripts can call other scripts, allowing scripts to be "chained"; that is, to execute one after the other. This script is an example of a connection script that connects to multiple servers, and executes multiple scripts.

First, let's create our initial connection script. We're going to use it upon connection to UnderNet. It will login to our X account before connecting to two other servers.

```
restrict server
/msg X@channels.undernet.org login username password
/join #merk
/xconnect palladium.libera.chat 6667
/xconnectssl irc.underworld.no 6697
```

This script:

1. Restricts the script's execution to server window contexts.
2. Sends a private message to UnderNet's user service bot, logging into an account.
3. Joins the **`#merk`** channel.
4. Connects to **`palladium.libera.chat`** on port 6667, executing any existing connection script when it connects.
5. Connects to **`irc.underworld.no`** via SSL/TLS, on port 6697, executing any existing connection script when it connects.

Scripts can also execute other scripts with the **`/script`** command.

Custom Day-of-the-Week Away Message

This script sets MERK's default away message to a custom message depending on what day of the week it is.

```
if $_DAY (is) Monday goto monday
if $_DAY (is) Tuesday goto tuesday
if $_DAY (is) Wednesday goto wednesday
if $_DAY (is) Thursday goto thursday
if $_DAY (is) Friday goto friday
if $_DAY (is) Saturday goto saturday
if $_DAY (is) Sunday goto sunday
/warn $_HOSTID * $_DAY is not a recognized day!
goto end
target monday
/config away_message I hate Mondays
goto done
target tuesday
/config away_message At least it's not Monday
goto done
target wednesday
/config away_message It's Wednesday, my dudes
goto done
target thursday
/config away_message It's almost Friday...
goto done
target friday
/config away_message It's Friday!
goto done
target saturday
/config away_message I love Saturday!
goto done
target sunday
/config away_message It's almost Monday...
```

This script uses **`if`** and the built-in alias **`$_DAY`** to check what day of the week it is, and sets the default away message, using **`goto`** and **`/config`**, with a special message for each individual day. If, for some reason, the name of the day of the week stored in **`$_DAY`** is not recognized, the script uses the **`/warn`** command to show an error message to the user on the current server window. When each day is handled, the script uses **`goto`** to jump to the “done” **`target`**, ending the script. The reason why the script doesn’t use **`end`** is so that this script can be **`insert`**ed into a connection script without stopping the connection script from continuing to execute.

This script can be rewritten to simply call **`/config`** on each **`if`** call, rather than using **`goto`**.

To make sure that this script is ran every time you start up MERK, this can be **`insert`**ed or entered into the global script.

"Trout" Macro

Many IRC clients have had the ability to create a "trout" command or macro: a command that takes a single nickname as an argument, and sends a CTCP action message to the current chat that says "slaps NICKNAME with a trout". Here's how you can do that in MERK!

First, we're going to write a script that takes a single argument and uses it to send a CTCP action message to the current chat. We'll save this script to a file named **`trout.merk`** in your scripts directory:

```
restrict channel private
usage 1 Usage: /trout NICKNAME
/me slaps $_1 with a trout
```

Now that we have our script, we'll use it to create a macro. We can put this command into the text input widget, a connection script, or another script:

```
/macro trout trout.merk "/trout NICKNAME" "Slaps someone with a trout"
```

Our "trout" macro is complete! Now, we can use our macro just like a command in the text input widget, scripts, or hotkeys. For example, to "slap" a user named "Bob", we'd use:

```
/trout Bob
```

This will sent a CTCP action message to the current chat window that says "[YOUR NICKNAME] slaps Bob with a trout".

Annoying Script

This script (which you shouldn't use) annoys a user. More importantly, it shows how to use the **`input`**, **`loop`**, **`random`**, and several other commands.

First, we prompt the user for a target to annoy. Then, we will send a notice to them that simply says "poke!" at random intervals for a random amount of time.

```
input person Who do you want to annoy?
if $person (is) * goto end
random length 5 20
loop $length
random w 10 300
wait $w
/notice $person poke!
pool
msgbox Annoyance complete!
end
```

What this script does is:

1. Asks the user to **`input`** a target to annoy.
2. If the user doesn't enter a name or channel, or cancels the dialog, the script exits.
3. Selects a **`random`** number between 5 and 20. This is how many times the script will send the annoying message.
4. Start a **`loop`**, repeating a number of times selected in the last step.
5. Select a **`random`** number from between 10 (10 seconds) and 300 (5 minutes).
6. Pause the script execution with the amount of time selected in the last step.
7. Send the target the "poke!" notice.
8. Jumps back to step 5, until we're reached the number of loops to execute.
9. Prints a message to the window the script was launched from telling the user the script has finished executing, and exits.

We can change the script to use a default target (we'll use "Bob"), and ask the user if they want to use that instead with the **`halt`** command. If the user presses "Ok" on the **`halt`** dialog, the script will end without annoying anyone:

```
input person Who do you want to annoy?
if $person (is) * goto default
target continue
random length 5 20
loop $length
random w 10 300
wait $w
/notice $person poke!
pool
msgbox Annoyance complete!
end
target default
halt You didn't enter a target. Press "Cancel" to annoy Bob!
/alias person Bob
goto continue
```

Plugins and Plugin Development

MERK plugins are written in Python (just like MERK), and can use any libraries accessible to Python. If MERK is ran on Windows using the PyInstaller distribution, plugins are limited to the Python standard library, Qt5, and Twisted.

MERK plugins are stored in the **`plugins`** directory, and are loaded automatically when MERK starts. Each **`.py`** file in the **`plugins`** directory can contain one or more MERK plugin classes. Each Python module should only contain only *one* **`Plugin`** class, but there's no mechanism to prevent placing multiple **`Plugin`** in the same file. To install a plugin, just copy the plugin's Python file into the **`plugins`** directory, use the **`--install`** command-line option, drag-and-drop a plugin ZIP file onto the main application window, or use the plugin manager. To uninstall a plugin, either delete the Python module from the **`plugins`** directory, use the **`--uninstall`** command-line option, or use the plugin manager. To uninstall *all* plugins, use the **`--uninstall all`** command-line option. Call **`--uninstall`** without any arguments to see a list of installed plugin filenames. The **`/window`** command can also be used to install and uninstall plugins.

The plugin API[17] gives users direct access to the instances of the Twisted IRC client objects that MERK uses to communicate with IRC servers, and send and receive messages. Although knowledge of how the Twisted API functions should not be necessary to write plugins, it makes functionality that would otherwise be impossible possible. The Twisted IRC Client documentation can help with performing advanced IRC activities.

Plugins can also contain methods that can be executed from the text input widget with the **`/call`** command. These methods take two arguments: the **`Window`** object of window calling the method, and a **`list`** of strings all the arguments passed to it. The method must not have the same name as any **`Plugin`** event or inherited method, and can only take these two arguments (besides the **`self`** instance reference). The **`/call`** command will execute *all* Plugin methods with the method name passed to it, so if multiple plugins contain methods with the same name, ***they will all be executed***.

At least a basic understanding of the IRC protocol is assumed. If you don't understand the IRC protocol, or concepts like channels, channel and user modes, and hostmasks, among other things, this document may be difficult to work through. The two main design documents for the IRC protocol, RFC 1459 and RFC 2812, can be found in the "Help" menu, and reading these will help your plugin development (and your understanding of how MERK and IRC works).

17 *An application programming interface (API)...is a type of software interface, offering a service to other pieces of software.* Wikipedia

Pausing and Unpausing Plugins

In the plugin manager, plugins can be “paused”. Whenever a plugin is “paused”, any events sent to that plugin will not execute, with several exceptions: **`init()`**, **`unload()`**, **`uninstall()`** , **`pause()`**, and **`unpause()`** will all still be triggered and execute. Any callable methods will be disabled. The plugin will remain loaded in memory while being paused. Plugins that are paused are displayed in italics in the plugin manager.

When “unpaused”, any events sent to a plugin will trigger and execute.

When a plugin is initially “paused”, the **`pause()`** event is triggered. This allows plugins to remove macros from memory, or other functionality that the plugin may provide. When the plugin is “unpaused”, the **`unpause()`** event is triggered, so plugins may place macros back into memory or anything else the plugin may need to do. These events are *only* executed when a plugin is “paused” or “unpaused”, and do not execute on load.

Plugins, when initially loaded, are always “unpaused”, and will trigger and execute any events or methods they contain. A plugins state as “paused” or “unpaused” is only saved while the application is running, and is not saved between executions. If a plugin is “paused” when MERK exits, it will load “unpaused” the next time MERK runs.

Creating MERK Plugins

All MERK plugins are classes derived from a class built into MERK named **`Plugin`**. To start off a plugin, this class must be imported from MERK:

```
from merk import Plugin
```

MERK plugins *must* be derived from this class. Plugin source files can contain multiple **`Plugin`** classes, and each class will be treated as a separate plugin.

This class is the only thing exported from the **`merk`** class by default, so a "splat" import works too:

```
from merk import *
```

Once the **`Plugin`** class is imported, a new class can be created that inherits from that class; this is a new MERK plugin. Here's an example plugin that doesn't really do anything but print a greeting to STDOUT when the plugin is loaded:

```
from merk import Plugin

class ExamplePlugin(Plugin):

    NAME = "Example Plugin"
    AUTHOR = "Dan Hetrick"
    VERSION = "1.0"
    SOURCE = "https://github.com/nutjob-laboratories/merk"

    def init(self):
        print("Hello world!")
```

`Plugin` classes contain methods that MERK triggers when certain events occur. In the example above, the **`init()`** method is triggered every the plugin is loaded or reloaded. This event method that doesn't accept any event arguments. The **`init()`** plugin event will be executed every time the plugin is loaded into memory, or reloaded.

Plugins are cleared from memory when they are reloaded, by default. This can be changed by changing the **`clear_plugins_from_memory_on_reload`** setting with the settings dialog or with the **`/config`** command. If this option is turned on, **`unload()`** will still be executed every time, even if the plugin is not unloaded from memory.

The **`uninstall()`** plugin event is triggered when the plugin is uninstalled with the plugin manager or the **`/window`** command, and is intended to “clean up” after a plugin is removed. The **`unload()`** plugin event is triggered every time a plugin is unloaded from memory, like when a plugin is reloaded or on application exit. Both of these events, like **`init()`**, does not accept any arguments.

All other event methods accept a single argument, a dictionary containing information about the event, and objects that your plugin may need to complete tasks. For example, the **`message()`** event is triggered when MERK receives a message, and passes the nickname of the user that sent the message, the message contents, the target the message was sent to (either a channel or a nickname), and the MERK Window class representing the window that received that message in the dictionary.

With this information, we can craft an event that prints all incoming messages to STDOUT:

```
def message(self,**args):
    print(f"{args["nickname"]}: {args["message"]}")
```

The MERK Window class contains methods for interacting with subwindows in the MERK GUI. For example, the **`print()`** method allows plugins to print messages directly to a window. This example **`message()`** event method prints whatever message was sent to the window that it was sent to:

```
def message(self,**args):
    args["window"].print(f"{args["nickname"]} said {args["message"]}")
```

Each plugin also has access to a "console" window it can use to display data. Every plugin only has one console, and it is initially hidden when created (this can be changed in settings). This console can be printed to, much like other MERK subwindows, as well as moved, resized, and the like. To access a plugin's console, use the **`console()`** **`Plugin`** inherited method, which returns a **`Console`** object. Viewing a plugin's console is possible programmatically, or through double clicking a plugin in the plugin manager.

The Twisted IRC client object passed to most events (and obtainable from the MERK Window class) can be used to perform, well, IRC actions, and will function normally. It is the raw, instantiated object that MERK uses to communicate with the IRC server. Sending message done with this object will "impact" MERK like "normal". For example, if the **`.msg()`** method is called on the Twisted IRC client object to send a channel or private message, that message will appear as chat inside MERK; the message will be sent, and (hopefully) received by the target user or channel. Joining and parting channels will work normally, as will setting channel or user modes; if the Twisted IRC object is used to join or leave a channel, MERK will join that channel, automatically creating or destroying subwindows as needed.

As IRC is a text based protocol, most arguments passed to events are strings, unless otherwise noted.

`Plugin`s have four different attributes: **`NAME`**, **`AUTHOR`**, **`VERSION`**, and **`SOURCE`**. These are used by MERK to display information about each loaded plugin. A plugin doesn't *have* to have these attributes; if these attributes are not set in the plugin's source code, **`NAME`**, **`AUTHOR`**, and **`SOURCE`** will be set to "Unknown", and **`VERSION`** will be set to "1.0". All attributes should be

strings. If **SOURCE** is set to a URL, it will be displayed as a link the plugin manager. Plugins can also have an icon (a 48x48 pixel PNG image) that is displayed in the plugin manager; to set the icon for a plugin, place the desired icon in the same directory as the plugin source file, with the same file name, replacing the Python file's **.py** file extension with **.png**.

Plugins must also have at least one event method or one **/call**able method. If a **Plugin** doesn't have *any* event or **/call** methods, an error will be displayed and the plugin will not be loaded. Plugins can react to over 30 different events; most of them are related to IRC, but a handful (like **init()**, mentioned above) are triggered by MERK itself. Individual events can be disabled and prevented from executing in the settings dialog.

Plugins also have a number of built-in methods for interacting with MERK. These methods can be used to get information about MERK, what servers it's connected to, what chat windows are displayed, interacting with subwindows, and the like. Every **Plugin** class inherits these methods by default; they can be overridden in code, if desired.

The **/call** command can also be used to directly execute specially-crafted **Plugin** methods, allowing for custom commands written completely in Python.

Plugins run in the same thread[18] that MERK runs in, and thus, function calls in plugins can block[19] or pause normal functioning of the application. Python and Qt have many options to run processes in a new or different thread to prevent this (the PyQt5 QThread class is good for this).

Since plugins run in the same thread as MERK, errors in the Python code of a plugin can crash MERK or prevent MERK from starting up. If this occurs, you have a few recovery options.

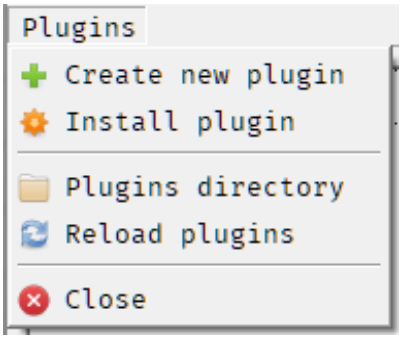
- You can use the **--disable-plugins** command-line flag to disable plugins; this will prevent plugins from loading and MERK can start up normally.
- If you know the filename of the plugin, you can navigate to the **plugins** directory and delete the plugin's file, or use the **--uninstall FILENAME** command-line option.
- You can delete all installed plugins with the **--uninstall all** command-line flag. ***This will delete any and all installed plugins and their icons! This will* not *move the files to the "recycle bin", and the files will be lost!*** This will also allow MERK to start up normally, without having to disable plugins.

Plugins can be created, edited, imported, and deleted with the plugin manager. Creating or editing a plugin will open the Python editor, made just for creating and editing plugins. The Python editor features syntax highlighting for Python, auto-indent, and more. The Python editor can open and edit any Python file.

18 *...a thread of execution is the smallest sequence of programmed instructions that can be managed independently by a scheduler.* Wikipedia

19 *...a process that is blocked is waiting for some event.* Wikipedia

Creating a plugin in MERK is easy. First, open the plugin manager, open the "Plugins" menu, and click on "Create new plugin":



This will open a "blank" **`Plugin`** (a code skeleton[20] with all attributes and events) in the Python editor. This "blank" **`Plugin`** has all the code needed for a plugin already in it; it's got default values for **`NAME`**, **`AUTHOR`**, **`VERSION`**, and **`SOURCE`** already filled in, and all plugin event methods. If saved, this **`Plugin`** won't do anything at all. There's no code to actually *do* anything when an event is triggered. Edit this **`Plugin`**, add any code you'd like to events so that the plugin accomplishes what you'd like, add any **`/call`**able methods you'd like, and save it. Click the "Reload plugins" option in the "Plugins" menu to load your **`Plugin`** into memory, or restart MERK. Congratulations! You've just written your first MERK **`Plugin`**!

`Plugin`s can also be written in your text editor or Python IDE of choice. Save your plugin files to the **`plugins`** directory, or use the plugin manager to install the new plugin, and restart MERK.

20 *A program skeleton may also be utilized as a template that reflects syntax and structures...* Wikipedia

Writing Methods for `/call`

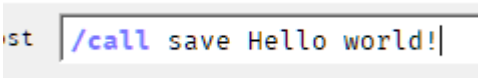
`/call` can be used to create "commands" for MERK, completely in Python. A method executable by **`/call`** must be formatted in a specific way: it must take two arguments (the **`Window`** that the **`/call`** command was executed in, and a **`list`** of strings), and it cannot have the same name as a **`Plugin`** event or inherited method. When a **`/call`** command is issued by the user, any plugin that contains a method in this format with the name passed to the command will be executed; this means that a single **`/call`** can execute multiple methods. Arguments to the **`/call`** command, after the method name, are tokenized like arguments to the **`/script`** command; that is, you can pass arguments that contain whitespace by containing the argument in quotes.

Methods that are executable by **`/call`** *must* be in a **`Plugin`** class. **`/call`** cannot execute "stand-alone" functions in modules.

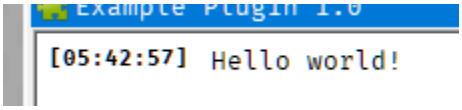
As an example, let's create a **`Plugin`** with a method that allows the user to make notes in a plugin's **`Console`**. We'll name our method "save":

```
from merk import Plugin

class ExamplePlugin(Plugin):

    NAME = "Example Plugin"
    AUTHOR = "Dan Hetrick"
    VERSION = "1.0"
    SOURCE = "https://github.com/nutjob-laboratories/merk"

    def save(self,window,arguments):
        console = self.console()
        console.print(" ".join(arguments))
```

We'll add our method to a new plugin, and restart MERK. After connecting to a server, we'll execute a **`/call`** command to execute our method:



This writes out "Hello world!" to the plugin's console:



If this method was added to multiple plugins, this method would be executed *in each plugin, once for each plugin*.

As this command can potentially be hazardous, **`/call`** can be disabled either in the settings dialog, or by using **`/config`** to set **`enable_call_command`** to "false".

Packaging and Installing MERK Plugins

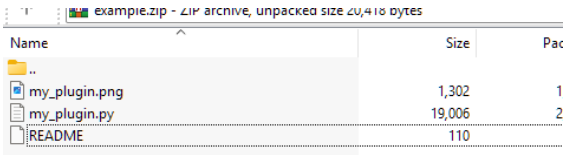
Plugins can be "packaged" in two ways: either as a single Python source file, or as a ZIP[21] archive. All filenames should be all lowercase, including file extensions. To package a MERK plugin in a ZIP file, place any plugin source files (and their icons) in the root directory of the ZIP. Only Python (**`*.py`**) and PNG files (**`*.png`**) in the ZIP will be extracted into the **`plugins`** directory. If there are any MERK scripts (**`*.merk`**) in the ZIP, these will be extracted into the **`scripts`** directory.

Importing a plugin via the plugin manager will ask before overwriting existing existing files in the **`plugins`** directory. The filename of a plugin is *only* used internally, is only displayed to the user in the plugin manager, and is not used for any other purpose. To force MERK to automatically overwrite files on import, use **`/config`** to set **`overwrite_files_on_plugin_import`** to "true", or turn this setting on in the settings dialog, on the "Plugins" page; with this set, any existing files in the plugins directory will be overwritten, regardless of whether a plugin is being imported as a Python source file or a packaged ZIP file. This may be desirable during plugin development. Installing a plugin via the **`--install`** command-line option will *always* overwrite existing files.

Plugins can also have icons that are displayed in the plugin manager. Icons should be in the PNG[22] format, and should have the same name (minus the file extension) as the plugin, and it should be in the same directory (usually **`plugins`**) as the plugin. For example, if a plugin's filename is **`example.py`**, the icon's filename should be **`example.png`**. The icon will be displayed in the plugin manager as a 48x48 pixel image.

Any MERK scripts (with a **`.merk`** file extension) found in a plugin package will be copied into the **`scripts`** directory. If a script of the same name already exists in the **`scripts`** directory, the plugin installation will ask before overwriting it, much like with Python source files. If overwriting files is turned on in the settings dialog or the configuration file, MERK scripts will be overwritten automatically.

As an example, let's "create" a plugin package. Our plugin's code is contained in a file named **`my_plugin.py`**. We want to display an icon for your plugin in the plugin manager, so we create a 48x48 pixel image, and name it **`my_plugin.png`**. We'll also create a **README** file, with information about the plugin. When we create our ZIP file, we'll put all three files in the root directory of the ZIP, not in a subdirectory. Here's what our ZIP file would look like in WinRAR:



21 *...is an archive file format that supports lossless data compression.* Wikipedia
22 *Portable Network Graphic.* Wikipedia

Once we've created our ZIP file, it's ready to be installed! The easiest way to create a ZIP plugin package is by using the plugin manager to export a plugin. Just select a plugin, right click on it, and select the menu option to "Export plugin", and MERK will create a correctly formatted ZIP file with the plugin and icon (if one exists) inside.

To install a ZIP packaged plugin, either extract the ZIP archive into the **`plugins`** directory, use the --**`install`** command-line option, or use the "Install plugin" option in the "Plugins" menu in the plugin manager. To install a plugin in a single Python source file, copy the source file and icon (if necessary) into the **`plugins`** directory, use the --**`install`** command-line option, or use the "Install plugin" option in the plugin manager.

Plugin Class

Inherited Methods

These methods are inherited by every **`Plugin`**-derived class, and can be called from the **`self`** object passed to every event.

`alias`

Arguments

1. **`None`** (retrieves alias table)
2. **`String`** (retrieves alias value)
3. **`String`**, **`String`** (creates or sets an alias value)

Returns

1. **`Dict`** (alias table)
2. **`String`** or **`None`** (alias value)
3. **`Boolean`**

Description

Allows to view and edit the alias table, used by scripts and the text input widget. This method does *not* allow any built-in aliases to be retrieved or edited.

1. If called with no arguments, a **`dict`** containing all alias values (with the corresponding alias as keys) is returned.
2. If called with a single **`string`** as an argument, an alias with the name contained in the **`string`** is searched for in the table, and that alias's value is returned as a **`string`**; if that alias doesn't exist, **`None`** is returned.
3. If called with two **`string`**s as arguments, an alias with the name contained in the first **`string`** is created with the value contained in the second **`string`**; if this creation is successful, **`True`** is returned, and if unsuccessful, **`False`** is returned.

Example

```
print(f"All aliases: {self.alias()}")

if self.alias("target")!=None:
    print("Alias '$target' exists!")
else:
    print("Alias '$target' doesn't exist!")

if self.alias("example","my value"):
    print("Alias '$example' created!")
else:
    print("Alias '$example' not created!")
```

`all_channels`	**Arguments**	None
	Returns	**`List`** of MERK Windows
	Description	Returns a list of all open channel subwindows. Each entry in the list is a MERK Window.

	Example	`for w in self.all_channels():`
		`  name = w.name()`
		`  print(f"{name}")`

`all_privates`	**Arguments**	None
	Returns	**`List`** of MERK Windows
	Description	Returns a list of all open private chat subwindows. Each entry in the list is a MERK Window.

	Example	`for w in self.all_privates():`
		`  name = w.name()`
		`  print(f"{name}")`

`all_servers`	**Arguments**	None
	Returns	**`List`** of MERK Windows
	Description	Returns a list of all open server subwindows. Each entry in the list is a MERK Window.

	Example	`for w in self.all_servers():`
		`  name = w.name()`
		`  print(f"{name}")`

`all_windows`	**Arguments**	None
	Returns	**`List`** of MERK Windows
	Description	Returns a list of all open subwindows. Each entry in the list is a MERK Window.

	Example	`for w in self.all_windows():`
		`  name = w.name()`
		`  print(f"{name}")`

`asciimojize`	**Arguments**	**`String`** (string to insert ASCIImojis into)
	Returns	**`String`** (string with ASCIImojis)
	Description	Converts ASCIImoji shortcodes into ASCIImojis, and returns the altered string.
	Example	`bear = self.asciimojize("(bear)")` `print(bear)`

bind

Arguments	**`String`** (key sequence) **`String`** (command to execute)
Returns	**`Boolean`**
Description	Creates a new hotkey. The key sequence that triggers the hotkey should contain no spaces. The command can be any command that can be entered into a text input widget on a subwindow. See **`/bind`** for more information. Returns **`True`** if the hotkey was set, and **`False`** if setting the hotkey failed.

Example	`if self.bind("Ctrl+Shift+X","/quitall"):`
	`  print("Hotkey set!")`
	`else:`
	`  print("Hotkey not set!")`

browser

Arguments	**`String`** (URL to open)
Returns	Nothing
Description	Opens a URL in the default browser, if the URL is a valid URL.
Example	`self.browser("https://google.com")`

channel

Arguments	**`Client`** (Twisted IRC client object to query) **`String`** (name of the channel)
Returns	MERK Window
Description	Returns the window for a channel on **`client`**. Returns **`None`** if the window is not open or can't be found.
Example	`channel = self.channel(client,"#merk")`

channels

Arguments	**`Client`** (Twisted IRC client object to query)
Returns	**`List`** of MERK Windows
Description	Returns a list of all channel windows on **`client`**. Returns an empty list if none are found.
Example	`channels = self.channel(client)`

clients

Arguments	None
Returns	**`List`** of Twisted IRC client objects
Description	Returns a list of all active clients that MERK is connected to. Each client will be "registered" with the server; that is, each client can issue commands or be used with other built-in methods.

Example	`for c in self.clients():`
	`  print(f"Server: {c.server}")`
	`  print(f"Port: {c.port}")`

color	**Arguments**	**String** (string to insert IRC color codes into)
	Returns	**String** (string with IRC color codes)
	Description	Inserts IRC color codes into a string using the same mechanism MERK uses in user input. Pass the formatted string to the method, and the same string with IRC color codes "injected" into it is returned.
	Example	`msg = self.color("<0,4Red Alert!>")`
colored	**Arguments**	**String**
	Returns	**Boolean** (**True** if string contains IRC color and formatting codes, **False** if not)
	Description	Detects if a string contains IRC color and formatting codes.

	Example	`if self.colored("Hello"):`
		`  print("Contains IRC colors!")`
		`else:`
		`  print("Does not contain IRC colors!")`

connect	**Arguments**	**String** (server) **Integer** (port) **String** (password, defaults to **None**) **Boolean** (connect via SSL/TLS, defaults to **False**) **Boolean** (reconnect on disconnection, defaults to **False**)
	Returns	Nothing
	Description	Connects MERK to an IRC server. If a connection script for this server exists, it will *not* be executed on connection.
	Example	`self.connect("localhost",6697,None,True)`
console	**Arguments**	None
	Returns	Plugin Console
	Description	Returns the plugin's console window.
	Example	`c = self.console()` `c.print("Hello world!")`
current	**Arguments**	None
	Returns	MERK Window or **None**
	Description	Returns the MERK Window of the currently active server, channel, or private chat window, if one of these windows is currently active; if some other type of window is currently active, returns **None**.
	Example	`w = self.current()` `if w!=None: w.print("Hello, world!")`

deasciimojize	**Arguments**	**String** (string with ASCIImojis)
	Returns	**String** (string with ASCIImoji shortcodes)
	Description	Converts ASCIImojis in a string into ASCIImoji shortcodes.
	Example	`s = self.deasciimojize("(⌐■_■)")` `print(s)`
demojize	**Arguments**	**String** (string with emojis)
	Returns	**String** (string with emoji shortcodes)
	Description	Converts emojis in strings into emoji shortcodes.
	Example	`s = self.demojize("😂")` `print(s)`
emojize	**Arguments**	**String** (string to insert emojis into)
	Returns	**String** (string with emojis)
	Description	Converts emoji shortcodes into emojis, and returns the altered string.
	Example	`joy = self.emojize(":joy:")` `print(joy)`
fade	**Arguments**	None or **Integer** (transparency percentage)
	Returns	None or **Integer** (transparency percentage)
	Description	Sets the transparency of the main application window, from 1% to 100%. If called without arguments, returns the percentage of transparency of the main application window.
	Example	`self.fade(95)` `print(f"MERK is {self.fade()}% see through")`
find	**Arguments**	**String** (name of the file to search for) **String** (file extension; optional)
	Returns	**String** or **None**
	Description	Searches for a file in the directory where MERK stores it's plugin files, scripts, configuration files, and the installation directory (if that option is turned on in settings), in that order. Returns the full path of the first matching file found. Returns **None** if the file is not found
	Example	`filename = self.find("bell","wav")`
folder	**Arguments**	**String** (directory path)
	Returns	Nothing
	Description	Opens up a folder in the default file manager.
	Example	`self.folder("C:\")`

home

Arguments	None
Returns	**`String`** (directory path)
Description	Returns the path to the directory where MERK stores all its configuration files.
Example	`directory = self.home()`

id

Arguments	None
Returns	**`String`**
Description	Each plugin has a unique identifier, which is generated when the plugin is loaded. This method returns the plugin's identifier. This value is stable across different application runtimes.
Example	`indentifier = self.id()`

ignore

Arguments	**`String`** (nickname or hostmask to ignore)
Returns	**`Boolean`**
Description	Adds a user to the ignore list. Wildcards can be used in the entry; use **`*`** for any character(s) and **`?`** for a single character. Returns **`True`** is the addition was successful, and **`False`** if not.
Example	`if self.ignore("*@annoying.com"):` `  print("Added to ignore list!")`

ignores

Arguments	None
Returns	**`List`** of **`Strings`** (ignore list)
Description	Returns the complete list of users being ignored. This may be an empty list if there are no users in the ignore list.
Example	`ignoring = self.ignores()`

is_away

Arguments	**`Client`** (Twisted IRC client object to query)
Returns	**`Boolean`**
Description	Checks if MERK is set to "away" status on a specific client, and return **`True`** if MERK is away, and **`False`** if not.

Example	`if self.is_away(client):`
	`  print("Away")`
	`else:`
	`  print("Not away")`

is_ignored

Arguments	**`String`** (nickname) **`String`** (hostmask)
Returns	**`Boolean`**
Description	Checks if a nickname or hostmask is in the ignore list. If either the nickname or the hostmask to check is not known, set that argument to **`None`**; however, either the nickname *or* the hostmask must be set. Returns **`True`** if the nickname or hostmask is in the ignore list, or **`False`** if not.

Example	`if self.is_ignored("user","*@annoying"):`
	`  print("User is ignored!")`
	`else:`
	`  print("User is not ignored!")`

list

Arguments	**`Client`** (Twisted IRC client object to query)
Returns	**`List`** of **`Lists`** of **`Strings`** (channel list)
Description	Retrieves a list of channels on the IRC server that client is connected to. If MERK has not requested or received a channel list from the server, this list will be empty; to request a fresh list from the server, consider executing the **`/refresh`** command. Each entry in the returned list is a list, with the following order: **`channel name, user count, topic`**

Example	`for chan in self.list(client):`
	`  name = chan[0]`
	`  user_count = chan[1]`
	`  topic = chan[2]`

location

Arguments	None
Returns	**`List`** of **`Integers`** (X and Y value of the main window)
Description	Returns the location of the main application window in a List of Integers, with the first value being the X value, and the second being the Y value.
Example	`x,y = self.location()`

macro	**Arguments**	**`String`** (macro name) **`String`** (script to execute) **`String`** (usage text; optional) **`String`** (help text; optional)
	Returns	**`Boolean`**
	Description	Creates a macro; that is, creates a command that can be entered into the text input widget of a subwindow that executes a script. Macros made with this method *cannot* overwrite existing macros with the same name. Returns **`True`** if the macro was successfully created, and **`False`** if not. See **`/macro`** for more information.
	Example	`if self.macro("hello","hello.merk"):` `  print("/hello macro created!")`
markdown	**Arguments**	**`String`** (string to insert IRC formatting codes into)
	Returns	**`String`** (string with IRC formatting codes)
	Description	Inserts IRC text formatting codes into a string using the same mechanism MERK uses for input. Pass the formatted string to the method, and the same string with IRC formatting codes "injected" into it is returned.
	Example	`msg = self.markdown("__**Hello!**__")`
markup	**Arguments**	**`String`** (string to insert IRC formatting and color codes into)
	Returns	**`String`** (string with IRC formatting and color codes)
	Description	Inserts IRC text formatting and color codes into a string using MERK markup. Any emoji or ASCIImoji shortcodes in the string will be converted into emojis or ASCIImojis, respectively.
	Example	`msg = self.markup("(cat) <0,4**Hello!**>")`
max	**Arguments**	None
	Returns	Nothing
	Description	Maximizes the application window.
	Example	`self.max()`
maximized	**Arguments**	None
	Returns	**`Boolean`**
	Description	Returns **`True`** if the application window is maximized, otherwise returns **`False`**.
	Example	`if self.maximized(): print("Maximized")`

min	**Arguments**	None
	Returns	Nothing
	Description	Minimizes the application window.
	Example	`self.min()`
minimized	**Arguments**	None
	Returns	**Boolean**
	Description	Returns **True** if the application window is minimimzed, otherwise returns **False**.
	Example	`if self.minimized(): print("Minimized")`
modes	**Arguments**	**Client** (Twisted IRC client object to query)
	Returns	**String** (channel modes)
	Description	Returns all modes set on the user on a given IRC connection.
	Example	`my_modes = self.modes(client)`
move	**Arguments**	**Integer** (X value) **Integer** (Y value)
	Returns	**Boolean**
	Description	Moves the main application window to the specified coordinates. If the move would put the window in a location where it could not be interacted with, this method will fail and return **False**; Otherwise, it returns **True**.
	Example	`if self.move(200,500):` `  print("Move successful!")`
private	**Arguments**	**Client** (Twisted IRC client object to query) **String** (name of the private chat) **Boolean** (optional; creates a private chat window if the window is not found)
	Returns	MERK Window
	Description	Returns the window for an open private chat with a user on **client**. Returns **None** if the window is not open or can't be found. If the third argument is used, and set to **True**, a new private chat window will be created, shown, and a MERK Window for the new window will be returned.
	Example	`chat = self.private(client,"user")`

privates	**Arguments**	**Client** (Twisted IRC client object to query)
	Returns	**List** of MERK Windows
	Description	Returns a list of all private chat windows on **client**. Returns an empty list if none are found.
	Example	`channels = self.channel(client)`
restore	**Arguments**	None
	Returns	Nothing
	Description	Restores the application window.
	Example	`self.restore()`
server_window	**Arguments**	**Client** (Twisted IRC client object to query)
	Returns	MERK Window
	Description	Returns the server window associated with **client**.
	Example	`server_window = self.server(client)`
script	**Arguments**	**Client** (Twisted IRC client object to query) **String** (text of script or filename) **List** of **Strings** (arguments to the script)
	Returns	**Boolean**
	Description	Executes a script in the server window of a **client**. If executing a script by filename, the script's extension (**.merk**) can be omitted. Script filenames will be searched for in the **scripts**, configuration, and application install directories (if that option is enabled in "Settings"), in that order. If a file is not found, than the **String** will be executed as as script. Scripts are executed in a new thread. Returns **True** if the script was executed, and **False** if not.

	Example	`if self.script(client,"my_script",[]):`
		`  print("Script executed")`
		`else:`
		`  print("Script not executed")`
		
		`join_script="""/join #channel`
		`wait 15`
		`/msg #channel Hello everyone!"""`
		`if self.script(client,join_script,[]):`
		`  print("Join script executed")`
		`else:`
		`  print("Join script not executed")`

size	**Arguments**	**Integer** (width) – Optional, see description **Integer** (height) – Optional, see description
	Returns	Nothing or **List** of **Integers**
	Description	Resizes the main application window. If called without arguments or only only one argument, returns a **List** of **Integers**, with the width of the window being the first entry, and the height being the second.
	Example	`self.size(800,600)` `width,height = self.size()`
strip	**Arguments**	**String** (string with IRC color and formatting codes)
	Returns	**String** (string stripped of all IRC color and formatting codes)
	Description	Strips all IRC color and formatting codes from a string, and returns the plain text.
	Example	`clean = self.strip(message)`
unbind	**Arguments**	**String** (key sequence)
	Returns	Nothing
	Description	Removes a hotkey. If the sequence is set to *****, all hotkeys will be removed. See **/bind** for more information.
	Example	`self.unbind("Ctrl+Shift+X")`
uncolor	**Arguments**	**String** (text with IRC color codes)
	Returns	**String** (text converted into IRC color “markup”)
	Description	Converts any IRC color codes in a string into the same “markup” MERK can use for input to insert IRC colors into text. The output of this method can be used with **color()** to reinsert the IRC color codes.
	Example	`topic = self.uncolor(window.topic())` `print(topic)`
unignore	**Arguments**	**String** (nickname or hostmask to remove)
	Returns	**Boolean**
	Description	Removes an entry from the ignore list. Returns **True** is the removal was successful, and **False** if not.
	Example	`if self.unignore("*@annoying.com"):` `  print("Removed from ignore list!")`

unmacro	**Arguments**	**`String`** (macro to remove)
	Returns	**`Boolean`**
	Description	Removes a macro from memory. Returns **`True`** is the removal was successful, and **`False`** if not.
	Example	`if self.unmacro("mycommand"):` `  print("Macro removed!")`
unmarkdown	**Arguments**	**`String`** (text with IRC formatting codes)
	Returns	**`String`** (text converted into IRC markdown formatting)
	Description	Converts any IRC text formatting codes into the same markdown MERK can use for input. The output of this method can be used with **`markdown()`** to reinsert the IRC formatting codes.
	Example	`c = self.unmarkdown(message)` `print(c)`
unmarkup	**Arguments**	**`String`** (text with IRC color and formatting codes)
	Returns	**`String`** (text converted into IRC color and markdown format)
	Description	Takes in a string with IRC color or text formatting control codes, and returns a string with MERK markup for that formatting. Any emojis or ASCIImojis in the string will be turned into shortcodes. The returned string can be converted back into IRC color, formatting, and emojis/ASCIImojis with the **`markup()`** inherited method.
	Example	`c = self.unmarkup(message)` `print(c)`
windows	**Arguments**	**`Client`** (Twisted IRC client object to query)
	Returns	**`List`** of MERK Windows
	Description	Returns a list of all open subwindows associated with **`client`**'s context. This will include server windows, channels, and private chats. Each entry in the list is a MERK Window.

	Example	`for w in self.windows(client):`
		`  name = w.name()`
		`  print(f"{name}")`

xconnect		
	Arguments	**`String`** (server) **`Integer`** (port) **`String`** (password, defaults to **`None`**) **`Boolean`** (connect via SSL/TLS, defaults to **`False`**) **`Boolean`** (reconnect on disconnection, defaults to **`False`**)
	Returns	Nothing
	Description	Connects MERK to an IRC server. If a connection script for this server exists, it *will* be executed on connection.
	Example	**`self.xconnect("localhost",6697,None,True)`**

Event Methods

These event methods are how **`Plugin`**s "work". Each method is called every time a specific event occurs in MERK. Every **`Plugin`** *must* have at least one event method, or the **`Plugin`** will not load. Unless otherwise noted, event arguments are **`String`**s. If a window does not exist for an event, or the window cannot be found, the **`window`** argument will be set to **`None`**, but the event will still be triggered.

`action`

Arguments	**`window`**	MERK Window or **`None`**
	`client`	Twisted IRC client object that triggered the event
	`user`	The user that sent the message, in the format nickname@hostmask
	`nickname`	The nickname of the user that sent the message
	`hostmask`	The hostmask of the user that sent the message; if not known, this will be **`None`**
	`channel`	The target of the message, which may be a channel or the nickname in use
	`message`	The message that was sent
Event	When MERK receives a CTCP action message	
Description	This event is triggered when MERK receives a channel or private CTCP action message. If the sending user is being ignored, this event will still be triggered.	
Example		

```
def action(self,**args):
    client = args["window"].client()
    print(f"Server: {client.server}")
    print(f"Port: {client.port}")
    print(f"Target: {args["channel"]}")
    print(f"User: {args["user"]}")
    print(f"Message: {args["message"]}")
```

`activate`

Arguments	**`window`** MERK Window	
Event	When a window is "activated"	
Description	This event is triggered when a subwindow is "activated" (brought into focus or clicked on). This event will only trigger for server, channel, or private chat subwindows.	
Example		

```
def activate(self,**args):
    window = args["window"]
    last = args["last"]
    print(f"Last window: {last.name()}")
    print(f"Current: {window.name()}")
```

away

Arguments	**client**	Twisted IRC client object that triggered the event
	user	The user that went away
	message	The away message the user used
Event	When a user goes "away"	
Description	This event is triggered when a user sets their status to "away" in the presence of the MERK client	

Example	`def away(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`
	`  print(f"User: {args["user"]}")`
	`  print(f"Message: {args["message"]}")`	

back

Arguments	**client**	Twisted IRC client object that triggered the event
	user	The user that came back
Event	When a user comes "back"	
Description	This event is triggered when a user sets their status to "back" in the presence of the MERK client	

Example	`def back(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`
	`  print(f"User: {args["user"]}")`	

close

Arguments	**name**	The name of the subwindow
Event	When a subwindow closes	
Description	This event is triggered whenever a chat subwindow closes.	
Example	`def close(self,**args):` `  name = args["name"]`	

connected

Arguments	**window**	MERK Window
	client	Twisted IRC client object that triggered the event
Event	When MERK completes registration on an IRC server	
Description	This event is triggered when MERK completes registration on an IRC server.	

Example	`def connected(self,**args):`
	`  window = args["window"]`
	`  client = args["client"]`
	`  print(f"Server: {client.server}")`
	`  print(f"Port: {client.port}")`	

connecting

Arguments	**client** Twisted IRC client object that triggered the event
Event	When MERK begins registration with an IRC server
Description	This event is triggered when MERK begins registration to an IRC server.

Example	`def connecting(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`

ctick

Arguments	**uptime** **Integer**. The number of seconds MERK has been running
Event	Once per second
Description	This event is triggered every second while MERK is running.
Example	`def ctick(self,**args):` `  print(f"Uptime: {args["uptime"].server}")`

disconnect

Arguments	**client** Twisted IRC client object that triggered the event
	message The quit message used
Event	When MERK disconnects from an IRC server
Description	This event is triggered when MERK disconnects from an IRC server. **message** may be an empty string if a message was not provided or if the disconnection occurred before registration.

Example	`def disconnect(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`
	`  print(f"Quit Message: {args["message"]}")`

error

Arguments	**client** Twisted IRC client object that triggered the event
	message The error message
Event	When MERK receives an error message from the server
Description	This event is triggered whenever MERK receives an error message from an IRC server.

Example	`def error(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`
	`  print(f"Error: {args["message"]}")`

init

Arguments	None
Event	On load
Description	This event is triggered when MERK loads the plugin, before any connections are attempted. This even will be triggered every time the plugin is loaded or reloaded; this behavior can be turned off in the settings dialog or with **`/config`**.
Example	`def init(self):` `  print("Plugin loaded\n")`

invite

Arguments	**`client`**	Twisted IRC client object that triggered the event
	`user`	The user sent the invite
	`channel`	The channel the invite it for
Event	When a channel invite is received	
Description	This event is triggered when a channel invite is received from another user.	

Example	`def invite(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`
	`  print(f"User: {args["user"]}")`
	`  print(f"Channel: {args["channel"]}")`	

ison

Arguments	**`client`**	Twisted IRC client object that triggered the event
	`users`	**`List`** of **`Strings`** (the users that are online)
Event	When the server sends a response to the ISON command	
Description	This event is triggered when the server sends a response the **`/ison`** command. If none of the users specified in the request are online, **`users`** will be empty.	

Example	`def ison(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`
	`  for u in args["users"]:`
	`  print(f"{u} is online")`	

isupport

Arguments	**client**	Twisted IRC client object that triggered the event
	options	**List** of **Strings** (the options the server supports)
Event	When the server sends a list of options the server supports	
Description	This event is triggered when the server sends a list of options the server supports, in the form of a long string. Each string is in the format "option=value".	

Example	`def isupport(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`
	`  for option in args["options"]:`
	`  print(option)`	

join

Arguments	**window**	MERK Window or **None**
	client	Twisted IRC client object that triggered the event
	user	The user that joined the channel
	channel	The channel the user joined
Event	When a user joins a channel MERK is present in	
Description	This event is triggered when user joins a channel that MERK is currently present in	

Example	`def join(self,**args):`
	`  client = args["window"].client()`
	`  print(f"Server: {client.server}")`
	`  print(f"Port: {client.port}")`
	`  print(f"User: {args["user"]}")`
	`  print(f"Channel: {args["channel"]}")`	

joined

	channel	The channel the client joined
	client	Twisted IRC client object that triggered the event
Event	When MERK joins a channel	
Description	This event is triggered when MERK joins a channel	

Example	`def joined(self,**args):`
	`  client = args["client"]`
	`  print(f"Server: {client.server}")`
	`  print(f"Port: {client.port}")`
	`  print(f"Channel: {args["channel"]}")`	

kick

Arguments	**window**	MERK Window or **None**
	client	Twisted IRC client object that triggered the event
	user	The user that issued the kick
	channel	The channel that the kick occurred in
	target	The user that was kicked
	message	The kick message (if there is one)
Event	When a user is kicked	
Description	This event is triggered when a user is kicked from a channel that MERK is present in.	

Example	`def kick(self,**args):`
	`  client = args["client"]`
	`  print(f"Server: {client.server}")`
	`  print(f"Port: {client.port}")`
	`  print(f"Channel: {args["channel"]}")`
	`  print(f"Kicker: {args["user"]}")`
	`  print(f"Target: {args["target"]}")`
	`  print(f"Message: {args["message"]}")`	

kicked

Arguments	**client**	Twisted IRC client object that triggered the event
	user	The user that issued the kick
	channel	The channel that the kick occurred in
	message	The kick message (if there is one)
Event	When MERK is kicked from a channel	
Description	This event is triggered when the user is kicked from a channel.	

Example	`def kicked(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`
	`  print(f"Channel: {args["channel"]}")`
	`  print(f"Kicker: {args["user"]}")`
	`  print(f"Message: {args["message"]}")`	

left

Arguments	**client**	Twisted IRC client object that triggered the event
	channel	The channel the client left
Event	When MERK leaves a channel	
Description	This event is triggered when MERK leaves a channel	

Example	`def left(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`
	`  print(f"Channel: {args["channel"]}")`	

line_in

Arguments	`client`	Twisted IRC client object that triggered the event
	`line`	The line received by the client, UTF-8 encoded
Event	Incoming data	
Description	This event is triggered every time MERK receives a line of data from an IRC server.	

Example	`def line_in(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`
	`  print(f"Line: {args["line"]}")`	

line_out

Arguments	`client`	Twisted IRC client object that triggered the event
	`line`	The line sent by the client
Event	Outgoing data	
Description	This event is triggered every time MERK sends a line of data to an IRC server.	

Example	`def line_out(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`
	`  print(f"Line: {args["line"]}")`	

lost

Arguments	`client`	Twisted IRC client object that triggered the event
Event	When MERK loses connection to an IRC server	
Description	This event is triggered when MERK's connection to an IRC server is lost.	

Example	`def lost(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`	

me

Arguments	**window**	MERK Window or **None**
	client	Twisted IRC client object that sent the message
	target	The target of the message, either a user or a channel
	message	The message that was sent
Event	When MERK sends a message	
Description	This event is triggered when MERK sends a message to a user or channel. If there's a corresponding subwindow for **target**, then **window** will be set to that window; if not, then window will be set to **None**.	

Example	`def me(self,**args):`
	`  window = args["window"]`
	`  client = args["client"]`
	`  target = args["target"]`
	`  message = args["message"]`	

message

Arguments	**window**	MERK Window or **None**
	client	Twisted IRC client object that triggered the event
	user	The user that sent the message, in the format nickname@hostmask
	nickname	The nickname of the user that sent the message
	hostmask	The hostmask of the user that sent the message; if not known, this will be **None**
	channel	The target of the message, which may be a channel or the nickname in use
	message	The message that was sent
Event	When MERK receives a message	
Description	This event is triggered when MERK receives a channel or private message. If the sending user is being ignored, this event will still be triggered.	

Example	`def message(self,**args):`
	`  client = args["window"].client()`
	`  print(f"Server: {client.server}")`
	`  print(f"Port: {client.port}")`
	`  print(f"Target: {args["channel"]}")`
	`  print(f"User: {args["user"]}")`
	`  print(f"Message: {args["message"]}")`	

mode

Arguments	**client**	Twisted IRC client object that triggered the event
	user	The user that set the mode. If the mode was set by the server, this will be set to "*".
	target	The user the mode was set on
	mode	The mode that was set
	arguments	**List** of **Strings**; any arguments passed to the mode.
Event	When a mode is set.	
Description	This event is triggered when a user or channel mode is set.	

Example	`def mode(self,**arguments):`
	`  client = arguments["client"]`
	`  user = arguments["user"]`
	`  target = arguments["target"]`
	`  mode = arguments["mode"]`
	`  args = arguments["arguments"]`	

motd

Arguments	**client**	Twisted IRC client object that triggered the event
	text	The text of the message of the day
Event	When the server sends the message of the day	
Description	This event is triggered when MERK receives the MOTD from the server.	

Example	`def motd(self,**args):`
	`  client = args["client"]`
	`  print(f"Server: {client.server}")`
	`  print(f"Port: {client.port}")`
	`  print(f"{args["text"]}")`	

nick

Arguments	**client**	Twisted IRC client object that triggered the event
	nickname	The new nickname
Event	Nickname change	
Description	This event is triggered every the user's nickname changes. This will be triggered on connection when the user's nickname is set.	
Example	`def nick(self,**args):` `  print(f"New nickname: {args["nickname"]}")`	

notice

Arguments	**window**	MERK Window or **None**
	client	Twisted IRC client object that triggered the event
	user	The user that sent the message, in the format nickname@hostmask
	nickname	The nickname of the user that sent the message
	hostmask	The hostmask of the user that sent the message; if not known, this will be **None**
	channel	The target of the message, which may be a channel or the nickname in use
	message	The message that was sent
Event	When MERK receives a notice	
Description	This event is triggered when MERK receives a channel or private notice. If the sending user is being ignored, this event will still be triggered.	

Example	`def notice(self,**args):`
	`  client = args["window"].client()`
	`  print(f"Server: {client.server}")`
	`  print(f"Port: {client.port}")`
	`  print(f"Target: {args["channel"]}")`
	`  print(f"User: {args["user"]}")`
	`  print(f"Message: {args["message"]}")`	

part

Arguments	**window**	MERK Window or **None**
	client	Twisted IRC client object that triggered the event
	user	The user that left the channel
	channel	The channel the user left
Event	When a user leaves a channel MERK is present in	
Description	This event is triggered when user leaves a channel that MERK is currently present in	

Example	`def part(self,**args):`
	`  client = args["window"].client()`
	`  print(f"Server: {client.server}")`
	`  print(f"Port: {client.port}")`
	`  print(f"User: {args["user"]}")`
	`  print(f"Channel: {args["channel"]}")`	

pause

Arguments	None
Event	When a plugin is paused
Description	This event is triggered when plugin is paused
Example	`def pause(self):` `  print("Paused!")`

ping

Arguments	**client**	Twisted IRC client object that triggered the event
Event	Whenever the server sends a ping	
Description	This event is triggered when the server sends a ping to the client.	

Example	`def ping(self,**args):`
	`  args["client"]`
	`  print(f"Server: {client.server}")`
	`  print(f"Port: {client.port}")`
	`  print("Ping? Pong!")`	

quit

Arguments	**client**	Twisted IRC client object that triggered the event
	user	The user that quit
	message	The quit message the user used
Event	When a user quits IRC in view of MERK	
Description	This event is triggered when user quits IRC in a channel or private chat that MERK is in.	

Example	`def quit(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`
	`  print(f"User: {args["user"]}")`
	`  print(f"Message: {args["message"]}")`	

rename

Arguments	**client**	Twisted IRC client object that triggered the event
	old	The user's old nickname
	new	The user's new nickname
Event	When a user changes nicknames	
Description	This event is triggered when a user changes a nickname. This event will *not* trigger when MERK changes nicknames.	

Example	`def rename(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`
	`  print(f"Old nickname: {args["old"]}")`
	`  print(f"New nickname: {args["new"]}")`	

server

Arguments	**client**	Twisted IRC client object that triggered the event
	message	The message
Event	When a server message is received	
Description	This event is triggered if a server message is received by MERK.	

Example	`def server(self,**arguments):`
	`  client = arguments["client"]`
	`  message = arguments["message"]`	

subwindow

Arguments	**window**	MERK Window
Event	When a new chat subwindow is created	
Description	This event is triggered whenever MERK creates a new chat subwindow.	

Example	`def subwindow(self,**arguments):`
	`  window = arguments["window"]`
	`  print(window.name())`	

tick

Arguments	**client**	Twisted IRC client object that triggered the event
	uptime	**Integer**.The number of seconds the client has been connected
Event	Client uptime	
Description	This event is triggered once a second, every second that MERK is connected to a server	

Example	`def tick(self,**args):`
	`  print(f"Server: {args["client"].server}")`
	`  print(f"Port: {args["client"].port}")`
	`  print(f"Uptime: {args["uptime"]}")`	

topic

Arguments	**window**	MERK Window or **None**
	client	Twisted IRC client object that triggered the event
	user	The user that set the topic
	channel	The channel the topic was set on
	topic	The new channel topic
Event	When a channel's topic is set.	
Description	This event is triggered when a channel's topic is set	

Example	`def topic(self,**arguments):`
	`  client = arguments["client"]`
	`  window = arguments["window"]`
	`  user = arguments["user"]`
	`  channel = arguments["channel"]`
	`  topic = arguments["topic"]`	

uninstall

Arguments	None
Event	When a plugin is uninstalled with the plugin manager.
Description	This event is triggered when a plugin is uninstalled with the plugin manager. The event is triggered directly before the plugin is deleted. This event will *not* be triggered if the plugin is uninstalled via a command-line flag.
Example	`def uninstall(self):` `  print("Plugin uninstalling...\n")`

unload

Arguments	None
Event	When a plugin is unloaded. This will be triggered when the plugin list is “reloaded”, when the plugin is uninstalled, and on the exit of the application.
Description	This event is triggered when a plugin is unloaded, when MERK exits.
Example	`def unload(self):` `  print("Plugin unloaded!\n")`

unmode

Arguments	**client**	Twisted IRC client object that triggered the event
	user	The user that removed the mode. If the mode was set by the server, this will be set to "*".
	target	The user the mode removed from
	mode	The mode that was removed
	arguments	**List** of **Strings**; any arguments passed to the mode.
Event	When a mode is removed.	
Description	This event is triggered when a user or channel mode is removed.	

Example	`def unmode(self,**arguments):`
	`  client = arguments["client"]`
	`  user = arguments["user"]`
	`  target = arguments["target"]`
	`  mode = arguments["mode"]`
	`  args = arguments["arguments"]`	

unpause

Arguments	None
Event	When a plugin is unpaused
Description	This event is triggered when plugin is unpaused
Example	`def unpause(self):` `  print("Unpaused!")`

<table>
<tr><td rowspan="5">uptime</td><td rowspan="2">Arguments</td><td>window</td><td>MERK Window</td></tr>
<tr><td>uptime</td><td>Integer. How many seconds the window has been connected to the server</td></tr>
<tr><td>Event</td><td colspan="2">Every second a server, channel, or private chat window is in existence and connected</td></tr>
<tr><td>Description</td><td colspan="2">This event is triggered once a second for every server, channel, or private chat window.</td></tr>
<tr><td>Example</td><td colspan="2"><pre>def uptime(self,**arguments):
    window = arguments["window"]
    uptime = arguments["uptime"]</pre></td></tr>
</table>

MERK Window Class

The MERK Window class is passed as an argument to many **`Plugin`** events; a list of MERK Window classes is also returned by the **`windows()`** and **`all_windows()`** inherited **`Plugin`** methods. This is an instance of a class that allows plugins to interact with a MERK subwindow (either a channel, private chat, or server window). This instance can be used to send IRC messages and display them in the client, execute commands on that window's context, get information about the window or the chat contained in the window, or otherwise manipulate the subwindow. Before using a Window object passed from an event method, check to make sure that the argument passed is not equal to **`None`**; if MERK cannot find the subwindow associated with the event (usually because the subwindow doesn't exist), the "window" argument passed to the event method will be set to **`None`**. For example, if a private message is sent to a user that does not have an existing private chat window, the Window object that would be passed by the **`me()`** event does not exist, so the "window" argument will be set to **`None`**. The Window objects returned by inherited **`Plugin`** methods will never be set to **`None`**.

Methods

`action`	**Arguments**	**`String`** (target of the message) **`String`** (the message)
	Returns	Nothing
	Description	Sends an CTCP action message. This can message can be sent to any user or channel. If the target of the message (a user or channel) has an open window, the message with be displayed in that window's chat display.
`active`	**Arguments**	None
	Returns	**`Boolean`**
	Description	Returns **`True`** if the window currently has focus, and **`False`** if the window does not.
`alias`	**Arguments**	**`String`** (the text to interpolate)
	Returns	**`String`**
	Description	Interpolates any existing aliases (both created and built-in) into text, and returns the result.
`bans`	**Arguments**	None
	Returns	**`List`** of **`Strings`** or **`None`**
	Description	If the window is a channel window, returns a list of all banned users in that channel. Otherwise, returns **`None`**.

chat	**Arguments**	None
	Returns	**List** of **Lists** of **Strings**
	Description	Returns all the chat that is currently being displayed in the window. Each entry in the list is a list in the following format: 0. **Type**. This will either be "chat" for chat messages, or "action" for CTCP action messages. 1. **User**. The user that sent the message. 2. **Message**. The content of the chat message. This will *only* contain chat, and will not contain system or server messages. Depending on how much chat is being displayed, this **list** may be fairly large. If the window is a server window, or there is no chat being displayed, this will return an empty **list**.
clear	**Arguments**	None
	Returns	Nothing
	Description	Clears the chat display.
client	**Arguments**	None
	Returns	**Twisted IRC client object**
	Description	Returns the Twisted IRC client object associated with that window's context.
close	**Arguments**	None
	Returns	None
	Description	Closes the subwindow. If the subwindow is for a channel, this will cause MERK to leave that channel.
describe	**Arguments**	**String** (the message to send)
	Returns	**Boolean**
	Description	Uses the window to send a CTCP action message, displaying it in the chat window. Only functional in channel and private windows. Returns **True** if successful, **False** if not.
execute	**Arguments**	**String** (the command to execute)
	Returns	Nothing
	Description	Executes a command in the window's context.

fade	**Arguments**	None or **Integer** (the transparency percentage)
	Returns	**Integer** or Nothing
	Description	If called without any arguments, returns the transparency of the window, from 1 to 100. If called with **Integer** (from 1 to 100), sets the transparency of the window.
hide	**Arguments**	None
	Returns	Nothing
	Description	Hides the window.
history	**Arguments**	None
	Returns	**List** of **Strings**
	Description	Returns the command history of the window's text input widget. If command history tracking is turned off, or empty, this will return an empty **list**. This will be in order of most recent to least recent. The maximum size of this list is 20 entries by default, but may be shorter or longer, depending on the **command_history_length** configuration setting.
key	**Arguments**	None or **String**
	Returns	**String** or **None** or **Boolean**
	Description	If not argument is passed, returns the key set on a channel as a **string**, or **None** if there is no key set. If a **string** is passed, and the channel is unlocked, attempts to set **+k** on the channel with **string** as the key; if the channel is already locked, attempts to set **-k** on the channel with **string** as the key. Returns **True** if the mode change attempt was made. Returns **None** if the window is not a channel, the channel does not have a key, or a mode change was not attempted.
max	**Arguments**	None
	Returns	Nothing
	Description	Maximizes the window.
maximized	**Arguments**	None
	Returns	**Boolean**
	Description	Determines whether a window is maximized or not. Returns **True** if the window is maximized, and **False** if not.

message	**Arguments**	**String** (target of the message) **String** (the message)
	Returns	Nothing
	Description	Sends an IRC message. This can message can be sent to any user or channel. If the target of the message (a user or channel) has an open window, the message with be displayed in that window's chat display.
min	**Arguments**	None
	Returns	Nothing
	Description	Minimizes the window.
minimized	**Arguments**	None
	Returns	**Boolean**
	Description	Determines whether a window is minimized or not. Returns **True** if the window is minimized, and **False** if not.
modes	**Arguments**	None
	Returns	**String** or **None**
	Description	If the window is a channel window, returns a string with all the modes set on that channel. Otherwise, returns **None**.
move	**Arguments**	**Integer** (the X-value of the new position), **Integer** (the Y-value of the new position)
	Returns	**True** if the move was successful, **False** if not
	Description	Moves a subwindow to the specified coordinates. If the move would put the window in a location where it could not be interacted with, this method will fail and return **False**.
name	**Arguments**	None
	Returns	**String** (the name of the window)
	Description	Returns the name of the window. For server windows, this will be the server's hostname; for channel windows, this will be the name of the channel; and for private chat windows, this will be the nickname of the person the private chat is with.

nicks	**Arguments**	None
	Returns	**List** of **Strings**
	Description	If the window is a channel window, returns a list of all nicknames (with hostmasks, if known, separated by "!") in that channel. Each nickname will have channel status prefixes removed. Otherwise, returns an empty **list**.
note	**Arguments**	**String** (the message to send)
	Returns	**Boolean**
	Description	Uses the window to send a notice message, displaying it in the chat window. Only functional in channel and private windows. Returns **True** if the message was sent, and **False** if not.
notice	**Arguments**	**String** (target of the message) **String** (the message)
	Returns	Nothing
	Description	Sends a notice. This can message can be sent to any user or channel. If the target of the message (a user or channel) has an open window, the message with be displayed in that window's chat display.
position	**Arguments**	None
	Returns	**List** of **Integers** (X value, Y value)
	Description	Returns the position of the window in a list in the following format: 0. **X value**. The horizontal position of the window. 1. **Y value**. The vertical position of the window. If the window's position cannot be determined, both values in the **list** will be set to **0**.
print	**Arguments**	**String** (the text to print)
	Returns	Nothing
	Description	Prints text to the window's chat display. HTML may be used.
prints	**Arguments**	**String** (the text to print)
	Returns	Nothing
	Description	Prints a system message to the window's chat display. HTML may be used.

restore	**Arguments**	None
	Returns	Nothing
	Description	"Restores" the window; if the window has been minimized or maximized, this window returns to a normal state.
say	**Arguments**	**String** (the message to send)
	Returns	**Boolean**
	Description	Uses the window to send a message, displaying it in the chat window. Only functional in channel and private windows. Returns **True** if the message was sent, and **False** if not.
script	**Arguments**	**String** (the text of the script or script filename) **List** of **Strings** (the arguments to the script)
	Returns	Nothing
	Description	Executes a script in the window's context. If executing a script by filename, the script's extension (**.merk**) can be omitted. Script filenames will be searched for in the **scripts**, configuration, and application install directories, in that order. Scripts are executed in a new thread.
show	**Arguments**	None
	Returns	Nothing
	Description	If the window is hidden, the window will be shown. Also, moves focus to the window.
size	**Arguments**	**Integer** (width) – Optional, see description **Integer** (height) – Optional, see description
	Returns	Nothing or **List** of **Integers**
	Description	Resizes the window. If called without arguments or only only one argument, returns a **List** of **Integers**, with the width of the window being the first entry, and the height being the second.

status	**Arguments**	None or **String** (status to query)
	Returns	**String** (channel status) or **Boolean**
	Description	If called without argument, **status()** will return a **string** containing the user's highest level channel status: 'owner', 'admin', 'operator', 'halfop', 'protected', 'voiced', or 'normal'. If called on a non-channel window, this will always return 'normal'. If called with a **string** containing a channel status, this will return **True** if the user has that channel status in the channel, and **False** if the user does not. If called on a non-channel window with 'normal' as the argument, this will return **True**.
title	**Arguments**	None or **String**
	Returns	**String** or Nothing
	Description	Sets the window's title to **string**; if **string** is omitted, the window's current title is returned as a **string**. Unless changed, a window's title will always be it's name. For channels, that will be the channel's name. For private chat windows, that will the nickname of the user being chatted with. For servers, that will be the hostname of the server after registration, and the **server:port** used to connect to the server before registration.
topic	**Arguments**	None or **String** (new channel topic)
	Returns	**String** or **Boolean** or **None**
	Description	If the window is a channel window, and no argument is passed, returns the channel topic as a **string**; otherwise, returns **None**. If a **string** is passed as the argument, the channel's topic will be set to the **string**; if the window is a channel window, this will return **True**, otherwise this will return **False**.
type	**Arguments**	None
	Returns	**String** ("server", "private", "channel", or "unknown")
	Description	Returns "channel" if the channel is a channel window, "private" if the window is a private chat window, "server" if the window is a server window, or "unknown" if the window type is not known.

uptime	**Arguments**	None
	Returns	**`Integer`**
	Description	Returns the number of seconds the window has existed.
users	**Arguments**	None or **`String`**
	Returns	**`List`** of **`Strings`**
	Description	If the window is a channel window and called with no arguments, returns a list of all nicknames (with hostmasks, if known) in that channel. Each nickname will have channel status prefixes (such as @ for channel operators). If called with a **`string`** (a channel status: 'owner', 'admin', 'operator', 'halfop', 'protected', 'voiced', or 'normal'), returns a list of user nicknames with that status; each nickname does *not* contain a status prefix or hostmask. If the window is not a channel window, does not have any users, or does not contain any users with the requested status, returns an empty **`list`**.

Plugin Console Class

The plugin **`Console`** class allows plugins to display data, without having to write the data to an existing MERK subwindow. Using the **`print()`** and **`prints()`** methods work exactly like with the MERK Window class, and displays time-stamped strings in the console's text display.

When the console subwindow is created, it is initially hidden, and cannot be seen by users. Use the **`show()`** method to show the window to users, and **`hide()`** to hide it again. The console subwindow is created the first time a plugin calls the **`console()`** method, and exists until the **`Console`** **`close()`** method is called; each time the **`console()`** method is called after the first call, a **`Console`** class representing the same window is returned. The console subwindow can also be shown or hidden from the plugin right click context menu in the plugin manager.

Use the **`html()`** method to display raw, non-timestamped HTML in the console's text display.

Methods

`clear`	**Arguments**	None
	Returns	Nothing
	Description	Clears the text display in the console subwindow.
`close`	**Arguments**	None
	Returns	None
	Description	Closes the console subwindow. Any data displayed in the console is lost, and the next time **`console()`** is called, the console will be re-created.
`dump`	**Arguments**	None
	Returns	**`List`** of **`Lists`** of **`Strings`**
	Description	Returns the contents of the console display in a **`list`** of **`lists`** of **`strings`**, with each entry in the following format: 0. **Timestamp**. When the text was printed to the console. 1. **The text**. What was printed to the console.
`hide`	**Arguments**	None
	Returns	Nothing
	Description	Hides the console subwindow.

html	**Arguments**	**String** (the HTML to print)
	Returns	Nothing
	Description	Prints raw HTML to the console. If **String** contains IRC colors or formatting, it will be rendered as HTML. HTML added this way will *not* be returned by the **dump()** method.
max	**Arguments**	None
	Returns	Nothing
	Description	Maximizes the console subwindow.
maximized	**Arguments**	None
	Returns	**Boolean**
	Description	Determines whether the console subwindow is maximized or not. Returns **True** if the window is maximized, and **False** if not.
min	**Arguments**	None
	Returns	Nothing
	Description	Minimizes the console subwindow.
minimized	**Arguments**	None
	Returns	**Boolean**
	Description	Determines whether the console subwindow is minimized or not. Returns **True** if the window is minimized, and **False** if not.
move	**Arguments**	**Integer** (the X-value of the new position), **Integer** (the Y-value of the new position)
	Returns	**True** if the move was successful, **False** if not
	Description	Moves the console subwindow to the specified coordinates. If the move would put the window in a location where it could not be interacted with, this method will fail and return **False**.
position	**Arguments**	None
	Returns	**List** of **Integers** (X value, Y value)
	Description	Returns the position of the console subwindow in a list in the following format: 0. **X value**. The horizontal position of the window. 1. **Y value**. The vertical position of the window. If the window's position cannot be determined, both values in the **list** will be set to **0**.

print	**Arguments**	**String** (the text to print)
	Returns	Nothing
	Description	Prints text to the console subwindow's text display, with a timestamp. HTML may be used. If IRC color display is turned on, IRC colors and formatting will be displayed.
prints	**Arguments**	**String** (the text to print)
	Returns	Nothing
	Description	Prints a system-styled message to the console subwindow's text display, with a timestamp. HTML may be used. If IRC color display is turned on, IRC colors and formatting will be displayed.
restore	**Arguments**	None
	Returns	Nothing
	Description	"Restores" the console subwindow; if the window has been minimized or maximized, this window returns to a normal state.
show	**Arguments**	None
	Returns	Nothing
	Description	If the console subwindow is hidden, the window will be shown. The console subwindow is always hidden when it is first created.
size	**Arguments**	**Integer** (width) – Optional, see description **Integer** (height) – Optional, see description
	Returns	Nothing or **List** of **Integers**
	Description	Resizes the console. If called without arguments or only only one argument, returns a **List** of **Integers**, with the width of the console being the first entry, and the height being the second.
title	**Arguments**	None or **String**
	Returns	**String** or Nothing
	Description	Sets the console subwindow's title to **string**; if **string** is omitted, the window's current title is returned as a **string**. Unless changed, the window's title will always be the name of the plugin's **NAME** and **VERSION**, separated by a space.

Example Plugins

All of these examples, and ZIP packages for the plugins, can be found here.

Away Notifications

In this example, we're going to write a plugin that notifies every chat that MERK is in whenever it sets its status to "away" or "back". To accomplish this, we're going to use the **`ctick()`** event method, the **`is_away()`** and **`windows()`** built-in methods, and some MERK Window classes.

Our plugin will use **`ctick()`** to check every client for a status change once per second. First, we declare a class variable we'll need in **`init()`**: a list to store notified clients. If a client's away status has changed to either "away" or "back", our plugin will send a CTCP action message to every open chat window telling them about our status change. To prevent the plugin from spamming a status once per second, the plugin will store which Twisted IRC clients have been notified in our **`list`** class variable: when the client goes away, the Twisted IRC object is stored, and when the client comes "back", the Twisted IRC object is removed from storage. This plugin should work with any number of server connections, and will only notify the connections that have had their status changed. This plugin also doesn't do any error checking; it calls the MERK Window method **`.describe()`** on server windows, which doesn't work, but it doesn't notify the user and no error will be displayed.

```
from merk import Plugin

class AwayNotify(Plugin):

    NAME = "Away Notify"
    AUTHOR = "Dan Hetrick"
    VERSION = "1.0"
    SOURCE = "https://github.com/nutjob-laboratories/merk"

    def init(self):
        self.notified = []

    def ctick(self,**args):
        uptime = args["uptime"]

        for client in self.clients():
            if self.is_away(client):
                if not client in self.notified:
                    self.notified.append(client)
                    for window in self.windows(client):
                        window.describe("is now away")
            else:
                if client in self.notified:
                    self.notified.remove(client)
                    for window in self.windows(client):
                        window.describe("is now back")
```

Mention Notification

This plugin will give the user a "visual" notification if their name is mentioned in a chat: the subwindow that their nickname was mentioned in will have its title changed to show that their nickname was mentioned. When the window is clicked on (or activated), the title will revert back to normal.

```
from merk import Plugin

class MentionNotify(Plugin):

    NAME = "Mention Notification"
    AUTHOR = "Dan Hetrick"
    VERSION = "1.0"
    SOURCE = "https://github.com/nutjob-laboratories/merk"

    def init(self):
        self.notified = []

    def message(self,**args):
        window = args["window"]
        channel = args["channel"]
        user = args["user"]
        message = args["message"]
        client = args["client"]

        if window!=None:
            if window.type()!="server":
                if client.nickname in message:
                    if not channel in self.notified:
                        if not window.active():
                            self.notified.append(channel)
                            window.title(window.name()+" - Mentioned")

    def activate(self,**args):
        window = args["window"]
        if window.name() in self.notified:
            self.notified.remove(window.name())
            window.title(window.name())
```

Unread Messages Notifications

This plugin changes the title of windows that have unread messages. Server windows are skipped.

```
from merk import Plugin

class MentionNotify(Plugin):

    NAME = "Unread Notification"
    AUTHOR = "Dan Hetrick"
    VERSION = "1.0"
    SOURCE = "https://github.com/nutjob-laboratories/merk"

    def message(self,**args):
        window = args["window"]
        channel = args["channel"]
        user = args["user"]
        message = args["message"]
        client = args["client"]

        if window!=None:
            if window.type()!="server":
                if window.title()==window.name():
                    if not window.active():
                        window.title(window.name()+" - Unread messages")

    def activate(self,**args):
        window = args["window"]
        if window.title()!=window.name():
            window.title(window.name())
```

Mention Log

This plugin writes every message sent to MERK that mentions the client's nickname to the plugin's console.

```
from merk import Plugin

class MentionLogPlugin(Plugin):

    NAME = "Mention Log"
    AUTHOR = "Dan Hetrick"
    VERSION = "1.0"
    SOURCE = "https://github.com/nutjob-laboratories/merk"

    def init(self):
        c = self.console()
        c.html("<h1>Mention Log</h1>")

    def message(self,**args):
        window = args["window"]
        client = args["client"]
        channel = args["channel"]
        user = args["user"]
        nickname = args["nickname"]
        hostmask = args["hostmask"]
        message = args["message"]

        c = self.console()
        if client.nickname in message:
            entry = f"{client.hostname} {channel} {nickname}: {message}"
            c.print(entry)
```

Notepad

This plugin is fairly complex, and involves a script. This plugin will implement a "notebook" for users, allowing them to print messages to the plugin's console.

First, we're going to write our script. This script will use the **`/call`** command to execute a **`Plugin`** method. Save the following script to a file named **`note.merk`**, and save it in your scripts folder:

```
usage + Usage: /note TEXT
/call save $_0
```

The **`$_0`** built-in alias contains all the arguments passed to a script, and the **`+`** argument to **`usage`** tells the script to accept one or more arguments. If there are no arguments passed to the script, **`usage`** makes sure that the script will display usage text and exit.

Now that we have our script, we can create the plugin that uses it. Here, we'll create the **`Plugin`** method that **`/call`** uses, and create a macro to call our script in the plugin's **`init()`** event. We'll use the **`uninstall()`** event to remove the macro and delete our script when the plugin is uninstalled, and code to remove (and add) the macro when the plugin is paused or unpaused.

```
from merk import Plugin
import os

class Notepad(Plugin):

    NAME = "Notepad"
    AUTHOR = "Dan Hetrick"
    VERSION = "1.0"
    SOURCE = "https://github.com/nutjob-laboratories/merk"

    def init(self):
        self.macro("note","note.merk","/note MESSAGE","Writes a note")

    def pause(self):
        self.unmacro("note")

    def unpause(self):
        self.macro("note","note.merk","/note MESSAGE","Writes a note")

    def uninstall(self):
        self.unmacro("note")
        script = self.find("note","merk")
        if script!=None: os.remove(script)

    def save(self,window,arguments):
        console = self.console()
        console.print(" ".join(arguments))
```

If both the plugin and the script are packaged in a ZIP file, both files will be installed (in the proper place) when the ZIP is installed with the plugin manager or via drag-and-drop.

External IP Address

This plugin also involves a script. This plugin will implement a use of the **`urllib`** library to fetch the user's external IP address from a website, and print it to the current window. Since the **`urllib`** library is part of Python's standard library, this plugin will work on both the Python and PyInstaller versions of MERK. First, we're going to write our script. This script will use the **`/call`** command to execute a **`Plugin`** method. Save the following script to a file named **`ip.merk`**, and save it in your scripts folder:

```
usage 0 Usage: /ip
/call get_ip
```

This script doesn't take any arguments, and just uses **`/call`** to call our plugin method. Now that we have our script, we can create the plugin that uses it. Here, we'll create the **`Plugin`** method that **`/call`** uses, and create a macro to call our script in the plugin's **`init()`** event. Finally, we'll use the **`uninstall()`** event to remove our macro and delete our script if the plugin is uninstalled, and handle pausing and unpausing the plugin.

```
from merk import Plugin
import urllib.request
import urllib.error
import os

class MyIPPlugin(Plugin):

    NAME = "My IP"
    AUTHOR = "Dan Hetrick"
    VERSION = "1.0"
    SOURCE = "https://github.com/nutjob-laboratories/merk"

    def init(self):
        self.macro("ip","ip.merk","/ip","Gets the external IP address")

    def pause(self):
        self.unmacro("ip")

    def unpause(self):
        self.macro("ip","ip.merk","/ip","Gets the external IP address")

    def uninstall(self):
        self.unmacro("ip")
        script = self.find("ip","merk")
        if script!=None: os.remove(script)

    def get_ip(self,window,arguments):
        try:
            with urllib.request.urlopen("https://v4.ident.me") as response:
                external_ip = response.read().decode('utf8')
                window.print(external_ip)
        except urllib.error.URLError as e:
            window.print(f"Error retrieving external IP: {e.reason}")
        except Exception as e:
            window.print(f"An unexpected error occurred: {e}")
```

If both the plugin and the script are packaged in a ZIP file, both files will be installed (in the proper place) when the ZIP is installed with the plugin manager or via drag-and-drop.

Threads with QThread

This plugin is an example of two things: using classes in a plugin that do *not* inherit from **`Plugin`**, and using the **`QThread`** object in PyQt5 to write a callable function that normally would be blocking. Here, when our callable method is executed, it creates a thread where the **`time`** library is used to wait for 10 seconds, and when the thread ends, a greeting is sent to the window where the callable method was, well, called. If the window is a channel or private chat window, the greeting is sent as a message to the channel or private chat, and if the window is a server window, it is simply printed to the window:

```
from merk import Plugin
from PyQt5.QtCore import *
import time

class DelayThread(QThread):
    threadEnd = pyqtSignal()
    def __init__(self,wait):
        super(DelayThread, self).__init__()
        self.time = wait

    def run(self):
        time.sleep(self.time)
        self.threadEnd.emit()

class SlowHello(Plugin):

    NAME = "Slow Hello"
    AUTHOR = "Dan Hetrick"
    VERSION = "1.0"
    SOURCE = "https://github.com/nutjob-laboratories/merk"

    def say_hello(self):
        if self.window.type()=="channel" or self.window.type()=="private":
            self.window.say("Hello!")
        else:
            self.window.print("Hello!")

    def slow_hello(self,window,arguments):
        self.window = window
        self.thread = DelayThread(10)
        self.thread.threadEnd.connect(self.say_hello)
        self.thread.start()
```

Normally, calling **`time.sleep()`** would block MERK from doing anything while “sleeping”. However, by creating a separate thread, we can run **`time.sleep()`** in that thread and allow MERK to continue running and updating without issues.

Since PyQt5 built into the PyInstaller version of MERK, this plugin can run in any version of MERK without installing any additional libraries.

Rainbow Messages

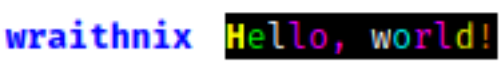
This creates a command that sends a rainbow colored message to the current chat. It consists of a script and a plugin combined. First, let's create a script, and save it to the **`scripts`** directory under the name **`rainbow.merk`**:

```
restrict channel private
usage + Usage: /rainbow TEXT…
/call rainbow $_0
```

With that out of the way, let's write our plugin! This creates a **`/call`**able method that takes a message and colors it with random colors, using MERK "markup", before sending it to the chat. We create a macro named **`/rainbow`** to call this method, and handle pausing, unpausing, and uninstalling the plugin.

```
from merk import Plugin
import random, os

class RainbowChat(Plugin):

    NAME = "Rainbow"
    AUTHOR = "Dan Hetrick"
    VERSION = "1.0"
    SOURCE = "https://github.com/nutjob-laboratories/merk"

    def init(self):
        self.macro("rainbow","rainbow.merk","/rainbow TEXT...","Says some rainbow text")

    def unload(self):
        self.unmacro("rainbow")

    def pause(self):
        self.unmacro("rainbow")

    def unpause(self):
        self.macro("rainbow","rainbow.merk","/rainbow TEXT...","Says some rainbow text")

    def uninstall(self):
        self.unmacro("rainbow")
        script = self.find("rainbow","merk")
        if script!=None: os.remove(script)

    def rainbow(self,window,arguments):
        output = []
        colors = [0,9,11,4,13,8,7]
        msg = ' '.join(arguments)
        for letter in msg:
            output.append(f"<{random.choice(colors)},1{letter}>")
        if len(output)>0:
            chat ='**'+''.join(output)+'**'
            window.say(chat)
```

Once installed, if you were to enter **`/rainbow Hello, world!`** into the text entry widget in any chat window, you'd see something like this:



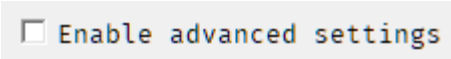
Advanced Settings

The last section in the settings dialog shows a number of settings that can be used to fundamentally change how MERK works. ***WARNING:*** Changing these settings may break your installation of MERK, break any existing scripts, or fill up your hard drive. If this occurs, please see ***Resetting MERK to Default Settings***.

Changing these settings is not recommended, but may be desired.

Options

To change any of these settings, advanced settings must be enabled by clicking this checkbox:



Unchecking this checkbox will reset all the advanced settings to the value stored in the configuration file, and no settings will be saved. This *must* be enabled when clicking the "Apply" or "Apply & Restart" buttons for any changes to be saved. When changing any any advanced setting, ***restarting MERK is recommended***.

- **Connection heartbeat.** This sets how often MERK "pings" the IRC server to keep the network connection active. The default is once every 120 seconds.
- **Max message length**. IRC has a hard limit of 512 characters for chat messages, and this must include the user's nickname and the appropriate chat command. 400 characters is the limit that MERK uses to determine if a chat message needs to be split into multiple messages.
- **Flood protection for long messages**. Chat messages that have been split up for exceeding the maximum message length will be sent once per second, to avoid triggering a server's flood protection. Turned on by default.
- **Show server pings in server windows**. MERK sends "pings" to the IRC server to keep the network connection active. Checking this box will display whenever the client receives a "ping" reply from the server.
- **Prevent illegal nickname input**. This option will prevent nicknames that do not follow the IRC RFCs from being entered in the GUI or from commands. Turned on by default.
- **Prevent illegal channel name input**. This option will prevent channel names that do not follow the IRC RFCs from being entered in the GUI or from commands. Turned on by default.
- **Search install directory for files**. Checking this option will search MERK's installation directory when using **`/find`** or other file-related commands.
- **Save all system messages to log**. Checking this option will save nearly all messages displayed if logging is turned on. This will drastically increase the size of saved logs.
- **Write all network input and output to STDOUT**. This will show all IRC network traffic in STDOUT, which is normally printed to the console MERK is being ran from. This will not display anything if MERK is being executed with the PyInstaller executable (without using additional software to view STDOUT).
- **Write all script errors to STDOUT**. This will write all script errors triggered during script execution to STDOUT. This will not display anything if MERK is being executed with the PyInstaller executable (without using additional software to view STDOUT).
- **Write all network input and output to a file in the user's settings directory**. This will write all network input and output to a file or files in the **`settings`** directory. Each IRC connection will have its input and output written to a file named **`SERVER-PORT.txt`**. So, a connection to **`irc.libera.chat`** on port **`6697`** would have its input and output written to **`irc.libera.chat-6697.txt`**. **WARNING!** This will write a *lot* of data to your hard drive.

Settings Editable by `/config`

The following is a list of all settings editable with the **`/config`** command, along with their default values. Settings are in the format "**`setting: value`**". Values in quotation marks are strings, values set to **`false`** or **`true`** are boolean, and numerical values are integers. Values that appear in italics are generated when MERK is ran for the first time, and vary depending on the version of the application.

Several settings require a string value that must be specific strings:

- **`qt_window_style`**. *This must be set to a valid value string, depending on your installation of Qt, and will vary. Trying to set it to an invalid value will list valid values for your platform. Not all Qt styles work with MERK, and will be unselectable by any means (namely, the “cleanlooks” and “gtk2” styles). The default value is “Windows”.*
- **`windowbar_justify`**. *Must be "left", "right", or "center". The default is “center”.*
- **`menubar_justify`**. *Must be "left", "right", or "center". The default is “left”.*
- **`default_spellcheck_language`**. *Must be "en", "fr", "es", "de", "pt", "it", "nl", or "ru" (for English, French, Spanish, German, Portuguese, Italian, Norwegian, and Russian, respectively). The default is “en” (English).*
- **All settings ending in `_style` (except for `qt_window_style`).** *Must be strings containing only "bold" or "italic", separated by spaces.*
- **`subwindow_order`**. *Must be "creation", "stacking", or "activation". The default is “creation”.*
- **All settings ending in `_color`**. *Must be a valid Qt color descriptor (such as an SVG color keyword name), or an HTML hexadecimal color value (like "#FFFFFF" for white or "#000000" for black).*
- **`sound_notification_file`**. *Must be a string containing a valid path for a WAV file. The default value is a file located in MERK's resource file.*
- **`timestamp_format`**. *Must be a string containing a valid **`strftime`** format. The default is “%H:%M:%S”.*
- **`emoji_shortcode_language`**. *Must be "alias", "en", "pt", "it", "fr", "de", "fa", "id", "zh", "ja", "ko", "ru", "ar", or "tr". For more information, please see the Emoji library repository. The default is “alias”.*
- **`plugin_manager_console_icon`**. *Should be an emoji or ASCIImoji shortcode, although this is not required. This will be used in a plugin’s entry in the plugin manager to show that a plugin’s console is open. The default is “:left_speech_bubble:”.*
- **`mdi_workspace_background`**. *Must be a string containing a valid path for an image filetype that QPixmap can open; this includes bitmaps, JPEGs, GIFs, PNGs, and many other formats. The default is an empty string (“”); to reset this setting back to default, pass **`*`** as the value to the **`/config`** command.*
- **`mdi_background_image_style`**. *Must be “scale”, “center”, or “tile”. This sets whether the MDI workspace background image is scaled/stretched to the size of the workspace, centered in the workspace, or tiled across the workspace, respectively. The default is “scale”.*
- ***`subwindow_background`***. *Must be a string containing a valid path for an image filetype that QPixmap can open; this includes bitmaps, JPEGs, GIFs, PNGs, and many other formats. The default is an empty string (“”); to reset this setting back to default, pass **`*`** as the value to the **`/config`** command.*
- **`bad_nickname_fallback`**. *This is the nickname used when the “erroneous nickname” error is thrown by the server during registration. It should be less than 9 characters, as 9 characters is the maximum nickname length for some networks, like EFnet. The setting will then be padded with random numbers to get the nickname up to 9 characters. If this setting is longer than or equal to 9 characters, the default of “Guest” will be used instead. This setting should be fairly short, so that nickname collisions are avoided. Other than the character limit, this must follow all the other rules of IRC nicknames: it cannot start with a number or a hyphen, cannot contain any control codes, and cannot contain **`!`**, **`@`**, **`.`**, **`:`**, **`,`**, **`/`**, **`\`**, **`*`**, **`?`**, **`+`**, **`=`**, **`$`**, **`%`**, **`<`**, **`>`**, **`&`**, **`“`**, or **`‘`**.*
- ***`windowbar_sorting`***. *Must be “creation”, “reverse”, “alpha”, or “ralpha”. This sets the order that entries in the windowbar are displayed. “creation” (the default) displays subwindows in the order they were created, and “reverse” shows them in the reverse of this order. “alpha” shows the windows in alphabetical order, and “ralpha” shows them in reverse alphabetical order.*

Settings and Default Values

- alias_interpolation_symbol: "$"
- always_scroll_to_bottom: false
- always_show_current_first_in_windowbar: true
- app_interaction_cancels_autoaway: false
- apply_syntax_highlighting_to_input_widget: true
- ask_before_disconnect: true
- ask_before_exit: false
- ask_before_reconnect: false
- auto_join_on_invite: true
- autoaway: false
- autoaway_time: 3600
- autocomplete_aliases: true
- autocomplete_channels: true
- autocomplete_commands: true
- autocomplete_filenames: true
- autocomplete_macros: true
- autocomplete_methods: true
- autocomplete_nicks: true
- autocomplete_servers: true
- autocomplete_settings: true
- autocomplete_shortcodes: true
- autocomplete_shortcodes_in_away_message_widget: true
- autocomplete_shortcodes_in_quit_message_widget: true
- autocomplete_user_settings: true
- automatic_reconnection_timer: 30
- automatically_refresh_channel_list: false
- automatically_rerender_chat_for_nick_highlighting: true
- away_message: "Away at $_DATE $_TIME"
- bad_nickname_fallback: "Guest"
- channel_list_staleness_in_minutes: 120
- channel_mode_right_click_menu: true
- chat_message_max_length: 400
- clear_plugins_from_memory_on_reload: true
- click_systray_icon_to_minimize_to_tray: true
- close_editor_on_plugin_uninstall: true
- closing_main_window_minimizes_to_tray: true
- closing_server_window_disconnects_from_server: false
- command_history_length: 20
- convert_urls_to_links: true
- create_window_for_incoming_private_messages: true

- create_window_for_incoming_private_notices: false
- create_window_for_outgoing_private_messages: false
- cursor_blink: true
- cursor_blink_rate: 1060
- dark_mode: false
- default_spellcheck_language: "en"
- default_subwindow_height: 480
- default_subwindow_width: 640
- delay_automatic_reconnection: false
- delete_script_aliases_on_end: true
- disconnect_on_sasl_failure: true
- display_active_subwindow_in_title: true
- display_all_server_errors: false
- display_dates_in_logs: true
- display_error_message_for_restrict_and_only_violation: true
- display_full_user_info_in_mode_messages: true
- display_irc_colors: true
- display_irc_colors_in_topics: true
- display_messagebox_on_plugin_error: true
- display_nick_on_server_windows: false
- display_server_errors_in_current_window: true
- display_server_motd_as_raw_text: false
- display_server_pings_in_server_window: false
- display_timestamp: true
- do_intermittent_log_saves: true
- do_not_allow_select_on_userlist: false
- do_not_apply_styles_to_text: false
- do_not_apply_text_style_to_input_widget: false
- do_not_apply_text_style_to_userlist: false
- do_not_create_private_chat_windows_for_ignored_users: true
- do_not_pad_nickname_in_chat_display: false
- do_not_reply_to_ctcp_source: false
- do_not_reply_to_ctcp_version: false
- do_not_show_application_name_in_title: false
- do_not_show_environment_in_ctcp_version: false
- do_not_show_server_name_in_application_title: false
- doubleclick_nick_display_to_change_nick: true
- doubleclick_to_restore_from_systray: true
- doubleclick_userlist_to_open_private_chat: true
- editor_prompt_save_on_close: true
- editor_syntax_highlighting: true

- editor_word_wrap: false
- elide_away_message_in_userlist_context_menu: true
- elide_hostmask_in_userlist_context_menu: true
- elide_long_nicknames_in_chat_display: true
- emoji_shortcode_language: "alias"
- enable_aliases: true
- enable_application_drag_and_drop: true
- enable_asciimoji_shortcodes: true
- enable_autocomplete: true
- enable_browser_command: true
- enable_built_in_aliases: true
- enable_call_command: true
- enable_command_history: true
- enable_config_command: true
- enable_delay_command: true
- enable_emoji_shortcodes: true
- enable_goto_command: true
- enable_hotkeys: true
- enable_if_command: true
- enable_ignore: true
- enable_insert_command: true
- enable_irc_color_markup: true
- enable_markdown_markup: true
- enable_plugin_action_event: true
- enable_plugin_activate_event: true
- enable_plugin_away_event: true
- enable_plugin_back_event: true
- enable_plugin_close_event: true
- enable_plugin_connected_event: true
- enable_plugin_connecting_event: true
- enable_plugin_ctick_event: true
- enable_plugin_disconnect_event: true
- enable_plugin_error_event: true
- enable_plugin_editor: true
- enable_plugin_init_event: true
- enable_plugin_invite_event: true
- enable_plugin_ison_event: true
- enable_plugin_isupport_event: true
- enable_plugin_join_event: true
- enable_plugin_joined_event: true
- enable_plugin_kick_event: true

- enable_plugin_kicked_event: true
- enable_plugin_left_event: true
- enable_plugin_line_in_event: true
- enable_plugin_line_out_event: true
- enable_plugin_lost_event: true
- enable_plugin_me_event: true
- enable_plugin_message_event: true
- enable_plugin_mode_event: true
- enable_plugin_motd_event: true
- enable_plugin_nick_event: true
- enable_plugin_notice_event: true
- enable_plugin_part_event: true
- enable_plugin_pause_event: true
- enable_plugin_ping_event: true
- enable_plugin_quit_event: true
- enable_plugin_rename_event: true
- enable_plugin_server_event: true
- enable_plugin_subwindow_event: true
- enable_plugin_tick_event: true
- enable_plugin_topic_event: true
- enable_plugin_uninstall_event: true
- enable_plugin_unload_event: true
- enable_plugin_unmode_event: true
- enable_plugin_unpause_event: true
- enable_plugin_uptime_event: true
- enable_plugins: true
- enable_read_and_write_command: true
- enable_scripting: true
- enable_spellcheck: true
- enable_style_editor: true
- enable_topic_editor: true
- enable_user_command: true
- enable_userlist_context_menu: true
- enable_wait_command: true
- escape_html_in_print_and_prints_messages: false
- examine_topic_in_channel_list_search: true
- execute_channel_scripts: true
- execute_global_script: true
- execute_hotkey_as_command: true
- execute_init_event_on_plugin_reload: true
- fetch_hostmask_frequency: 5

- flood_protection_for_sending_long_messages: true
- force_all_windows_to_use_default_style: false
- force_monospace_text_rendering: false
- get_hostmasks_on_channel_join: true
- global_script_filename: "global.merk"
- halt_script_execution_on_error: true
- hide_horizontal_scrollbar_on_userlists: true
- hide_logo_on_initial_connection_dialog: false
- hide_server_windows_when_registration_completes: false
- hide_windowbar_if_empty: true
- highlight_all_nicknames_in_input_widget: false
- highlight_current_line_in_editor: false
- highlight_nicknames_in_chat: true
- import_scripts_in_plugin_packages: true
- input_widget_cursor_width: 1
- interface_button_icon_size: 18
- interface_button_size: 18
- intermittent_log_save_interval: 1800000
- interpolate_aliases_into_away_message: true
- interpolate_aliases_into_quit_message: true
- interpolate_aliases_into_user_input: true
- issue_command_symbol: "/"
- load_channel_logs: true
- load_private_logs: true
- log_channel_joins: true
- log_channel_nickname_changes: true
- log_channel_notice: true
- log_channel_parts: true
- log_channel_quits: true
- log_channel_topics: true
- main_menu_help_name: "Help"
- main_menu_irc_name: "IRC"
- main_menu_settings_name: "Settings"
- main_menu_tools_name: "Tools"
- main_menu_windows_name: "Windows"
- main_window_always_on_top: false
- managers_always_on_top: true
- mark_end_of_loaded_log: true
- maximize_app_on_startup: false
- maximize_subwindows_on_creation: false
- maximum_font_size_for_settings_dialog: 12

- maximum_insert_file_depth: 10
- maximum_loaded_log_size: 500
- minimum_nick_length_for_highlighting: 0
- mdi_background_image_style: "scale"
- mdi_workspace_background: *blank string*
- menubar_bold_on_hover: true
- menubar_can_float: false
- menubar_docked_at_top: true
- menubar_justify: "left"
- minimize_to_system_tray: false
- nickname_pad_length: 15
- nick_attempts_during_registration: 3
- notify_on_lost_or_failed_connection: true
- notify_on_repeated_failed_reconnections: true
- overwrite_files_on_plugin_import: false
- plain_user_lists: false
- plugin_manager_console_icon: ":left_speech_bubble:"
- prevent_illegal_channel_input: true
- prevent_illegal_nickname_input: true
- prevent_sending_bad_commands_as_chat: true
- print_script_errors_to_stdout: false
- prompt_for_away_message: false
- prompt_for_file_on_calling_script_with_no_arguments: false
- prompt_on_failed_connection: true
- python_editor_auto_indent: true
- python_editor_show_whitespace: false
- qt_window_style: "Windows"
- quit_message: "MERK $_VERSION"
- reject_all_channel_notices: false
- reload_plugins_after_uninstall: true
- reload_plugins_on_editor_close: true
- request_channel_list_on_connection: true
- require_exact_argument_count_for_usage: true
- rubberband_subwindow_move: false
- rubberband_subwindow_resize: false
- save_channel_logs: true
- save_connection_history: true
- save_private_logs: true
- save_main_window_location: false
- script_thread_quit_timeout: 1000
- scroll_chat_to_bottom_on_resize: true

- search_for_all_terms_in_channel_list_search: true
- search_install_directory_for_files: false
- settings_dialog_font_size: 10
- show_app_full_screen: false
- show_away_and_back_messages: true
- show_away_status_in_nick_display: true
- show_away_status_in_userlists: true
- show_channel_join_messages: true
- show_channel_list_button_on_server_windows: true
- show_channel_list_entry_in_windows_menu: true
- show_channel_list_in_systray_menu: true
- show_channel_mode_change_messages: true
- show_channel_name_and_modes: true
- show_channel_name_in_subwindow_title: true
- show_channel_nick_messages: true
- show_channel_part_messages: true
- show_channel_quit_messages: true
- show_channel_topic_bar: true
- show_channel_topic_in_title: false
- show_channel_topic_in_tooltip: true
- show_channel_topic_in_window_title: false
- show_channel_topic_messages: true
- show_channel_uptime: true
- show_chat_context_menu_options: true
- show_connection_dialog_on_startup: true
- show_connection_uptime: true
- show_connection_script_in_windows_menu: true
- show_connections_in_systray_menu: true
- show_directories_in_systray_menu: true
- show_hidden_channel_windows_in_windowbar: true
- show_hidden_private_windows_in_windowbar: true
- show_hidden_server_windows_in_windowbar: true
- show_ignore_status_in_userlists: true
- show_ison_response_in_current_window: false
- show_input_menu: true
- show_line_numbers_on_connection_dialog: false
- show_links_in_systray_menu: true
- show_links_to_known_irc_networks: true
- show_list_refresh_button_on_server_windows: false
- show_long_message_indicator: true
- show_lusers_response_in_current_window: false

- show_menubar_context_menu: true
- show_network_logs_in_systray_menu: true
- show_network_logs_in_windows_menu: true
- show_plugin_consoles_on_creation: true
- show_script_execution_errors: true
- show_server_information_in_windows_menu: true
- show_server_window_toolbar: true
- show_settings_in_systray_menu: true
- show_spellcheck_settings_in_menus: true
- show_status_bar_on_chat_windows: true
- show_status_bar_on_editor_windows: true
- show_status_bar_on_list_windows: true
- show_status_bar_on_server_windows: false
- show_systray_icon: true
- show_systray_menu: true
- show_tips_at_startup: true
- show_user_colors_in_userlists: true
- show_user_count_display: true
- show_user_info_on_chat_windows: true
- show_userlist_on_left: false
- show_userlists: true
- show_windowbar: true
- show_windowbar_context_menu: true
- simplified_dialogs: false
- sound_notification_disconnect: true
- sound_notification_file: ":/sound-notification.wav"
- sound_notification_invite: true
- sound_notification_kick: true
- sound_notification_mode: true
- sound_notification_nickname: true
- sound_notification_notice: true
- sound_notification_private: true
- sound_notifications: false
- spellcheck_in_bold: false
- spellcheck_in_color: false
- spellcheck_in_italics: false
- spellcheck_in_strikeout: false
- spellcheck_underline_color: "#FF0000"
- spellchecker_distance: 1
- subwindow_background: *blank string*
- subwindow_resize_edge_margin: 5

- subwindow_order: "creation"
- subwindow_snapping: true
- subwindow_snap_drag_distance_trigger: 5
- subwindow_snap_distance: 20
- syntax_alias_color: "Red"
- syntax_alias_style: "bold italic"
- syntax_background_color: "white"
- syntax_channel_color: "darkRed"
- syntax_channel_style: "bold"
- syntax_command_color: "darkBlue"
- syntax_command_style: "bold"
- syntax_comment_color: "Magenta"
- syntax_comment_style: "bold italic"
- syntax_foreground_color: "black"
- syntax_nickname_color: "darkRed"
- syntax_nickname_style: "bold"
- syntax_operator_color: "blue"
- syntax_operator_style: "bold"
- syntax_script_only_color: "darkGreen"
- syntax_script_only_style: "bold"
- syntax_shortcode_color: "Magenta"
- syntax_shortcode_style: "bold italic"
- system_message_prefix: "♦"
- systray_notification_channel: true
- systray_notification_disconnect: true
- systray_notification_invite: true
- systray_notification_kick: true
- systray_notification_list: true
- systray_notification_mode: true
- systray_notification_nickname: true
- systray_notification_notice: true
- systray_notification_private: true
- systray_notification_speed: 500
- systray_notifications: true
- timestamp_24_hour: true
- timestamp_format: "%H:%M:%S"
- timestamp_show_seconds: true
- twisted_irc_client_heartbeat: 120
- typing_input_cancels_autoaway: true
- underline_self_in_userlists: false
- unknown_network_name: "Unknown"

- use_menubar: true
- userlist_width_in_characters: 15
- use_style_color_for_self_in_userlists: false
- windobar_include_readme: false
- window_interaction_cancels_autoaway: false
- windowbar_bold_active_window: true
- windowbar_bold_on_hover: true
- windowbar_can_float: false
- windowbar_channel_topic_in_tooltip: false
- windowbar_doubleclick_to_maximize: true
- windowbar_entry_context_menu: true
- windowbar_include_channel_lists: false
- windowbar_include_channels: true
- windowbar_include_editors: true
- windowbar_include_log_manager: false
- windowbar_include_private: true
- windowbar_include_servers: false
- windowbar_justify: "center"
- windowbar_on_top: true
- windowbar_show_connecting_server_windows_in_italics: true
- windowbar_show_icons: false
- windowbar_show_unread_mentions: false
- windowbar_show_unread_messages: true
- windowbar_sorting: "creation"
- windowbar_underline_active_window: true
- windowbar_unread_message_animation_length: 1000
- write_network_input_and_output_to_console: false
- write_network_input_and_output_to_file: false
- write_outgoing_private_messages_to_current_window: true
- write_private_messages_to_server_window: true